

Лекции по C++

2 семестр

Дмитрий Величко

Март 2026

Содержание

1	Встроенные типы данных, представление чисел в памяти, строки	13
1.1	Фундаментальные типы	13
1.1.1	Размеры типов не фиксированы стандартом	13
1.2	Представление целых чисел в памяти	13
1.2.1	Беззнаковые числа	13
1.2.2	Знаковые числа: дополнительный код (two's complement) . .	14
1.2.3	Порядок байтов (endianness)	14
1.3	Числа с плавающей точкой (IEEE 754)	14
1.4	Строки	15
1.4.1	C-строки (const char*)	15
1.4.2	std::string	15
1.4.3	std::string_view (C++17)	15
2	Преобразования типов: явные и неявные	16
2.1	Неявные преобразования (implicit conversions)	16
2.1.1	Целочисленные продвижения (integral promotion)	16
2.1.2	Стандартные арифметические преобразования	16
2.1.3	Опасность: знаковое и беззнаковое	16
2.1.4	Сужающие преобразования (narrowing)	16
2.1.5	Пользовательские неявные преобразования	17
2.2	Явные преобразования (casts)	17
3	Структуры и объединения	17
3.1	Структуры (struct)	17
3.1.1	Выравнивание и padding	18
3.2	Объединения (union)	18
3.2.1	Tagged union и std::variant	19
4	Указатели	19
4.1	Что такое указатель	19
4.2	Нулевой указатель	19
4.3	Указатели и const	19
4.4	Арифметика указателей	20
4.5	Указатель на void и на функцию	20

4.6	Висячие и невалидные указатели	20
5	Массивы	20
5.1	Встроенные (C-style) массивы	20
5.1.1	Распад массива в указатель (array decay)	21
5.2	Многомерные массивы	21
5.3	std::array (C++11)	21
5.4	std::vector — динамический массив	21
6	Устройство памяти, сегменты	22
6.1	Сегменты памяти процесса	22
6.2	Стек vs Куча	23
6.3	Время жизни и классы хранения	23
6.4	Переполнение стека (stack overflow)	23
7	Стек вызова функций	23
7.1	Кадр стека (stack frame)	23
7.1.1	Как растёт стек	24
7.2	Указатели стека	24
7.3	Соглашения о вызове (calling convention)	24
7.4	Раскрутка стека (stack unwinding)	25
7.5	Хвостовая рекурсия	25
8	Ссылки	25
8.1	Что такое ссылка	25
8.1.1	Ссылка vs указатель	25
8.2	Ссылки в параметрах функций	25
8.3	Висячие ссылки	26
8.4	Константные ссылки и продление жизни	26
8.5	Rvalue-ссылки (C++11)	26
9	Перегрузка функций	26
9.1	Что это	26
9.2	Что участвует в выборе	27
9.3	Разрешение перегрузки (overload resolution)	27
9.4	Перегрузка по const и ссылочности	27
9.5	Перегрузка vs переопределение vs сокрытие	27
9.6	Значения по умолчанию	28
10	Пространства имён (namespace)	28
10.1	Зачем нужны namespace	28
10.2	Использование имён из namespace	28
10.3	Вложенные namespace	28
10.4	Анонимный namespace	28
10.5	Псевдонимы namespace	29
10.6	Argument-Dependent Lookup (ADL)	29

11 Компиляция: этапы, ошибки	29
11.1 Этапы сборки программы на C++	29
11.2 Единица трансляции и модель компиляции	30
11.2.1 Защита от повторного включения	30
11.3 Объявление vs определение (ODR)	30
11.4 Виды ошибок	30
11.4.1 Типичная ошибка линковки	31
11.5 Полезные флаги компилятора	31
12 ООП: абстракция, инкапсуляция, наследование, полиморфизм	31
12.1 Четыре столпа ООП	31
12.2 Абстракция	31
12.3 Инкапсуляция	32
12.4 Наследование	32
12.4.1 Виды наследования	32
12.4.2 Множественное наследование и проблема ромба	33
12.5 Полиморфизм	33
12.5.1 Динамический полиморфизм (виртуальные функции)	33
12.5.2 Как это работает: vtable и vptr	33
12.5.3 override и final	34
12.5.4 Статический полиморфизм	34
13 Классы. Специальные методы	34
13.1 Специальные функции-члены	34
13.2 Конструкторы	34
13.2.1 Список инициализации членов	35
13.2.2 Делегирующие конструкторы	35
13.3 Деструктор	35
13.4 Правило трёх / пяти / нуля	35
13.5 = default и = delete	36
13.6 Статические члены	36
13.7 const-методы и mutable	36
13.8 friend	37
13.9 this	37
14 Перегрузка операторов	37
14.1 Зачем перегружать операторы	37
14.2 Способы перегрузки: метод vs свободная функция	37
14.3 Арифметические операторы и каноничная форма	38
14.4 Операторы сравнения и spaceship (C++20)	38
14.5 Оператор вывода <<	38
14.6 Оператор индексирования []	39
14.7 Оператор вызова () — функциональные объекты	39
14.8 Операторы инкремента	39
14.9 Операторы преобразования	39
14.10 Что нельзя перегружать	39
14.11 Умные указатели и операторы *, ->	40

15 Шаблоны функций и классов. Специализация	40
15.1 Зачем нужны шаблоны	40
15.2 Шаблоны функций	40
15.2.1 Вывод аргументов шаблона (template argument deduction) . .	40
15.3 Шаблоны классов	40
15.3.1 Class Template Argument Deduction (CTAD, C++17)	41
15.4 Параметры шаблона	41
15.5 Полная специализация	41
15.6 Частичная специализация	42
15.7 Двухфазная компиляция и typename/template	42
15.8 Где определять шаблоны	42
15.9 Переменные-шаблоны и constexpr	42
15.10 Концепты (C++20) — кратко	42
16 RAII. Умные указатели	43
16.1 Идиома RAII	43
16.2 Проблема сырых указателей	43
16.3 std::unique_ptr — единоличное владение	43
16.3.1 unique_ptr для массивов и кастомный делитер	44
16.4 std::shared_ptr — разделяемое владение	44
16.4.1 Накладные расходы	44
16.5 std::weak_ptr — слабая ссылка	44
16.5.1 Решение проблемы циклических ссылок	44
16.6 Сравнение умных указателей	45
16.7 Передача умных указателей в функции	45
17 Исключения в C++	45
17.1 Ключевое слово throw	45
17.1.1 Базовый синтаксис	45
17.1.2 Что можно бросать?	46
17.1.3 Что происходит при throw?	46
17.1.4 throw как выражение	46
17.2 Раскрутка стека (Stack Unwinding)	47
17.2.1 Определение	47
17.2.2 Пошаговая механика	47
17.2.3 Визуализация стека	48
17.2.4 Что уничтожается, а что нет	48
17.2.5 Частично сконструированные объекты	49
17.2.6 Раскрутка стека и массивы	49
17.2.7 Когда раскрутка НЕ происходит	50
17.3 try/catch	50
17.3.1 Базовый синтаксис	50
17.3.2 Множественные блоки catch	50
17.3.3 Ловля по значению vs по ссылке	51
17.3.4 Вложенные try/catch	51
17.4 Приведение типов в catch	52
17.4.1 Наследование в catch	52
17.4.2 Правильный порядок	53

17.5	Где встречаются исключения в стандартной библиотеке	53
17.6	Runtime-ошибки vs Исключения C++	53
17.6.1	Три вида runtime-ошибок на уровне ОС	54
17.6.2	Пути к <code>std::terminate()</code>	54
17.7	Копирование при броске исключения	55
17.7.1	Где хранится объект исключения?	55
17.7.2	Оптимизация: <code>throw</code> временного объекта	56
17.7.3	Повторный проброс	56
17.8	Спецификатор <code>noexcept</code>	57
17.8.1	Что такое <code>noexcept</code> ?	57
17.8.2	Зачем это нужно?	57
17.8.3	Условный <code>noexcept</code>	57
17.8.4	Что будет, если <code>noexcept</code> -функция бросит?	58
17.8.5	Что пометить <code>noexcept</code> ?	58
17.9	Исключения в деструкторах	59
17.9.1	Почему нельзя бросать из деструктора?	59
17.9.2	Как обрабатывать ошибки в деструкторе?	59
17.9.3	Функция <code>std::uncaught_exceptions()</code>	60
17.10	<code>Function-try-block</code>	60
17.11	Гарантии безопасности исключений (Exception Safety)	61
17.11.1	Четыре уровня гарантий	62
17.11.2	Пример: строгая гарантия в <code>push_back</code>	62
17.11.3	Copy-and-swap idiom — строгая гарантия для <code>operator=</code>	63
17.12	<code>std::exception_ptr</code> — исключения между потоками	63
17.12.1	Основные функции	63
17.12.2	Пример: передача исключения между потоками	64
17.12.3	Пример: сбор нескольких исключений	64
17.13	<code>std::nested_exception</code> — вложенные исключения	65
17.13.1	Распаковка вложенных исключений	66
17.14	Стоимость исключений: zero-cost exception model	66
17.15	Итого: чек-лист по исключениям	67
18	Контейнеры и итераторы	68
18.1	<code>std::vector</code> : реализация изнутри	68
18.1.1	Почему нельзя писать <code>new T[newcap]</code>	68
18.1.2	Placement new	68
18.1.3	Почему нельзя <code>delete[] arr_</code>	68
18.1.4	Реализация <code>reserve</code>	68
18.1.5	Реализация <code>push_back</code>	69
18.2	<code>vector<bool></code> : специализация с битовым хранением	69
18.2.1	Прокси-класс <code>BitReference</code>	70
18.2.2	Трюк Ильи Мещерина: узнать тип выражения	70
19	Библиотека <code><algorithm></code>	72
19.1	Зачем она вообще нужна	72
19.2	<code>std::sort</code>	72
19.3	<code>std::find</code> и <code>std::find_if</code>	72
19.4	<code>std::count</code> и <code>std::count_if</code>	73

19.5	<code>std::for_each</code> и <code>std::transform</code>	73
19.6	<code>std::copy</code> и <code>std::back_inserter</code>	73
19.7	Идиома <code>erase-remove</code>	73
19.8	<code>std::accumulate</code> из <code><numeric></code>	73
20	Дек (<code>std::deque</code>)	75
20.1	Инвализация указателей и ссылок в <code>vector</code>	75
20.2	Два главных отличия дека от вектора	75
20.3	Внутреннее устройство дека	75
21	STL: ассоциативные контейнеры	75
21.1	Обзор	75
21.2	<code>std::set</code> — упорядоченное множество	76
21.3	<code>std::map</code> — упорядоченный словарь	76
21.3.1	Осторожно с <code>operator[]</code>	76
21.3.2	Эффективная вставка	77
21.4	<code>multiset</code> и <code>multimap</code>	77
21.5	Неупорядоченные контейнеры (хеш-таблицы)	77
21.5.1	Устройство хеш-таблицы	77
21.5.2	Свой хеш для пользовательского типа	77
21.6	Упорядоченные vs неупорядоченные: что выбрать	78
21.7	Кастомный компаратор для упорядоченных контейнеров	78
22	Адаптеры контейнеров	78
22.1	<code>std::stack</code>	79
23	Итераторы и их категории	80
23.1	Что такое итератор	80
23.2	Категории итераторов	80
23.3	<code>iterator_traits</code>	80
23.4	Реализация <code>std::distance</code>	81
23.5	Реализация итератора вектора	81
24	Потоковые итераторы	83
24.1	<code>std::istream_iterator</code>	83
24.2	<code>std::ostream_iterator</code>	83
24.3	Итераторы файловых потоков	83
25	Аллокаторы и управление памятью	85
25.1	Зачем аллокатор нужен контейнеру	85
25.2	Устройство стандартного аллокатора	85
25.3	Проблема: <code>list</code> аллоцирует не <code>T</code>	86
25.4	<code>allocator_traits</code>	86
25.4.1	Политики распространения аллокатора	87
25.5	<code>Allocator-aware</code> контейнер	87
25.6	Перегрузка <code>operator new</code> и <code>operator delete</code>	88
25.6.1	Глобальная перегрузка	88
25.6.2	Классовая перегрузка	88
25.6.3	Когда использовать что	89

26 Многопоточность в C++	90
26.1 Определение потока исполнения	90
26.1.1 Что такое поток?	90
26.1.2 Аналогия	90
26.1.3 Логическая многопоточность vs Аппаратный параллелизм	90
26.1.4 Типы многозадачности	91
26.2 Работа с потоками в C++	91
26.2.1 Создание потока	91
26.2.2 Передача аргументов в поток	91
26.2.3 Использование лямбд	92
26.2.4 <code>join()</code> vs <code>detach()</code>	92
26.2.5 Опасность: забыть <code>join/detach</code>	93
26.3 Область памяти и состояние гонки (Data Race)	93
26.3.1 Что такое Memory Location?	93
26.3.2 Что такое Data Race?	93
26.3.3 Наглядный пример гонки	94
26.4 Распространённые ошибки и заблуждения	95
26.4.1 Ошибка №1: <code>volatile</code> «чинит» гонки	95
26.4.2 Ошибка №2: Привязка к одному ядру решает гонки	95
26.4.3 Ошибка №3: <code>std::cout</code> безопасен для многопоточности	96
26.5 Синхронизация: Мьютексы	96
26.5.1 Что такое мьютекс?	96
26.5.2 Базовый интерфейс	96
26.5.3 Методы мьютекса	97
26.5.4 Правила и UB при работе с мьютексами	97
26.6 RAII в многопоточности	98
26.6.1 Проблема голых <code>lock/unlock</code>	98
26.6.2 <code>std::lock_guard</code> — простая RAII-обёртка	98
26.6.3 <code>std::unique_lock</code> — гибкая обёртка	99
26.6.4 <code>std::scoped_lock</code> (C++17) — захват нескольких мьютексов	99
26.6.5 Сравнение RAII-обёрток	100
26.7 Проектирование потокобезопасных объектов	100
26.7.1 Уровни потокобезопасности	100
26.7.2 API Race — коварная ловушка	101
26.8 Дедлоки (Deadlocks)	102
26.8.1 Что такое дедлок?	102
26.8.2 Классический пример: перевод денег	102
26.8.3 Задача об обедающих философах	103
26.8.4 Решения в C++	103
26.8.5 4 необходимых условия дедлока (условия Коффмана)	104
26.9 Итого: шпаргалка	104
27 Приведение типов (касты) в C++	105
27.1 «Запрещённые заклинания» — зачем нужны касты?	105
27.1.1 Почему не один каст, а четыре?	105
27.2 C-style cast — «Грубая сила»	105
27.2.1 Синтаксис	105
27.2.2 Как это работает внутри?	105

27.2.3	Почему это плохо?	106
27.2.4	Главная опасность	106
27.3	<code>static_cast</code> — «Безопасное преобразование»	106
27.3.1	Суть	106
27.3.2	Сценарий 1: Числовые преобразования	106
27.3.3	Сценарий 2: Enum ↔ int	107
27.3.4	Сценарий 3: void* → конкретный тип	107
27.3.5	Сценарий 4: Преобразование через конструктор	108
27.3.6	Сценарий 5: Приведение в иерархии наследования (downcast)	108
27.3.7	Чего <code>static_cast</code> НЕ может	108
27.4	<code>reinterpret_cast</code> — «Рентген памяти»	109
27.4.1	Суть	109
27.4.2	Аналогия	109
27.4.3	Пример 1: Заглядываем «внутри» float	109
27.4.4	Пример 2: Указатель → число и обратно	109
27.4.5	Пример 3: Работа с сетевыми пакетами	110
27.4.6	<code>reinterpret_cast</code> НЕ может снять const	110
27.4.7	Когда <code>reinterpret_cast</code> оправдан?	110
27.5	<code>const_cast</code> — «Снятие защиты»	111
27.5.1	Суть	111
27.5.2	Мягкая форма: добавление const	111
27.5.3	Жёсткая форма: снятие const	111
27.5.4	Главное правило и UB	112
27.5.5	Реальный use case: избежание дублирования кода	112
27.6	<code>dynamic_cast</code> — «Безопасный детектив» (краткий обзор)	113
27.7	Сводная таблица: какой каст когда	113
27.8	Упражнения	114
27.8.1	Упражнение 1: Найди ошибку	114
27.8.2	Упражнение 2: Перепиши без C-style cast	115
27.8.3	Упражнение 3: Что выведет программа?	115
27.8.4	Упражнение 4: Реализуй безопасное приведение	116
27.8.5	Упражнение 5: Детектив по кастам	117
27.8.6	Упражнение 6: Реализуй type-panning безопасно	118
27.9	Итоговая шпаргалка	119
28	Иерархия <code>std::exception</code>	120
28.1	Зачем нужна единая иерархия исключений?	120
28.2	Класс <code>std::exception</code>	120
28.2.1	Интерфейс	120
28.2.2	Пример использования	120
28.3	Дерево стандартных исключений	121
28.4	<code>std::logic_error</code> и <code>std::runtime_error</code>	121
28.4.1	Конструкторы	121
28.4.2	Когда что бросать?	122
28.5	Наследники <code>std::logic_error</code>	122
28.5.1	<code>std::invalid_argument</code>	122
28.5.2	<code>std::out_of_range</code>	122
28.5.3	<code>std::length_error</code>	123

28.5.4	<code>std::domain_error</code>	123
28.6	Наследники <code>std::runtime_error</code>	123
28.6.1	<code>std::overflow_error</code> и <code>std::underflow_error</code>	123
28.6.2	<code>std::system_error</code> (C++11)	124
28.7	Отдельные исключения (не наследуют <code>logic/runtime_error</code>)	124
28.7.1	<code>std::bad_alloc</code>	124
28.7.2	<code>std::bad_cast</code>	125
28.8	Создание собственных исключений	125
29	<code>std::optional</code> (C++17)	127
29.1	Зачем нужен <code>std::optional</code> ?	127
29.2	Создание	127
29.2.1	Конструкторы	127
29.3	Проверка наличия значения	128
29.3.1	<code>has_value()</code> и <code>operator bool()</code>	128
29.4	Доступ к значению	128
29.4.1	<code>operator*()</code> и <code>operator->()</code>	128
29.4.2	<code>value()</code>	128
29.4.3	<code>value_or(default)</code>	129
29.5	Модификация	129
29.5.1	<code>emplace(args...)</code>	129
29.5.2	<code>reset()</code>	129
29.5.3	Присваивание	130
29.6	Сравнение	130
29.7	Монадический интерфейс (C++23)	130
29.7.1	<code>and_then(f)</code>	130
29.7.2	<code>transform(f)</code>	131
29.7.3	<code>or_else(f)</code>	131
29.8	Где хранится значение?	132
29.9	Типичные паттерны использования	132
29.9.1	Функция, которая может не вернуть результат	132
29.9.2	Ленивая инициализация (кэширование)	133
29.9.3	Необязательные параметры структуры	133
30	<code>std::expected</code> (C++23)	135
30.1	Зачем нужен <code>std::expected</code> ?	135
30.2	Создание	135
30.2.1	Создание со значением (успех)	135
30.2.2	Создание с ошибкой (неудача)	136
30.3	Проверка состояния	136
30.3.1	<code>has_value()</code> и <code>operator bool()</code>	136
30.4	Доступ к значению	136
30.4.1	<code>operator*()</code> и <code>operator->()</code>	136
30.4.2	<code>value()</code>	137
30.4.3	<code>value_or(default)</code>	137
30.5	Доступ к ошибке	137
30.5.1	<code>error()</code>	137
30.6	Модификация	137

30.6.1	<code>emplace(args...)</code>	137
30.6.2	Присваивание	138
30.7	Монадический интерфейс (C++23)	138
30.7.1	<code>and_then(f)</code>	138
30.7.2	<code>transform(f)</code>	138
30.7.3	<code>or_else(f)</code>	139
30.7.4	<code>transform_error(f)</code>	139
30.8	Где хранится значение?	139
30.9	<code>std::expected<void, E></code>	140
30.10	Типичные паттерны использования	140
30.10.1	Парсинг с информативной ошибкой	140
30.10.2	Цепочка операций	141
30.11	Сводная таблица: <code>optional</code> vs <code>expected</code> vs исключения	142
31	CRTP — Curiously Recurring Template Pattern	143
31.1	Что такое CRTP?	143
31.1.1	Ключевая идея	143
31.2	Зачем это нужно? Проблема виртуальных функций	144
31.3	Как работает <code>static_cast</code> в CRTP?	144
31.4	Применение 1: Статический полиморфизм	144
31.4.1	С виртуальными функциями (обычный подход)	145
31.4.2	С CRTP (статический полиморфизм)	145
31.5	Применение 2: Добавление функциональности (mixins)	146
31.5.1	Пример: автоматические операторы сравнения	146
31.5.2	Пример: автоматический <code>operator!=</code> и постфиксный <code>++</code>	147
31.6	Применение 3: Подсчёт объектов	148
31.7	Применение 4: Singleton через CRTP	149
31.8	Применение 5: Метод <code>clone()</code> без повторения кода	150
31.9	Применение 6: Статический интерфейс («концепты до концептов»)	151
31.10	Защита от ошибок	152
31.10.1	Проблема: неправильный шаблонный аргумент	152
31.10.2	Решение: проверка в конструкторе	152
31.10.3	Решение C++20: <code>static_assert</code> с концептами	153
31.11	CRTP и множественное наследование	153
31.12	CRTP vs виртуальные функции: когда что?	154
31.13	Пример из реального мира: <code>std::enable_shared_from_this</code>	155
31.14	Deducing this (C++23) — замена CRTP?	155
31.15	Итого: шпаргалка по CRTP	156
32	Указатели на функции, методы, поля. Лямбды. Type erasure. <code>std::function</code>	157
32.1	Указатели на функции	157
32.1.1	Что такое указатель на функцию	157
32.1.2	Использование <code>typedef</code> и <code>using</code>	157
32.1.3	Передача указателя на функцию как параметр	157
32.1.4	Массив указателей на функции	158
32.1.5	Указатели на функции и шаблоны	158
32.1.6	Указатели на поексепт-функции (C++17)	158
32.2	Указатели на методы (member function pointers)	159

32.2.1 Синтаксис	159
32.2.2 Синтаксис <code>using</code>	159
32.2.3 Указатели на <code>const</code> -методы	159
32.2.4 Указатели на статические методы	160
32.2.5 Использование <code>std::invoke</code> (C++17)	160
32.3 Указатели на поля (pointer to data member)	160
32.3.1 Синтаксис	160
32.3.2 Практическое применение: обобщённый доступ	161
32.3.3 Доступ через <code>std::invoke</code>	161
32.4 Лямбда-выражения	162
32.4.1 Мотивация	162
32.4.2 Базовый синтаксис	162
32.4.3 Что такое лямбда на самом деле?	162
32.4.4 Захват переменных (capture)	163
32.4.5 Захват <code>this</code>	164
32.4.6 Init-capture (обобщённый захват, C++14)	164
32.4.7 Ключевое слово <code>mutable</code>	165
32.4.8 Лямбда без захвата конвертируется в указатель на функцию	165
32.4.9 Generic-лямбды (C++14)	165
32.4.10 Шаблонные лямбды (C++20)	166
32.4.11 Immediately Invoked Lambda Expression (IILE)	167
32.5 Указатели на функции, функторы и лямбды — сравнение	167
32.6 Type erasure (стирание типа)	167
32.6.1 Проблема	167
32.6.2 Идея type erasure	167
32.6.3 Пример: собственный AnyCallable	168
32.6.4 Как это работает (диаграмма)	169
32.6.5 Type erasure в стандартной библиотеке	169
32.7 <code>std::function</code>	170
32.7.1 Что это	170
32.7.2 Проверка на пустоту	170
32.7.3 Накладные расходы <code>std::function</code>	171
32.7.4 Хранение указателей на методы	171
32.7.5 <code>std::move_only_function</code> (C++23)	172
32.7.6 Пример: система событий (observer pattern)	172
32.8 Итого	174
33 Move-семантика в C++: value categories, rvalue references, perfect forwarding	174
33.1 Value Categories (Категории значений)	174
33.1.1 Lvalue (Long-lived)	174
33.1.2 Rvalue (Short-lived)	175
33.1.3 Тернарный оператор и оператор запятая	175
33.2 RValue Reference (Rvalue-ссылки)	175
33.3 Move Semantics (Семантика перемещения)	175
33.3.1 <code>std::move</code>	176
33.4 Perfect Forwarding (Идеальная передача)	176
33.4.1 Универсальные ссылки (Universal References)	176
33.4.2 Сворачивание ссылок (Reference Collapsing)	176

33.4.3	<code>std::forward</code>	177
33.5	Variadic Templates (Шаблоны с переменным количеством аргументов)	177
34	Вариативные шаблоны (variadic templates)	178
34.1	Что это	178
34.1.1	Пакеты параметров (parameter packs)	178
34.2	Рекурсивная обработка пакета (классический способ)	178
34.3	Fold-выражения (C++17)	179
34.4	Perfect forwarding в вариативных шаблонах	179
34.5	Пример: типобезопасный кортеж и доступ по индексу	179
34.6	Развёртка в разных контекстах	180
35	Метапрограммирование: constexpr, метафункции, SFINAE, концепты	180
35.1	Что такое метапрограммирование	180
35.2	constexpr — вычисления на этапе компиляции	180
35.2.1	if constexpr (C++17)	180
35.3	Метафункции и type traits	181
35.3.1	Своя метафункция	181
35.4	SFINAE	181
35.4.1	Идиома detection (проверка наличия члена)	181
35.5	Концепты (C++20)	182
35.5.1	Способы применить ограничение	182
35.5.2	requires-выражение — задаём свой концепт	182
35.6	Зачем всё это: библиотечный код	183

1. Встроенные типы данных, представление чисел в памяти, строки

1.1. Фундаментальные типы

В C++ есть набор **встроенных (фундаментальных) типов**, которые понимает сам компилятор без подключения заголовков:

Категория	Типы	Назначение
Целочисленные	char, short, int, long, long long	целые числа
Беззнаковые	unsigned char, ..., unsigned long long	неотрицательные целые
С плавающей точкой	float, double, long double	вещественные числа
Логический	bool	true / false
Символьный	char, wchar_t, char8_t, char16_t, char32_t	символы
Пустой	void	отсутствие значения

1.1.1. Размеры типов не фиксированы стандартом

Стандарт C++ гарантирует только **минимальные** размеры и соотношения, а не точные значения в байтах:

$\text{sizeof}(\text{char}) = 1 \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long})$

Гарантируется: $\text{char} \geq 8$ бит, short и $\text{int} \geq 16$ бит, $\text{long} \geq 32$ бит, $\text{long long} \geq 64$ бит. По стандарту $\text{sizeof}(\text{char}) == 1$ *всегда* (это единица измерения размеров).

```
1 #include <stdint>
2 // Типы фиксированной ширины если (нужен точный размер):
3 int8_t a; // ровно 8 бит
4 int32_t b; // ровно 32 бита
5 uint64_t c; // ровно 64 бита, беззнаковый
6 intptr_t d; // целое, в которое влезает указатель
```

1.2. Представление целых чисел в памяти

1.2.1. Беззнаковые числа

Беззнаковое число хранится как обычное двоичное представление. Для n бит диапазон значений: $[0, 2^n - 1]$.

```
uint8_t x = 200; // 1100 1000 = 128 + 64 + 8 = 200
```

При переполнении беззнаковые числа работают **по модулю** 2^n — это определённое поведение (wrap-around):

```

1 uint8_t x = 255;
2 x = x + 1;    // 0 (256 mod 256), НЕ undefined behavior

```

1.2.2. Знаковые числа: дополнительный код (two's complement)

Отрицательные числа хранятся в **дополнительном коде**. Старший бит — знаковый. Чтобы получить представление $-x$: инвертируем все биты x и прибавляем 1.

```

int8_t  5  =  0000 0101
int8_t -5  =  1111 1011 инверсия ( 5 -> 1111 1010, +1 -> 1111
                        1011)

```

Преимущество дополнительного кода: сложение знаковых и беззнаковых чисел выполняется **одной и той же** аппаратной операцией, и существует ровно один ноль. Диапазон для n бит: $[-2^{n-1}, 2^{n-1}-1]$ (отрицательных значений на одно больше).

Важно: переполнение *знакового* целого — это undefined behavior (UB), в отличие от беззнакового.

1.2.3. Порядок байтов (endianness)

Многобайтовое число можно хранить от младшего байта к старшему (little-endian, x86) или наоборот (big-endian, сеть):

```

uint32_t x = 0x12345678;
// little-endian в памяти :  78 56 34 12
// big-endian  в памяти   :  12 34 56 78

```

1.3. Числа с плавающей точкой (IEEE 754)

`float` (32 бита) и `double` (64 бита) хранятся по стандарту IEEE 754: знак, экспонента (смещённый порядок) и мантисса.

$$\text{значение} = (-1)^{\text{знак}} \times 1.\text{мантисса} \times 2^{\text{экспонента}-\text{смещение}}$$

Тип	Знак	Экспонента	Мантисса
float	1 бит	8 бит	23 бита
double	1 бит	11 бит	52 бита

Следствия:

- Не все числа представимы точно: $0.1 + 0.2 \neq 0.3$.
- Сравнивать вещественные через `==` опасно — нужно сравнивать с допуском (`epsilon`).
- Есть специальные значения: `+inf`, `-inf`, `NaN` (причём `NaN != NaN`).

```

1 #include <cmath>
2 bool almost_equal(double a, double b, double eps = 1e-9) {
3     return std::fabs(a - b) < eps;
4 }

```

1.4. Строки

В C++ есть две принципиально разные «строки».

1.4.1. C-строки (const char*)

Массив символов, завершающийся нулевым байтом '\0'. Длина не хранится — её вычисляют, идя до нуля (strlen, $O(n)$).

```

1 const char* s = "hello"; // в памяти : h e l l o \0 (6 байт)

```

Минусы: ручное управление памятью, нет информации о длине, легко допустить переполнение буфера.

1.4.2. std::string

Класс из <string>, управляющий динамическим буфером. Хранит указатель на данные, длину и вместимость (capacity), сам выделяет и освобождает память.

```

1 #include <string>
2 std::string s = "hello";
3 s += " world"; // динамически растёт
4 std::cout << s.size(); // длина 11

```

Small String Optimization (SSO). Короткие строки (обычно до 15–22 символов) хранятся **прямо внутри объекта std::string**, без выделения кучи. Это резко ускоряет работу с короткими строками.

1.4.3. std::string_view (C++17)

Невладельщая «ссылка» на последовательность символов: указатель + длина. Не копирует данные, не выделяет память. Идеален для передачи строк в функции только для чтения.

```

1 #include <string_view>
2 size_t count_spaces(std::string_view sv) { // без копирования
3     return std::count(sv.begin(), sv.end(), ' ');
4 }
5 count_spaces("hello world"); // из литерала
6 std::string s = "...";
7 count_spaces(s); // из std::string

```

Опасность: string_view не продлевает время жизни данных. Если исходная строка уничтожена, string_view становится висячим.

2. Преобразования типов: явные и неявные

2.1. Неявные преобразования (implicit conversions)

Компилятор сам вставляет преобразование, когда тип выражения не совпадает с ожидаемым. Это происходит при присваивании, передаче аргументов, в арифметике и т. д.

2.1.1. Целочисленные продвижения (integral promotion)

Типы меньше `int` (`char`, `short`, `bool`) перед арифметикой продвигаются до `int`:

```
1 char a = 10, b = 20;
2 auto c = a + b;    // a и b -> int, тип c == int a( не char!)
```

2.1.2. Стандартные арифметические преобразования

Когда операнды разных типов, они приводятся к «более широкому» общему типу:

```
1 int i = 5;
2 double d = 2.0;
3 auto x = i + d;    // i -> double, типрезультата double
```

2.1.3. Опасность: знаковое и беззнаковое

Если в выражении встречаются знаковый и беззнаковый типы одинаковой ширины, знаковый приводится к беззнаковому — источник коварных багов:

```
1 unsigned int u = 1;
2 int i = -1;
3 if (i < u) // i -> unsigned: -1 становится 4294967295
4     std::cout << "i < u";
5 else
6     std::cout << "i >= u";    // <-- выполнитсяЭТАветка !
```

2.1.4. Сужающие преобразования (narrowing)

Преобразование, при котором значение может не поместиться или потерять точность (`double` → `int`, `int` → `char`). В обычном коде допускается с потерей данных, но **запрещено** в списковой инициализации `{}`:

```
1 int x = 3.9;        // ОК, x == 3 дробная( частьотброшена )
2 int y{3.9};         // ОШИБКАкомпиляции : narrowing запрещён {}
```


2.1.5. Пользовательские неявные преобразования

Конструктор с одним аргументом и оператор приведения задают неявные преобразования для пользовательских типов:

```
1 struct Meters {
2     double value;
3     Meters(double v) : value(v) {}          // неявный конструктор
4     operator double() const { return value; } //
        неявное приведение
5 };
6 Meters m = 5.0;          // double -> Meters через( конструктор)
7 double d = m;           // Meters -> double через( operator double)
```

Чтобы запретить неявное преобразование через конструктор, используют `explicit`:

```
1 struct Meters {
2     explicit Meters(double v); // теперь Meters m = 5.0; --
        ОШИБКА
3 };
4 Meters m(5.0); // только явный вызов
```

2.2. Явные преобразования (casts)

C++ предоставляет четыре именованных оператора приведения вместо «всегомогущего» C-style каста (T)x. Они подробно разобраны в отдельном разделе о кастах; кратко:

Каст	Назначение
<code>static_cast<T></code>	безопасные «логические» преобразования (числа, иерархия вверх, void*)
<code>const_cast<T></code>	снятие/добавление <code>const/volatile</code>
<code>reinterpret_cast<T></code>	побитовая переинтерпретация (указатели, целые)
<code>dynamic_cast<T></code>	безопасное приведение вниз по полиморфной иерархии (с проверкой в runtime)

```
1 double d = 3.7;
2 int i = static_cast<int>(d); // явно: i == 3, намерение видно
```

Преимущества именованных кастов перед C-style: их легко искать в коде, они выражают намерение и не делают «слишком много» случайно.

3. Структуры и объединения

3.1. Структуры (struct)

`struct` объединяет несколько полей в один тип. В C++ `struct` и `class` — почти одно и то же; разница только в **умолчательной видимости**: у `struct` члены

public по умолчанию, у class — private.

```
1 struct Point {
2     int x;
3     int y;
4 };
5 Point p{3, 4};           // агрегатная инициализация
6 std::cout << p.x;       // доступ через точку
7 Point* pp = &p;
8 std::cout << pp->y;     // доступ через указатель
```

3.1.1. Выравнивание и padding

Поля структуры размещаются с учётом **выравнивания** (alignment): адрес поля должен быть кратен его требованию выравнивания. Поэтому компилятор вставляет «дыры» (padding):

```
1 struct Bad {
2     char a;    // 1 байт
3     // 3 байта padding
4     int b;     // 4 байта адрес (кратен 4)
5     char c;    // 1 байт
6     // 3 байта padding для (кратности размера)
7 }; // sizeof(Bad) == 12, а не 6!
8
9 struct Good {
10    int b;     // 4
11    char a;    // 1
12    char c;    // 1
13    // 2 байта padding
14 }; // sizeof(Good) == 8
```

Правило: располагайте поля от больших к меньшим, чтобы уменьшить padding.

3.2. Объединения (union)

В union все поля **делят одну и ту же память**. Размер объединения равен размеру наибольшего поля. В любой момент времени «активно» только одно поле.

```
1 union Value {
2     int i;
3     float f;
4     char bytes[4];
5 };
6 Value v;
7 v.i = 0x40490FDB;
8 std::cout << v.f; // прочитайте байты как float ~= 3.14159
```

Опасность: чтение неактивного поля — технически UB (хотя на практике часто используется для побитовой интерпретации; правильный способ — `std::bit_cast` в C++20 или `memcpy`).

3.2.1. Tagged union и std::variant

Чтобы безопасно знать, какое поле активно, к union добавляют тег:

```
1 struct Tagged {  
2     enum { INT, FLOAT } tag;  
3     union { int i; float f; };  
4 };
```

В современном C++ вместо ручного tagged union используют std::variant (C++17) — типобезопасное объединение, которое само следит за активным типом.

4. Указатели

4.1. Что такое указатель

Указатель — это переменная, хранящая **адрес** другого объекта в памяти. Тип указателя говорит, как интерпретировать байты по этому адресу.

```
1 int x = 42;  
2 int* p = &x;      // p хранит адрес x (& -- взятие адреса )  
3 std::cout << *p; // 42 -- разыменован доступ ( к значению по адресу )  
4 *p = 100;         // меняем x через указатель -> x == 100
```

Размер указателя одинаков для всех типов и равен ширине адреса платформы (8 байт на x86-64).

4.2. Нулевой указатель

nullptr (C++11) — специальное значение «никуда не указывает». Разыменование nullptr — UB (обычно падение).

```
1 int* p = nullptr;  
2 if (p) { /* не выполнится */ } // указатель конвертируется в bool
```

Используйте nullptr, а не NULL или 0: nullptr имеет собственный тип std::nullptr_t и корректно работает с перегрузкой функций.

4.3. Указатели и const

Расположение const меняет смысл:

```
1 const int* p;      // указатель на const int: нельзя менять *p, можно p  
2 int* const p;      // const указатель: нельзя менять p, можно *p  
3 const int* const p; // нельзя менять ни *p, ни p
```

Читать справа налево: «p is a const pointer to a const int».

4.4. Арифметика указателей

Прибавление к указателю n сдвигает адрес на $n \times \text{sizeof}(T)$ байт, а не на n байт:

```
1 int arr[5] = {10, 20, 30, 40, 50};
2 int* p = arr;
3 std::cout << *(p + 2); // 30 сдвиг( на 2*sizeof(int) = 8 байт)
4 ++p; // теперь p указывает на arr[1]
5 std::cout << p - arr; // 1 разность( указателей в элементах )
```

4.5. Указатель на void и на функцию

```
1 void* vp = &x; // бестиповый «» указатель,
// нельзя назвать напрямую
2 int* ip = static_cast<int*>(vp); // нужно вернуть тип
3
4 int (*fp)(int, int) = &add; // указатель на функцию
5 int r = fp(2, 3); // вызов через указатель
```

4.6. Висячие и невалидные указатели

- **Висячий (dangling)** — указывает на уже освобождённую память (после delete или выхода объекта из области видимости).
- **Утечка (leak)** — объект выделен new, но указатель потерян, и delete никогда не вызовется.
- **Двойное освобождение** — delete дважды по одному адресу — UB.

В современном C++ сырые new/delete заменяют **умными указателями** (std::unique_ptr, std::shared_ptr), которые освобождают память автоматически.

5. Массивы

5.1. Встроенные (C-style) массивы

Непрерывный блок однотипных элементов фиксированного размера, известного на этапе компиляции:

```
1 int arr[5] = {1, 2, 3, 4, 5};
2 arr[0] = 10; // индексация с нуля
3 std::cout << sizeof(arr); // 20 == 5 * sizeof(int)
4 std::cout << sizeof(arr)/sizeof(arr[0]); // 5 -- число элементов
```

Нет проверки границ: arr[10] — UB, компилятор не остановит.

5.1.1. Распад массива в указатель (array decay)

При передаче в функцию массив **распадается** (decay) в указатель на первый элемент — информация о размере теряется:

```
1 void f(int a[]) {           // насамомделе int* a
2     std::cout << sizeof(a); // размерУКАЗАТЕЛЯ (8), анемассива
3     !
4 }
5 int arr[5];
6 f(arr); // передаётся &arr[0], размерпотерян
```

Поэтому в С-стиле размер передают отдельным параметром: `f(int* a, size_t n)`.

5.2. Многомерные массивы

```
1 int m[2][3] = {{1,2,3}, {4,5,6}};
2 // впамятилежатподряд (row-major): 1 2 3 4 5 6
3 std::cout << m[1][2]; // 6
```

5.3. `std::array` (C++11)

Обёртка над С-массивом фиксированного размера, но с интерфейсом контейнера: знает свой размер, не распадается в указатель, поддерживает итераторы.

```
1 #include <array>
2 std::array<int, 5> a = {1, 2, 3, 4, 5};
3 std::cout << a.size(); // 5 -- размернотеряется
4 for (int x : a) { /* ... */ } // работают range-based for
5 // иалгоритмы
6 a.at(10); // бросает std::out_of_range с(
7 // проверкой)
```

5.4. `std::vector` — динамический массив

Когда размер неизвестен на этапе компиляции или должен меняться, используют `std::vector` — массив, выделяемый в куче и растущий по мере необходимости (подробно — в разделе о контейнерах).

```
1 #include <vector>
2 std::vector<int> v = {1, 2, 3};
3 v.push_back(4); // динамическирастёт
4 std::cout << v.size(); // 4
```

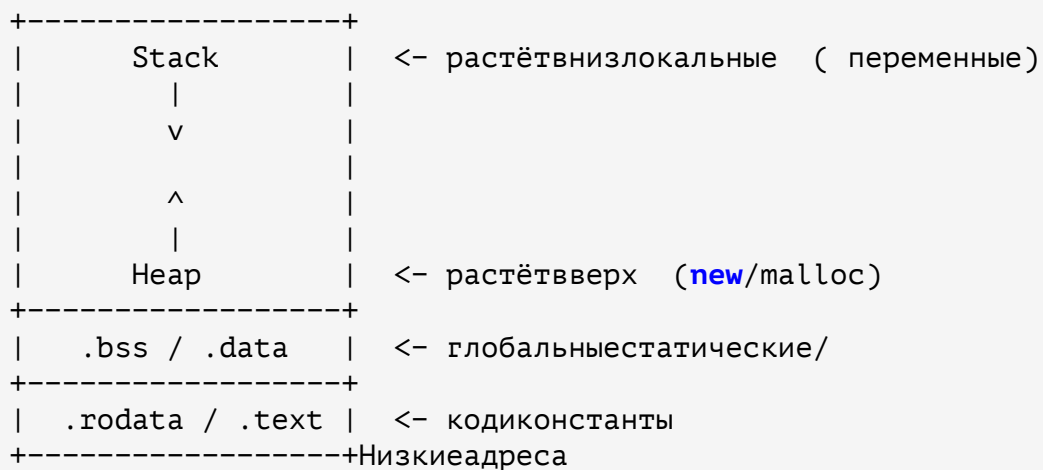
6. Устройство памяти, сегменты

6.1. Сегменты памяти процесса

Адресное пространство процесса делится на области (сегменты):

Сегмент	Что хранит
.text (code)	машинный код программы, только для чтения
.rodata	константы и строковые литералы (только чтение)
.data	инициализированные глобальные/статические переменные
.bss	неинициализированные глобальные/статические (обнуляются)
Куча (heap)	динамическая память (new/malloc), растёт вверх
Стек (stack)	локальные переменные, кадры вызовов, растёт вниз

Высокие адреса



6.2. Стек vs Куча

	Стек	Куча
Управление	автоматическое (компилятор)	ручное (new/delete)
Скорость	очень быстро (сдвиг указателя)	медленнее (поиск блока)
Размер	ограничен (обычно 1–8 МБ)	ограничен ОЗУ
Время жизни	до конца области видимости	пока не вызовут delete
Фрагментация	нет	возможна

```
1 void f() {  
2     int x = 5;           // на стеке ,  
                           исчезнет при выходе из f  
3     int* p = new int(5); // объект в куче , указатель p  
                           на стеке  
4     delete p;           // память в куче нужно вернуть вручную  
5 }                       // x уничтожается автоматически
```

6.3. Время жизни и классы хранения

- **Автоматическое** (локальные переменные) — на стеке, живёт до конца блока.
- **Статическое** (static, глобальные) — в .data/.bss, живёт всё время работы программы.
- **Динамическое** (new) — в куче, живёт до delete.
- **Thread-local** (thread_local) — по копии на каждый поток.

6.4. Переполнение стека (stack overflow)

Стек ограничен. Слишком глубокая рекурсия или огромный локальный массив исчерпывают его:

```
1 void recurse() { recurse(); } // бесконечная рекурсия -> stack  
  overflow  
2 void big() { int huge[10'000'000]; } //  
  слишком большой кадр стека
```

7. Стек вызова функций

7.1. Кадр стека (stack frame)

При каждом вызове функции на стеке создаётся **кадр (frame)**, содержащий:

- аргументы функции (часть может передаваться в регистрах);
- адрес возврата (куда вернуться после функции);
- сохранённый указатель кадра вызывающего;
- локальные переменные.

```

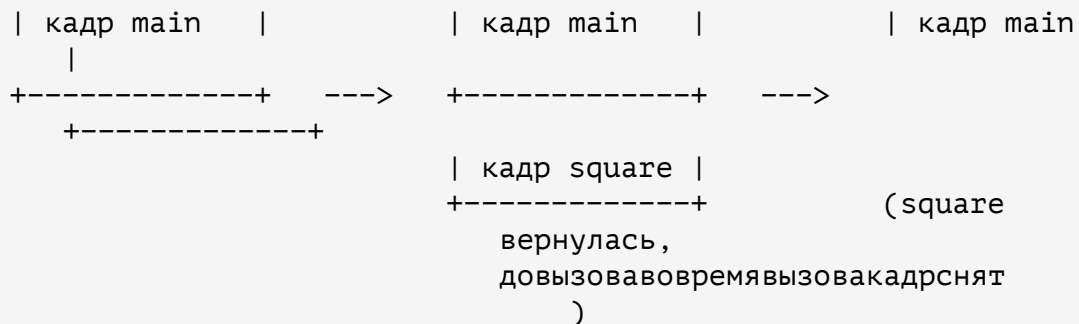
1  int square(int n) {           // кадр square: n, локальные
2      int r = n * n;
3      return r;
4  }
5  int main() {
6      int x = square(5);        // создаётся кадр square поверх кадра
                                // main
7  }

```

7.1.1. Как растёт стек

При вызове кадр **кладётся** на вершину стека, при возврате — **снимается**. Стек растёт в сторону младших адресов:

main() вызывает square(5):



7.2. Указатели стека

- SP (stack pointer) — указывает на вершину стека.
- BP/FP (base/frame pointer) — база текущего кадра, по нему адресуются локальные переменные и аргументы.

7.3. Соглашения о вызове (calling convention)

Соглашение определяет, как передаются аргументы (регистры/стек), кто чистит стек, в каком регистре возвращается результат. Примеры: System V AMD64 (Linux), Microsoft x64 (Windows), `cdecl`, `stdcall`. На x86-64 первые несколько целочисленных аргументов обычно идут в регистрах.

7.4. Раскрутка стека (stack unwinding)

При выбросе исключения стек **раскручивается**: кадры снимаются один за другим, и для каждого локального объекта вызывается деструктор. Это основа RAII и обработки ошибок (подробно — в разделе об исключениях).

7.5. Хвостовая рекурсия

Если рекурсивный вызов — последнее действие функции, компилятор может **переиспользовать** текущий кадр (tail-call optimization), превращая рекурсию в цикл и избегая роста стека:

```
1 int fact(int n, int acc = 1) {  
2     if (n <= 1) return acc;  
3     return fact(n - 1, acc * n);    // хвостовой вызов  
4 }
```

8. Ссылки

8.1. Что такое ссылка

Ссылка (lvalue reference) — это **псевдоним** (другое имя) для уже существующего объекта. В отличие от указателя, ссылку нельзя «перенаправить» и нельзя сделать «нулевой».

```
1 int x = 10;  
2 int& ref = x;    // ref -- другое имя для x  
3 ref = 20;        // меняем x через ссылку -> x == 20  
4 int* p = &ref;   // &ref -- это адрес x
```

8.1.1. Ссылка vs указатель

	Ссылка	Указатель
Инициализация	обязательна при объявлении	необязательна
Перенаправление	невозможно	возможно
Может быть «нулём»	нет	да (nullptr)
Синтаксис доступа	как к обычной переменной	нужно * / ->
Арифметика	нет	да

8.2. Ссылки в параметрах функций

Передача по ссылке избегает копирования и позволяет менять аргумент:

```

1 void increment(int& x) { ++x; }           // меняет оригинал
2 void print(const std::string& s);        // не копирует, но не меняет

```

Правило: большие объекты, которые не меняются, передавайте по `const T&` — без копии и без риска модификации.

8.3. Висячие ссылки

Нельзя возвращать ссылку на локальную переменную — она уничтожится:

```

1 int& bad() {
2     int local = 5;
3     return local; // ОШИБКА: ссылка на уничтоженную переменную (UB)
4 }

```

8.4. Константные ссылки и продление жизни

`const`-ссылка может привязаться к временному объекту и **продлевает его время жизни** до конца жизни ссылки:

```

1 const std::string& r = std::string("temp"); // временный живёт, пока жив r

```

8.5. Rvalue-ссылки (C++11)

`T&&` — ссылка на временный объект (rvalue). Лежит в основе move-семантики и perfect forwarding (подробно — в отдельном разделе):

```

1 std::string&& rr = std::string("temp"); // привязка к rvalue
2 void sink(std::string&& s);              // забирает ресурс временного

```

9. Перегрузка функций

9.1. Что это

Перегрузка (overloading) — несколько функций с **одним именем**, но разными списками параметров. Компилятор выбирает нужную по аргументам вызова.

```

1 int add(int a, int b) { return a + b; }
2 double add(double a, double b) { return a + b; }
3 std::string add(const std::string& a, const std::string& b) {
4     return a + b; }
5
6 add(1, 2); // int версия
7 add(1.5, 2.5); // double версия
8 add("a"s, "b"s); // string версия

```

9.2. Что участвует в выборе

Перегрузки различаются по:

- числу параметров;
- типам параметров;
- `const/volatile` и ссылочности (`&/&&`).

Не различаются только по типу возвращаемого значения — это ошибка:

```
1 int f(int);  
2 double f(int); // ОШИБКА: отличается только возвращаемый тип
```

9.3. Разрешение перегрузки (overload resolution)

Компилятор делает это в три шага:

1. **Набор кандидатов** — все функции с таким именем, видимые в точке вызова.
2. **Жизнеспособные (viable)** — те, что подходят по числу аргументов и для которых существуют преобразования.
3. **Лучшая** — та, что требует «наименьших» преобразований. Точное совпадение лучше продвижения, продвижение лучше стандартного преобразования, стандартное лучше пользовательского.

Если две функции одинаково хороши — **неоднозначность (ambiguous)** — ошибка компиляции:

```
1 void f(long);  
2 void f(double);  
3 f(5); // ОШИБКА: int -> long и int -> double равноправны
```

9.4. Перегрузка по `const` и ссылочности

```
1 struct S {  
2     void g() &; // вызывается на lvalue  
3     void g() &&; // вызывается на rvalue временном()  
4     int get() const; // на const объекте  
5     int& get(); // на неконстантном  
6 };
```

9.5. Перегрузка vs переопределение vs сокрытие

- **Overloading** — одно имя, разные сигнатуры, в одной области видимости.
- **Overriding** — замена `virtual`-функции в наследнике с той же сигнатурой.
- **Hiding** — имя в наследнике скрывает все перегрузки из базы (лечится `using Base::name;`).

9.6. Значения по умолчанию

Не путать с перегрузкой: один и тот же набор параметров, но часть имеет значение по умолчанию:

```
1 void log(const std::string& msg, int level = 0);
2 log("hello");           // level == 0
3 log("err", 3);          // level == 3
```

10. Пространства имён (namespace)

10.1. Зачем нужны namespace

namespace группирует имена и предотвращает **конфликты имён** между разными библиотеками. Вся стандартная библиотека лежит в `std`.

```
1 namespace math {
2     double pi() { return 3.14159; }
3     struct Vector { double x, y; };
4 }
5 double p = math::pi();           // доступ через ::
6 math::Vector v;
```

10.2. Использование имён из namespace

```
1 using namespace std;           // вносит ВСЕ имена std не(
    рекомендуется глобально )
2 using std::cout;               // вносит только cout
    предпочтительно()
```

Совет: не пишите `using namespace std;` в заголовках и в глобальной области — это возвращает проблему конфликтов имён.

10.3. Вложенные namespace

```
1 namespace company::project::utils { // C++17, компактная запись
2     void helper();
3 }
4 // эквивалентно вложенным namespace company { namespace project {
    ... } }
```

10.4. Анонимный namespace

Имена в безымянном namespace видны **только в этом файле** (аналог `static` для внутренней линковки):

```

1 namespace {
2     int internal_counter = 0;    // невидениздругих .cpp
3 }

```

10.5. Псевдонимы namespace

```

1 namespace fs = std::filesystem;    // короткийпсевдоним
2 fs::path p = "/tmp";

```

10.6. Argument-Dependent Lookup (ADL)

При вызове функции компилятор ищет её **также** в namespace, где определены типы аргументов. Поэтому работает `std::cout << x` без явного `std::` перед `operator<<`:

```

1 namespace lib { struct S{}; void process(S); }
2 lib::S s;
3 process(s);    // находит lib::process через ADL, хотянет lib::

```

11. Компиляция: этапы, ошибки

11.1. Этапы сборки программы на C++

Превращение исходника в исполняемый файл проходит четыре стадии:

Этап	Что делает
1. Препроцессинг	раскрывает <code>#include</code> , <code>#define</code> , <code>#ifdef</code> ; результат — единый текст (translation unit)
2. Компиляция	переводит C++ в ассемблер, затем в объектный код (.o); проверяет синтаксис и типы
3. Ассемблирование	переводит ассемблер в машинный код (объектный файл)
4. Линковка	связывает объектные файлы и библиотеки в исполняемый файл, разрешает символы

```

main.cpp препроцессор--[]--> main.i компилятор--[]--> main.s
      ассемблер
      --[]--> main.o линковщик--[]--> a.out

```

```

1 g++ -E main.cpp -o main.i    # толькопрепроцессинг
2 g++ -S main.cpp -o main.s    # доассемблера
3 g++ -c main.cpp -o main.o    # дообъектногофайлабез ( линковки)
4 g++ main.cpp -o app          # всёсразу

```

11.2. Единица трансляции и модель компиляции

Каждый .cpp компилируется **независимо** в свою единицу трансляции. Заголовки (.h) физически **вставляются** препроцессором в каждый .cpp, который их включает.

11.2.1. Защита от повторного включения

Чтобы заголовок не вставился дважды в одну единицу трансляции:

```
1 #pragma once           // современный способ
2 // или классический include guard:
3 #ifndef MY_HEADER_H
4 #define MY_HEADER_H
5 // ... содержимое ...
6 #endif
```

11.3. Объявление vs определение (ODR)

- **Объявление (declaration)** — говорит «такой символ существует» (можно много раз).
- **Определение (definition)** — предоставляет тело/память (**ровно один раз** на всю программу — One Definition Rule).

```
1 // header.h
2 int add(int, int);           // объявление прототип ()
3 extern int counter;          // объявление глобальной переменной
4
5 // source.cpp
6 int add(int a, int b) { return a+b; } // определение
7 int counter = 0;              // определение
```

11.4. Виды ошибок

Вид	Когда и пример
Ошибка препроцессора	нет файла в #include, незакрытый #ifdef
Ошибка компиляции	синтаксис, неизвестный тип, несоответствие типов
Ошибка линковки	undefined reference (объявлено, но не определено), multiple definition (нарушен ODR)
Ошибка времени выполнения	segfault, деление на ноль, исключение — ловится только при запуске

11.4.1. Типичная ошибка линковки

```
1 // foo.h
2 void foo();           // объявили
3 // main.cpp
4 #include "foo.h"
5 int main() { foo(); } // используем, но определения foo() нет нигде
6 // -> undefined reference to `foo()' ошибка (ЛИНКОВКИ,
    некомпилляции )
```

11.5. Полезные флаги компилятора

```
1 g++ -std=c++20 -Wall -Wextra -Wpedantic main.cpp # стандарт +
    предупреждения
2 g++ -g main.cpp # отладочная информация для (gdb)
3 g++ -O2 main.cpp # оптимизация
4 g++ -fsanitize=address main.cpp # детектор ошибок памяти (ASan)
```

12. ООП: абстракция, инкапсуляция, наследование, полиморфизм

12.1. Четыре столпа ООП

Объектно-ориентированное программирование строится на четырёх принципах:

Принцип	Суть
Абстракция	выделить существенное, скрыть детали реализации
Инкапсуляция	объединить данные и методы, ограничить прямой доступ к данным
Наследование	строить новые типы на основе существующих
Полиморфизм	один интерфейс — разное поведение для разных типов

12.2. Абстракция

Абстракция — предоставление **интерфейса** без раскрытия реализации. Пользователь класса знает *что* делает метод, но не *как*:

```
1 class Stack {
2 public:
3     void push(int x); // что делает -- интерфейс
4     int pop();
5     bool empty() const;
```

```

6 private:
7     // КАКустроено -- скрытовектор (, список, массив -- неважно)
8     std::vector<int> data_;
9 };

```

12.3. Инкапсуляция

Инкапсуляция — сокрытие внутреннего состояния за модификаторами доступа. Доступ к данным только через методы, которые поддерживают **инварианты** класса.

Модификатор	Кто имеет доступ
public	все
protected	сам класс и наследники
private	только сам класс (и friend)

```

1 class Account {
2 public:
3     void deposit(double sum) {
4         if (sum > 0) balance_ += sum;    // инвариант:
5         баланс корректен
6     }
7     double balance() const { return balance_; }
8 private:
9     double balance_ = 0;    // нельзя изменить напрямую снаружи
10 };

```

12.4. Наследование

Наследование позволяет классу-наследнику (derived) получить члены базового класса (base) и расширить/изменить их.

```

1 class Animal {
2 public:
3     void breathe() { /* общее поведение */ }
4 protected:
5     std::string name_;
6 };
7 class Dog : public Animal {    // Dog -- это Animal (is-a)
8 public:
9     void bark() { /* специфично для Dog */ }
10 };

```

12.4.1. Виды наследования

public наследование выражает отношение «**является**» (**is-a**) и используется чаще всего. protected/private — отношение «реализован через», встречается редко.

12.4.2. Множественное наследование и проблема ромба

C++ допускает наследование от нескольких классов. Если два базовых класса сами наследуют от общего предка, возникает **ромб (diamond)** — дублирование подоби́екта. Решается **виртуальным наследованием**:

```
1 class A { public: int x; };
2 class B : virtual public A {}; // virtual ->
    единственныйподобъект A
3 class C : virtual public A {};
4 class D : public B, public C {}; // D::x -- один, а не два
```

12.5. Полиморфизм

12.5.1. Динамический полиморфизм (виртуальные функции)

Вызов метода через указатель/ссылку на базовый класс приводит к вызову версии **реального** типа объекта — в runtime:

```
1 class Shape {
2 public:
3     virtual double area() const = 0; // чистовиртуальная (=0)
4     virtual ~Shape() = default; //
        виртуальныйдеструкторобязателен !
5 };
6 class Circle : public Shape {
7     double r_;
8 public:
9     Circle(double r) : r_(r) {}
10    double area() const override { return 3.14159 * r_ * r_; }
11 };
12 class Square : public Shape {
13     double s_;
14 public:
15     Square(double s) : s_(s) {}
16     double area() const override { return s_ * s_; }
17 };
18
19 void print_area(const Shape& s) {
20     std::cout << s.area(); // вызоветсянужнаяверсия (Circle
        или Square)
21 }
```

Абстрактный класс — содержит хотя бы одну чисто виртуальную функцию (= 0); объект такого класса создать нельзя, только наследника.

12.5.2. Как это работает: vtable и vptr

У каждого полиморфного класса есть **таблица виртуальных функций (vtable)** — массив указателей на его методы. Каждый объект хранит скрытый указатель **vptr** на vtable своего класса. Виртуальный вызов — это косвенный вызов через vtable:

Circle obj;		vtable Circle:
+-----+		+-----+
vptr	----->	Circle::area
r_		Circle::~Circle
+-----+		+-----+

Зачем виртуальный деструктор: при delete через указатель на базу без virtual-деструктора вызовется только ~Base, ресурсы наследника утекут — UB.

12.5.3. override и final

```

1 class Derived : public Base {
2     void f() override;    // компилятор проверит, что в базе есть
        virtual f
3     void g() final;       // запретить переопределять g дальше
4 };
5 class X final {};         // запретить наследоваться от X

```

12.5.4. Статический полиморфизм

Полиморфизм без runtime-затрат достигается шаблонами и CRTP (см. соответствующие разделы). Выбор реализации происходит на этапе компиляции.

13. Классы. Специальные методы

13.1. Специальные функции-члены

Компилятор может сам сгенерировать шесть особых методов:

Метод	Сигнатура
Конструктор по умолчанию	T()
Деструктор	~T()
Копирующий конструктор	T(const T&)
Копирующее присваивание	T& operator=(const T&)
Перемещающий конструктор	T(T&&)
Перемещающее присваивание	T& operator=(T&&)

13.2. Конструкторы

Конструктор инициализирует объект. Может быть перегружен.

```

1 class Point {
2     int x_, y_;
3 public:
4     Point() : x_(0), y_(0) {}           // по умолчанию
5     Point(int x, int y) : x_(x), y_(y) {} // с параметрами

```

```

6     explicit Point(int v) : x_(v), y_(v) {} // explicit --
    безневных
7 };

```

13.2.1. Список инициализации членов

Члены инициализируются **в списке инициализации**, а не в теле конструктора. Это важно: в теле было бы присваивание (поле уже создано), а в списке — прямая инициализация. Для const-полей и ссылок список обязателен.

```

1 class Widget {
2     const int id_; // const можно инициализировать ТОЛЬКО в списке
3     std::string name_;
4 public:
5     Widget(int id, std::string name)
6         : id_(id), name_(std::move(name)) {} // порядок --
        каковы поля !
7 };

```

Важно: члены инициализируются в порядке **объявления в классе**, а не в порядке в списке инициализации.

13.2.2. Делегирующие конструкторы

```

1 class Point {
2     int x_, y_;
3 public:
4     Point(int x, int y) : x_(x), y_(y) {}
5     Point() : Point(0, 0) {} // делегирует другому конструктору
6 };

```

13.3. Деструктор

Деструктор освобождает ресурсы при уничтожении объекта. Вызывается автоматически. Основа RAII.

```

1 class File {
2     FILE* f_;
3 public:
4     File(const char* name) : f_(fopen(name, "r")) {}
5     ~File() { if (f_) fclose(f_); } // ресурс освобождается сам
6 };

```

13.4. Правило трёх / пяти / нуля

- **Правило трёх:** если нужен пользовательский деструктор, копирующий конструктор или копирующее присваивание — скорее всего, нужны все три (класс владеет ресурсом).

- **Правило пяти:** добавьте move-конструктор и move-присваивание для эффективности.
- **Правило нуля:** лучше **вообще не писать** спецметоды — используйте типы, которые сами управляют ресурсами (`std::string`, `std::vector`, умные указатели), тогда компилятор сгенерирует всё правильно.

```

1 // Правилонуля : ничегонепишем -- всёсгенерируетсякорректно
2 class Person {
3     std::string name_;
4     std::vector<int> scores_;
5 }; // копирование, перемещение, удаление --
    автоматическиправильно

```

13.5. = default и = delete

```

1 class T {
2 public:
3     T() = default; //
        явнопросимсгенерировать
4     T(const T&) = delete; // запрещаемкопирование
5     T& operator=(const T&) = delete;
6     T(T&&) = default; // разрешаемперемещение
7 };

```

13.6. Статические члены

`static`-член принадлежит **классу**, а не объекту — общий для всех экземпляров:

```

1 class Counter {
2     static int count_; // объявлениеодна ( копиянакласс )
3 public:
4     Counter() { ++count_; }
5     ~Counter() { --count_; }
6     static int count() { return count_; } // staticметод-: без
        this
7 };
8 int Counter::count_ = 0; // определениевнеклассаили ( inline
    в C++17)

```

13.7. const-методы и mutable

`const`-метод обещает не менять объект; его можно вызывать на `const`-объекте. `mutable`-поле можно менять даже в `const`-методе (для кэшей, мьютексов):

```

1 class Cache {
2     mutable int cached_ = -1;    // можно менять в const-методе-
3 public:
4     int get() const {
5         if (cached_ < 0) cached_ = compute();
6         return cached_;
7     }
8 };

```

13.8. friend

friend-функция или класс получает доступ к private-членам:

```

1 class Vector2 {
2     double x_, y_;
3     friend Vector2 operator+(const Vector2&, const Vector2&);    //
        имеет доступ
4 };

```

13.9. this

this — указатель на текущий объект внутри нестатического метода. Тип — T* (или const T* в const-методе).

```

1 class Builder {
2     int v_ = 0;
3 public:
4     Builder& set(int v) { v_ = v; return *this; }    // fluent
        интерфейс-
5 };
6 Builder{}.set(1).set(2);    // цепочка вызовов

```

14. Перегрузка операторов

14.1. Зачем перегружать операторы

Перегрузка операторов позволяет пользовательским типам вести себя как встроенные: складываться через +, сравниваться через ==, печататься через <<. Это синтаксический сахар: a + b — это вызов operator+(a, b).

14.2. Способы перегрузки: метод vs свободная функция

```

1 struct Complex {
2     double re, im;
3
4     // как метод : левый операнд -- *this

```

```

5     Complex operator+(const Complex& o) const {
6         return {re + o.re, im + o.im};
7     }
8 };
9
10 // как свободная функция симметрично ( для обоих операндов )
11 Complex operator-(const Complex& a, const Complex& b) {
12     return {a.re - b.re, a.im - b.im};
13 }

```

Правило: бинарные симметричные операторы (+, ==) лучше делать свободными функциями — тогда работают неявные преобразования для *обоих* операндов. Операторы, меняющие объект (+=, =, []), — методами.

14.3. Арифметические операторы и каноничная форма

Принято определять += как метод, а + — через него:

```

1 struct Vec {
2     double x, y;
3     Vec& operator+=(const Vec& o) { x += o.x; y += o.y; return *
4         this; }
5 };
6 Vec operator+(Vec a, const Vec& b) { return a += b; } // через +=

```

14.4. Операторы сравнения и spaceship (C++20)

До C++20 нужно было определять все шесть операторов сравнения. В C++20 достаточно оператора <=> (three-way comparison) — остальные генерируются:

```

1 #include <compare>
2 struct Point {
3     int x, y;
4     auto operator<=>(const Point&) const = default; // <, >, <=, >=
5     bool operator==(const Point&) const = default; // ==, !=
6 };

```

14.5. Оператор вывода <<

Перегружается **только** как свободная функция (левый операнд — std::ostream, не наш тип):

```

1 std::ostream& operator<<(std::ostream& os, const Point& p) {
2     return os << '(' << p.x << ", " << p.y << ')';
3 }
4 std::cout << Point{1, 2}; // (1, 2)

```

14.6. Оператор индексирования []

```
1 class Array {
2     int data_[100];
3 public:
4     int& operator[](size_t i)      { return data_[i]; } //
        запись
5     const int& operator[](size_t i) const { return data_[i]; } //
        чтение
6 };
```

14.7. Оператор вызова () — функциональные объекты

Класс с `operator()` называется **функтором** — его объект можно «вызывать»:

```
1 struct Adder {
2     int base;
3     int operator()(int x) const { return x + base; }
4 };
5 Adder add5{5};
6 add5(10); // 15 -- объект вызывается как функция
```

Функторы — основа лямбд и предикатов для алгоритмов STL.

14.8. Операторы инкремента

Различают префиксный и постфиксный (отличаются фиктивным `int`-параметром):

```
1 struct Counter {
2     int v = 0;
3     Counter& operator++()      { ++v; return *this; } //
        префиксный ++c
4     Counter operator++(int) { Counter t = *this; ++v; return t; }
        // c++
5 };
```

14.9. Операторы преобразования

```
1 struct Fraction {
2     int num, den;
3     explicit operator double() const { return double(num) / den; }
4 };
5 double d = static_cast<double>(Fraction{1, 2}); // 0.5
```

14.10. Что нельзя перегружать

Нельзя перегрузить: `::` (разрешение области), `.` (доступ к члену), `.*`, `?:` (тернарный), `sizeof`, `typeid`. Нельзя придумывать новые операторы или менять их арность/приоритет.

14.11. Умные указатели и операторы *, ->

Перегрузка `operator*` и `operator->` позволяет классу вести себя как указатель — так устроены `unique_ptr` и итераторы:

```
1 template<typename T>
2 class Ptr {
3     T* p_;
4 public:
5     T& operator*() const { return *p_; }
6     T* operator->() const { return p_; }
7 };
```

15. Шаблоны функций и классов. Специализация

15.1. Зачем нужны шаблоны

Шаблоны — механизм **обобщённого программирования**: написать код один раз для **любого типа**, а компилятор сгенерирует конкретные версии. Это статический полиморфизм, без runtime-затрат.

15.2. Шаблоны функций

```
1 template<typename T>
2 T max(T a, T b) {
3     return (a > b) ? a : b;
4 }
5 max(3, 5);           // T = int выведен ( изаргументов )
6 max(2.5, 1.5);       // T = double
7 max<int>(3, 5);      // T заданявно
```

15.2.1. Вывод аргументов шаблона (template argument deduction)

Компилятор сам определяет `T` по типам аргументов. Если вывод неоднозначен, тип указывают явно:

```
1 template<typename T> void f(T x);
2 f(42);           // T = int
3 f<double>(42);   // T = double явно
```

15.3. Шаблоны классов

```
1 template<typename T>
2 class Stack {
3     std::vector<T> data_;
4 public:
5     void push(const T& x) { data_.push_back(x); }
```



```

6     T pop() { T t = data_.back(); data_.pop_back(); return t; }
7     bool empty() const { return data_.empty(); }
8 };
9 Stack<int> si;           // стекцелых
10 Stack<std::string> ss; // стекстрок

```

15.3.1. Class Template Argument Deduction (CTAD, C++17)

С C++17 для шаблонов классов аргументы тоже могут выводиться:

```

1 std::vector v = {1, 2, 3}; // std::vector<int>, типвыведен
2 std::pair p{1, 2.0};      // std::pair<int, double>

```

15.4. Параметры шаблона

- **Типовые:** `template<typename T>`.
- **Нетиповые (NTTP):** значения, известные на этапе компиляции: `template<int N>`, `template<auto V>`.
- **Шаблонные:** шаблон как параметр шаблона.

```

1 template<typename T, size_t N>
2 class Array {
3     T data_[N]; // N -- константавременикомпиляции
4 public:
5     constexpr size_t size() const { return N; }
6 };
7 Array<int, 5> a; // массивна 5 int

```

15.5. Полная специализация

Особая реализация шаблона для **конкретного** типа:

```

1 template<typename T>
2 struct TypeName { static const char* get() { return "unknown"; }
3 };
4 template<> // полнаяспециализациядля bool
5 struct TypeName<bool> { static const char* get() { return "bool"; }
6 };
7 template<> // полнаяспециализациядля int
8 struct TypeName<int> { static const char* get() { return "int"; }
9 };

```

15.6. Частичная специализация

Специализация не для одного типа, а для **семейства** типов. Доступна **только** для **шаблонов классов** (не функций):

```
1 template<typename T>
2 struct IsPointer { static constexpr bool value = false; };
3
4 template<typename T>           // частичная:
   длялюбыхуказателей   T*
5 struct IsPointer<T*> { static constexpr bool value = true; };
6
7 IsPointer<int>::value;        // false
8 IsPointer<int*>::value;       // true
```

Для **функций** вместо частичной специализации используют перегрузку.

15.7. Двухфазная компиляция и typename/template

Шаблон компилируется в два прохода: сначала проверяется код, не зависящий от параметров, при инстанцировании — зависимый. Перед зависимым именем типа нужен `typename`:

```
1 template<typename T>
2 void f() {
3     typename T::value_type x;    // typename: говоримэто "тип"
4     typename std::vector<T>::iterator it;
5 }
```

15.8. Где определять шаблоны

Определение шаблона должно быть видимо в точке инстанцирования, поэтому шаблоны обычно пишут **целиком в заголовках** (а не разделяют на `.h/.cpp`).

15.9. Переменные-шаблоны и constexpr

```
1 template<typename T>
2 constexpr T pi = T(3.1415926535897932385);
3 auto x = pi<float>;    // 3.14159f
```

15.10. Концепты (C++20) — кратко

Концепты ограничивают допустимые типы и дают понятные ошибки (подробно — в разделе о метапрограммировании):

```
1 #include <concepts>
2 template<std::integral T>    // толькоцелочисленные T
3 T half(T x) { return x / 2; }
```

16. RAII. Умные указатели

16.1. Идиома RAII

RAII (Resource Acquisition Is Initialization) — ключевая идиома C++: **ресурс захватывается в конструкторе и освобождается в деструкторе**. Поскольку деструктор вызывается автоматически (в т. ч. при исключении), ресурс не утечёт.

```
1 class FileGuard {
2     FILE* f_;
3 public:
4     explicit FileGuard(const char* name) : f_(fopen(name, "r")) {}
5     ~FileGuard() { if (f_) fclose(f_); } // освобождение гарантировано
6     FileGuard(const FileGuard&) = delete; // не копируем владение
7     FILE* get() const { return f_; }
8 };
9
10 void process() {
11     FileGuard fg("data.txt");
12     // ... даже если здесь бросится исключение ...
13 } // ~FileGuard() вызовется и закроет файл в любом случае
```

Под RAII попадают: память, файлы, мьютексы (`std::lock_guard`), сокеты, соединения с БД — любой ресурс с парой «захват/освобождение».

16.2. Проблема сырых указателей

```
1 void leak() {
2     Widget* w = new Widget();
3     if (error()) return; // УТЕЧКА: delete не вызовется
4     delete w;
5 }
```

Умные указатели — это RAII для динамической памяти: они **сами** вызывают `delete`.

16.3. `std::unique_ptr` — единоличное владение

Владеет объектом эксклюзивно. Нельзя копировать (только перемещать). Нулевые накладные расходы по сравнению с сырым указателем.

```
1 #include <memory>
2 std::unique_ptr<Widget> p = std::make_unique<Widget>(args);
3 p->method(); // какобычный указатель
4 // auto q = p; // ОШИБКА: копирование запрещено
5 auto q = std::move(p); // передача владения : теперь p == nullptr
6 // при выходе из области видимости q сам сделает delete
```

Используйте `make_unique`, а не `new`: короче, безопаснее при исключениях, без дублирования типа.

16.3.1. unique_ptr для массивов и кастомный делитер

```
1 std::unique_ptr<int[]> arr = std::make_unique<int[]>(10); //  
    массив  
2 std::unique_ptr<FILE, decltype(&fclose)> f(fopen("x","r"), &fclose  
    );
```

16.4. std::shared_ptr — разделяемое владение

Несколько shared_ptr владеют одним объектом совместно. Внутри — **счётчик ссылок (reference count)**. Объект уничтожается, когда счётчик доходит до нуля.

```
1 std::shared_ptr<Widget> p1 = std::make_shared<Widget>();  
2 std::shared_ptr<Widget> p2 = p1; // счётчик = 2 копирование(  
    разрешено)  
3 std::cout << p1.use_count(); // 2  
4 p2.reset(); // счётчик = 1  
5 // объект жив , пока есть хоть один владелец
```

make_shared эффективнее: выделяет объект и управляющий блок (со счётчиком) одним аллокатором.

16.4.1. Накладные расходы

shared_ptr весит как два указателя (на объект и на управляющий блок), а изменение счётчика — **атомарная** операция (потокбезопасно, но не бесплатно). Если разделение не нужно — используйте unique_ptr.

16.5. std::weak_ptr — слабая ссылка

weak_ptr наблюдает за объектом shared_ptr, **не увеличивая счётчик**. Чтобы получить доступ, его «повышают» через lock().

```
1 std::shared_ptr<Widget> sp = std::make_shared<Widget>();  
2 std::weak_ptr<Widget> wp = sp; // невладеет  
3  
4 if (auto locked = wp.lock()) { // shared_ptr, если объект ещё жив  
5     locked->method();  
6 } else {  
7     // объекту уже уничтожен  
8 }
```

16.5.1. Решение проблемы циклических ссылок

Два shared_ptr, ссылающиеся друг на друга, создают **цикл** — счётчики никогда не дойдут до нуля, память утечёт. Один из указателей делают weak_ptr:

```

1 struct Node {
2     std::shared_ptr<Node> next;    // владеетследующим
3     std::weak_ptr<Node> prev;    // НЕвладеетпредыдущим  ->
        нетцикла
4 };

```

16.6. Сравнение умных указателей

	Владение	Когда использовать
unique_ptr	единоличное	по умолчанию; один владлец
shared_ptr	разделяемое	несколько владельцев
weak_ptr	не владеет	наблюдение, разрыв циклов

16.7. Передача умных указателей в функции

- Функция **забирает владение** — unique_ptr<T> по значению (передавать через std::move).
- Функция **разделяет владение** — shared_ptr<T> по значению.
- Функция **только использует** объект — передавайте T* или T&, а не умный указатель.

```

1 void use(const Widget& w);           // простоиспользует  --
        ссылка
2 void take(std::unique_ptr<Widget> w); // забираетвладение
3 take(std::move(p));

```

17. Исключения в C++

17.1. Ключевое слово throw

17.1.1. Базовый синтаксис

Рассмотрим функцию деления:

```

1 #include <stdexcept>
2
3 int divide(int a, int b) {
4     if (b == 0) {
5         throw std::invalid_argument("division by zero");
6     }
7     return a / b;
8 }

```

При делении на ноль поведение программы неопределено, поэтому используется ключевое слово `throw`. Бросать можно **любое выражение**, однако принято бросать объекты стандартных исключений из `<stdexcept>`.

`throw` — один из способов выйти из функции наряду с `return`.

17.1.2. Что можно бросать?

Любой копируемый или перемещаемый объект:

```
1 throw 42; // int
2 throw std::string("error"); // std::string
3 throw std::runtime_error("fail"); // стандартноеисключение
4 throw 3.14; // double
5 throw std::vector<int>{1, 2, 3}; // дажевекторно (
    нестоит )
6
7 struct MyError { int code; std::string msg; };
8 throw MyError{404, "not found"}; //
```

пользовательскаяструктура

Рекомендация: бросайте только наследников `std::exception`. Это позволяет единообразно ловить все ошибки через `catch(const std::exception&)`.

17.1.3. Что происходит при `throw`?

1. Объект исключения **копируется** (или перемещается) в специальную область памяти, управляемую рантаймом.
2. Начинается **раскрутка стека** (stack unwinding).
3. Управление передаётся первому подходящему блоку `catch`.
4. Если подходящего `catch` нет — вызывается `std::terminate()`.

17.1.4. `throw` как выражение

`throw` — это выражение типа `void`. Его можно использовать в тернарном операторе и других контекстах:

```
1 int safe_divide(int a, int b) {
2     return b != 0 ? a / b : throw std::invalid_argument("zero
    divisor");
3 }
4
5 // Влямбде :
6 auto get_or_throw = [](std::optional<int> opt) -> int {
7     return opt.has_value() ? *opt : throw std::runtime_error("
    empty");
8 };
```

17.2. Раскрутка стека (Stack Unwinding)

17.2.1. Определение

При выбросе исключения происходит **раскрутка стека** (stack unwinding) — автоматическое уничтожение всех локальных объектов в обратном порядке их создания, с вызовом деструкторов, по цепочке вызовов от точки `throw` до подходящего блока `catch`.

17.2.2. Пошаговая механика

```
1  #include <iostream>
2  #include <stdexcept>
3
4  struct Guard {
5      std::string name;
6      Guard(std::string n) : name(std::move(n)) {
7          std::cout << "  + " << name << "\n";
8      }
9      ~Guard() {
10         std::cout << "  - " << name << "\n";
11     }
12 };
13
14 void f3() {
15     Guard g("f3_local");
16     throw std::runtime_error("boom"); // (1) Бросаем исключение
17     // Код ниже НИКОГДА не выполнится
18     std::cout << "after throw\n";
19 }
20
21 void f2() {
22     Guard g("f2_local");
23     f3(); // (2) Вызываем f3
24     std::cout << "after f3\n"; // Не выполнится
25 }
26
27 void f1() {
28     Guard g("f1_local");
29     f2(); // (3) Вызываем f2
30     std::cout << "after f2\n"; // Не выполнится
31 }
32
33 int main() {
34     try {
35         f1();
36     } catch (const std::exception& e) {
37         std::cout << "Caught: " << e.what() << "\n";
38     }
39 }
```

Вывод:

```

+ f1_local
+ f2_local
+ f3_local
- f3_local      <- раскрутка: уничтожается f3_local
- f2_local      <- раскрутка: уничтожается f2_local
- f1_local      <- раскрутка: уничтожается f1_local
Caught: boom

```

Обратите внимание: деструкторы вызываются в **обратном порядке** создания (LIFO), а строки "after ..." не выполняются — управление «перепрыгивает» через них.

17.2.3. Визуализация стека

```

Стеквызововвмомент
  throw:

| f3() | <- throw runtime_error("boom")
| f2() | ~Guard("f3_local")
| f1() | ~Guard("f2_local")
| main() | ~Guard("f1_local")
+-----+ catch (const std::exception& e) <- сюдапопадаем

```

Рантайм «разматывает» стек снизу вверх: для каждого кадра уничтожает локальные объекты, затем переходит к следующему, пока не найдёт catch.

17.2.4. Что уничтожается, а что нет

Объект	Уничтожается?	Почему
Локальные переменные на стеке	Да	RAII
Объекты, созданные new	Нет	Нет владельца на стеке
std::unique_ptr	Да	Деструктор вызовет delete
std::shared_ptr	Да	Деструктор уменьшит счётчик
Сырые указатели (int*)	Да*	Но память не освобождается!

*Переменная-указатель уничтожается, но delete для неё никто не вызывает — это **утечка памяти**.

```

1 void leak_example() {
2     int* raw = new int(42);           // Выделили память
3     std::unique_ptr<int> safe(new int(43)); // RAII обёртка
4
5     throw std::runtime_error("oops");
6
7     delete raw; // НИКОГДА не выполнится -> утечка!
8     // safe -- деструктор unique_ptr вызовет delete автоматически
9 }

```

Мораль: именно поэтому RAII критически важен. Любой ресурс (память, файл, мьютекс, соединение) должен быть обернут в объект с деструктором.

17.2.5. Частично сконструированные объекты

Если исключение вылетает **из конструктора**, деструктор этого объекта **не вызывается** (объект не считается созданным). Но деструкторы уже инициализированных полей **вызываются**:

```
1 struct Resource {
2     Resource(int id) { std::cout << "R(" << id << ")\n"; }
3     ~Resource()      { std::cout << "~R\n"; }
4 };
5
6 struct Widget {
7     Resource a, b, c;
8
9     Widget() : a(1), b(2), c(3) {
10        // Допустим, конструктор c бросил исключение
11        // Тогда:
12        //   a -- уничтожается был ( создан )
13        //   b -- уничтожается был ( создан )
14        //   c -- НЕ уничтожается не ( был до конца создан )
15        //   ~Widget() -- НЕ вызывается (Widget не был создан )
16    }
17};
```

```
1 struct Dangerous {
2     Resource a{1};
3     Resource b{2};
4
5     Dangerous() {
6         Resource* raw = new Resource(3);
7         throw std::runtime_error("oops");
8         delete raw; // Утечка! ~Dangerous() не вызовется ,
9                    // но ~a и ~b вызовутся
10    }
11};
```

17.2.6. Раскрутка стека и массивы

При раскрутке массива деструкторы вызываются только для **уже сконструированных** элементов:

```
1 struct Item {
2     static int count;
3     Item() {
4         if (++count == 4) throw std::runtime_error("4th item");
5         std::cout << "Item(" << count << ")\n";
6     }
7     ~Item() { std::cout << "~Item\n"; }
8 };
9 int Item::count = 0;
10
11 void test() {
```

```

12     Item arr[5];
13     // Создаётся Item(1), Item(2), Item(3)
14     // Item(4) бросает исключение
15     // Раскрутка: ~Item для arr[2], arr[1], arr[0] в(
        обратном порядке )
16     // arr[3] и arr[4] не были созданы -- деструкторы не вызываются
17 }

```

17.2.7. Когда раскрутка НЕ происходит

- **Не пойманное исключение:** если нигде нет подходящего `catch`, стандарт гарантирует только вызов `std::terminate()`. Раскрутка стека *implementation-defined* — может произойти, а может и нет.
- **`std::abort()` / `std::_Exit()`:** немедленное завершение без раскрутки.
- **Нарушение `noexcept`:** если функция помечена `noexcept` и из неё вылетает исключение — `std::terminate()` без гарантированной раскрутки.

17.3. try / catch

17.3.1. Базовый синтаксис

```

1  #include <iostream>
2  #include <stdexcept>
3
4  int main() {
5      try {
6          divide(1, 0);
7      } catch (const std::exception& e) {
8          std::cout << e.what() << "\n";
9      }
10 }

```

- Метод `what()` определён в `std::exception` и возвращает строку с описанием ошибки — ту самую, что была передана при создании объекта исключения.
- Исключение следует ловить **по константной ссылке**, чтобы избежать лишнего копирования и сохранить полиморфизм.

17.3.2. Множественные блоки `catch`

```

1  try {
2      do_something();
3  } catch (const std::out_of_range& e) {
4      // Обработка выхода за границы
5      std::cerr << "Index error: " << e.what() << "\n";
6  } catch (const std::invalid_argument& e) {

```

```

7 // Обработканеверногоаргумента
8 std::cerr << "Bad argument: " << e.what() << "\n";
9 } catch (const std::exception& e) {
10 // Всеостальныестандартныеисключения
11 std::cerr << "Error: " << e.what() << "\n";
12 } catch (...) {
13 // Абсолютнолюбоеисключениевключая ( int, string итд ..)
14 std::cerr << "Unknown error\n";
15 }

```

Правила:

- Блоки catch проверяются **сверху вниз**, побеждает **первый подходящий**.
- catch(...) ловит **всё**, поэтому должен быть последним.
- Более специфичные обработчики ставятся **перед** более общими.

17.3.3. Ловля по значению vs по ссылке

```

1 // ПЛОХО: ловляпозначению -- срезка (slicing)!
2 try {
3     throw std::runtime_error("specific error");
4 } catch (std::exception e) {
5     // e -- копия, типСРЕЗАНдо std::exception
6     // e.what() можетвернутьнето , чтоожидали
7 }
8
9 // ХОРОШО: ловляпоконстантнойссылке
10 try {
11     throw std::runtime_error("specific error");
12 } catch (const std::exception& e) {
13     // e -- ссылканаисходныйобъект , полиморфизмсохранён
14     // e.what() вернёт "specific error"
15 }

```

При ловле по значению происходит **object slicing**: если бросили наследника, а поймали базовый тип, часть данных наследника **теряется**.

17.3.4. Вложенные try/catch

```

1 void process() {
2     try {
3         try {
4             throw std::runtime_error("inner");
5         } catch (const std::logic_error& e) {
6             std::cout << "logic: " << e.what() << "\n"; //
7             // runtime_error непойманвнутренним catch -- летитдальше
8             Неподходит
9         } catch (const std::exception& e) {

```

```

10         std::cout << "outer: " << e.what() << "\n"; // "outer:
           inner"
11     }
12 }

```

17.4. Приведение типов в catch

В catch **не работают** обычные неявные преобразования и правила перегрузки. Доступны только:

- Добавление const / приведение к ссылке
- Приведение типа потомка к типу родителя

```

1 try {
2     throw 1; // int
3 } catch (double d) {
4     std::cout << "double\n"; // не поймает : int -> double
                           не работает
5 } catch (long long l) {
6     std::cout << "long long\n"; // не поймает : int -> long long
                           не работает
7 } catch (...) {
8     std::cout << "other\n"; // поймает
9 }

```

17.4.1. Наследование в catch

Блоки catch проверяются **сверху вниз**, побеждает первый подходящий:

```

1 struct Mom {};
2 struct Son : Mom {};
3
4 int main() {
5     try {
6         throw Son{};
7     } catch (const Mom& m) {
8         std::cout << "Mom\n"; // поймает: Son приводится к Mom
9     } catch (const Son& s) {
10        std::cout << "Son\n"; // сюда не дойдём
11    } catch (...) {
12        std::cout << "other\n";
13    }
14 }

```

Вывод: Mom. Именно поэтому обработчики потомков нужно ставить **перед** обработчиками родителей, иначе они никогда не сработают.

17.4.2. Правильный порядок

```
1 try {  
2     throw Son{};  
3 } catch (const Son& s) {  
4     std::cout << "Son\n"; // Теперьпоймает !  
5 } catch (const Mom& m) {  
6     std::cout << "Mom\n"; // Ловитостальныхнаследников Mom  
7 } catch (...) {  
8     std::cout << "other\n";  
9 }
```

17.5. Где встречаются исключения в стандартной библиотеке

Операция	Исключение
<code>vector::at(i)</code>	<code>std::out_of_range</code> при <code>i >= size()</code>
<code>operator new</code>	<code>std::bad_alloc</code> при нехватке памяти
<code>dynamic_cast<T&>(ref)</code>	<code>std::bad_cast</code> при неудачном приведении
<code>std::stoi("abc")</code>	<code>std::invalid_argument</code> (не число)
<code>std::stoi("999999999999")</code>	<code>std::out_of_range</code> (переполнение)
<code>map::at(key)</code>	<code>std::out_of_range</code> при отсутствии ключа
<code>optional::value()</code>	<code>std::bad_optional_access</code> если пуст
<code>variant::get<T>()</code>	<code>std::bad_variant_access</code> при неверном типе
<code>thread::join()</code> на отсоединённом	<code>std::system_error</code>

17.6. Runtime-ошибки vs Исключения C++

Это принципиально разные вещи.

17.6.1. Три вида runtime-ошибок на уровне ОС

Сигнал	Типичные причины
SIGSEGV (Segfault)	Выход за пределы массива, разыменование nullptr, переполнение стека
SIGFPE (Floating Point Exception)	Деление целого на ноль, переполнение целых
SIGABRT (Aborted)	Вызов <code>std::abort()</code> , <code>std::terminate()</code> , падение <code>assert</code>

Эти ошибки **не являются** исключениями C++ и **не ловятся** через `catch`:

```
1 int arr[5];
2 try {
3     arr[10] = 42; // SIGSEGV -- catch не поймает
4 } catch (...) {
5     // сюда мы НЕ попадём
6 }
7
8 try {
9     int x = 1 / 0; // SIGFPE -- тоже не исключение C++
10 } catch (...) {
11     // сюда мы НЕ попадём
12 }
```

Важно: деление на ноль — это исключение на уровне *процессора*, не языка. Процессор обращается к ОС, которая посылает SIGFPE. Это совпадение терминологии, но разные уровни абстракции.

17.6.2. Пути к `std::terminate()`

- Не пойманное исключение (uncaught exception)
- Явный вызов `std::terminate()` или `std::abort()`
- Упавший `assert` (он напрямую вызывает `abort()`)
- Исключение из `noexcept`-функции
- Исключение из деструктора во время раскрутки стека
- Два одновременных активных исключения (double exception)

С точки зрения ОС программа, завершившаяся из-за не пойманного C++-исключения, получает вердикт SIGABRT.

Также стоит отметить, что `std::runtime_error` — это *частный случай* `std::exception`, то есть обычное C++-исключение, не имеющее отношения к SIGFPE/SIGSEGV/SIGABRT.

17.7. Копирование при броске исключения

```
1  #include <iostream>
2
3  struct A {
4      A()          { std::cout << "A()\n"; }
5      ~A()         { std::cout << "~A()\n"; }
6      A(const A&) { std::cout << "A(copy)\n"; }
7  };
8
9  void f(int x) {
10     A a;
11     if (x == 0) {
12         throw a; // объект копируется в специальную память
13     }
14 }
15
16 int main() {
17     try {
18         f(0);
19     } catch (...) {
20         std::cout << "caught!\n";
21     }
22 }
```

Порядок вывода:

1. A() — создание a на стеке
 2. A(copy) — копирование в специальную память при throw
 3. ~A() — уничтожение a при раскрутке стека
 4. caught!
 5. ~A() — уничтожение копии при выходе из catch
- catch(A a) — ещё одно копирование из дин. памяти на стек.
 - catch(A& a) — работаем прямо с объектом исключения, без копирования.
 - catch(const A& a) — то же, но без возможности модификации (рекомендуемый вариант).

17.7.1. Где хранится объект исключения?

Стандарт не специфицирует точное место хранения. На практике — в **динамической памяти**. Исключение: `std::bad_alloc` — при нехватке памяти нельзя выделить новую, поэтому компилятор заранее резервирует буфер в **статической памяти** (*emergency buffer*).

Исключения дороги по стоимости: выделение динамической памяти происходит в любом случае. Поэтому их не стоит использовать в горячих путях кода.

17.7.2. Оптимизация: throw временного объекта

```
1 // Лишнее копирование :
2 void bad() {
3     std::runtime_error err("fail");
4     throw err; // Копирование err в специальную память
5 }
6
7 // Без лишнего копирования :
8 void good() {
9     throw std::runtime_error("fail"); //
10    Объект создаётся сразу в спец. памяти
11                                     //
12                                     // Компилятор может применить
13                                     // copy elision
14 }
```

17.7.3. Повторный проброс

```
1 try {
2     f(0);
3 } catch (const A& a) {
4     throw; // прокидываем то же исключение дальше без
5           // копирования)
6           // throw a; // создаём НОВЫЙ объект -- возможна срезка типа !
7 }
```

Критическая разница:

- `throw;` — пробрасывает **оригинальный** объект исключения. Тип сохраняется. Нет копирования.
- `throw a;` — создаёт **новый** объект, копируя `a`. Если реальный тип — наследник, происходит **object slicing**!

```
1 struct Base { virtual const char* what() const { return "Base"; }
2 };
3
4 struct Derived : Base { const char* what() const override { return
5     "Derived"; } };
6
7 void demonstrate_slicing() {
8     try {
9         throw Derived{};
10    } catch (const Base& e) {
11        std::cout << e.what() << "\n"; // "Derived" полиморфизм()
12        // throw;                       // Пробросит Derived --
13        // правильно
14        throw e;                        // Пробросит Base --
15        СРЕЗКА!
16    }
17 }
```


Правило: всегда используйте `throw`; без аргумента для повторного про-броса.

Если исключение не поймано нигде — гарантируется только вызов `std::terminate()`. Деструкторы при этом могут не вызваться.

17.8. Спецификатор `noexcept`

17.8.1. Что такое `noexcept`?

`noexcept` — спецификатор, обещающий компилятору, что функция **не бросает исключений**:

```
1 void safe_function() noexcept {
2     // Если отсюда вылетит исключение    -- std::terminate()
3 }
4
5 int add(int a, int b) noexcept {
6     return a + b; // Действительно не бросает
7 }
```

17.8.2. Зачем это нужно?

1. **Оптимизация:** компилятор может генерировать более эффективный код, зная, что исключений не будет (не нужна таблица раскрутки для этой функции).
2. **Семантика `std::vector`:** при реаллокации вектор использует `std::move_if_noexcept`. Если конструктор перемещения помечен `noexcept` — элементы перемещаются. Если нет — **копируются** (для strong exception safety).
3. **Документация:** явное указание контракта функции.

```
1 class MyString {
2 public:
3     // Без noexcept -- vector будет КОПИРОВАТЬ при реаллокации
4     MyString(MyString&& other) { /* ... */ }
5
6     // С noexcept -- vector будет ПЕРЕМЕЩАТЬ быстрее (!)
7     MyString(MyString&& other) noexcept { /* ... */ }
8 };
9
10 std::vector<MyString> vec;
11 vec.push_back(MyString()); // Реаллокация: move или copy?
```

17.8.3. Условный `noexcept`

`noexcept` может принимать выражение:

```

1 template<typename T>
2 void swap_values(T& a, T& b) noexcept(noexcept(T(std::move(a)))) {
3     T tmp = std::move(a);
4     a = std::move(b);
5     b = std::move(tmp);
6 }
7 // noexcept, если конструктор перемещения T -- noexcept

```

`noexcept(expr)` как оператор возвращает `true/false`:

```

1 static_assert(noexcept(42 + 1));           // true --
    арифметика не бросает
2 static_assert(!noexcept(std::stoi("42"))); // false -- stoi
    может бросить
3
4 void f() noexcept;
5 static_assert(noexcept(f()));             // true -- f помечена
    noexcept

```

17.8.4. Что будет, если `noexcept`-функция бросит?

```

1 void oops() noexcept {
2     throw std::runtime_error("surprise!"); // Компилируется!
3 }
4
5 int main() {
6     try {
7         oops();
8     } catch (...) {
9         // Сюда мы НЕ попадём !
10    }
11    // Программа завершится через std::terminate()
12 }

```

Компилятор **не проверяет** тело `noexcept`-функции на наличие `throw`. `noexcept` — это **обещание** программиста, а не проверка компилятора. Нарушение — `std::terminate()`.

17.8.5. Что помечать `noexcept`?

Функция	<code>noexcept</code> ?
Деструкторы	Да (по умолчанию с C++11)
Move-конструкторы	Да, если возможно
Move-присваивание	Да, если возможно
<code>swap()</code>	Да, если возможно
Функции, работающие с памятью	Зависит
Функции с внешними вызовами	Обычно нет

Важно: деструкторы в C++11 и новее **неявно** `noexcept`. Исключение из деструктора — почти всегда `std::terminate()`.

17.9. Исключения в деструкторах

17.9.1. Почему нельзя бросать из деструктора?

Деструкторы вызываются при раскрутке стека. Если во время раскрутки (уже есть активное исключение) деструктор бросит **второе** исключение — в C++ не может быть двух активных исключений одновременно, и вызывается `std::terminate()`:

```
1 struct Bad {
2     ~Bad() noexcept(false) {
3         throw std::runtime_error("from destructor");
4     }
5 };
6
7 void example() {
8     try {
9         Bad b;
10        throw std::runtime_error("first");
11        // Начинается раскрутка стека
12        // Вызывается ~Bad(), который бросает ВТОРОЕ исключение
13        // -> std::terminate()!
14    } catch (...) {
15        // Сюда не дойдем
16    }
17 }
```

17.9.2. Как обрабатывать ошибки в деструкторе?

```
1 class FileWriter {
2     FILE* file_;
3 public:
4     ~FileWriter() {
5         // Вариант 1: проглотить исключение обычный (подход)
6         try {
7             if (file_ && fclose(file_) != 0) {
8                 // Логируем ошибку, но НЕ бросаем
9                 std::cerr << "Warning: failed to close file\n";
10            }
11        } catch (...) {
12            // Никогда не позволяем исключению покинуть деструктор
13        }
14    }
15
16    // Вариант 2: явный метод close() --
17    // пользователь вызовет заранее
18    void close() {
19        if (file_ && fclose(file_) != 0) {
20            throw std::runtime_error("close failed");
21        }
22        file_ = nullptr;
23    }
24 }
```

```
23 };
```

17.9.3. Функция `std::uncaught_exceptions()`

C++17 добавил `std::uncaught_exceptions()` (с «s!»), которая возвращает **количество** активных непойманных исключений. Это позволяет деструктору узнать, вызван ли он в процессе раскрутки:

```
1  class ScopeGuard {
2      int exceptions_on_entry_;
3      std::function<void()> on_success_;
4      std::function<void()> on_failure_;
5  public:
6      ScopeGuard(std::function<void()> success, std::function<void()>
7          > failure)
8          : exceptions_on_entry_(std::uncaught_exceptions())
9            , on_success_(std::move(success))
10           , on_failure_(std::move(failure))
11      {}
12
13      ~ScopeGuard() {
14          if (std::uncaught_exceptions() > exceptions_on_entry_) {
15              on_failure_(); // Деструктор вызван из-за исключения
16          } else {
17              on_success_(); // Нормальный выход из области видимости
18          }
19      }
20
21  void test() {
22      ScopeGuard guard(
23          []{ std::cout << "Success!\n"; },
24          []{ std::cout << "Failure!\n"; }
25      );
26      throw std::runtime_error("oops"); // "Failure!"
27  }
```

17.10. Function-try-block

Особый синтаксис `try/catch` для **всего тела функции**, включая список инициализации конструктора:

```
1  class Connection {
2      DatabaseHandle db_;
3      Logger log_;
4
5  public:
6      Connection(const std::string& url)
7          try : db_(url), log_("connection.log") // Если db_ или log_
8              бросят --
```

```

8      {                                     // поймем ниже
9          db_.connect();
10     }
11     catch (const std::exception& e) {
12         std::cerr << "Failed to create Connection: " << e.what()
13             << "\n";
14         // ВАЖНО:
15         // для конструктора исключение автоматически перебрасывается
16         // !
17         // Объект НЕ был создан, вернуть его нельзя
18     }
19 };

```

Особенности function-try-block в конструкторах:

- В catch блоке **нельзя** обратиться к полям объекта — они уже могут быть уничтожены.
- Исключение **автоматически перебрасывается** при выходе из catch. Подавить его невозможно — объект не был создан.
- Можно заменить исключение на другое через `throw other;`.

Для обычных функций function-try-block просто оборачивает всё тело:

```

1  int parse(const std::string& s)
2  try {
3      return std::stoi(s);
4  }
5  catch (const std::exception& e) {
6      std::cerr << "Parse failed: " << e.what() << "\n";
7      return -1; // Для обычных функций -- можно вернуть значение
8  }

```

17.11. Гарантии безопасности исключений (Exception Safety)

При проектировании функций и классов важно определить, какие **гарантии** они предоставляют в случае исключения.

17.11.1. Четыре уровня гарантий

Гарантия	Суть	Пример
No-throw	Функция никогда не бросает исключений	Деструкторы, <code>swap()</code> , <code>noexcept</code> -функции
Strong (строгая)	При исключении состояние объекта откатывается к исходному («всё или ничего»)	<code>vector::push_back</code> , <code>std::atomic</code> операции
Basic (базовая)	При исключении объект в <i>валидном</i> , но <i>неопределённом</i> состоянии. Утечек нет	<code>std::string::operator=</code> (если аллокация не удалась)
No guarantee	Никаких гарантий. Возможны утечки и повреждение данных	Код без RAII

17.11.2. Пример: строгая гарантия в `push_back`

```

1  template<typename T>
2  void vector<T>::push_back(const T& value) {
3      if (sz_ == cap_) {
4          // 1. Выделяем новую память
5          T* newarr = allocate(cap_ * 2); // Может бросить
6          bad_alloc
7
8          // 2. Копируем/перемещаем/элементы
9          try {
10             copy_or_move(arr_, newarr, sz_); // Может бросить
11         } catch (...) {
12             deallocate(newarr); // Откатываемся:
13             освобождаем новую память
14             throw; // Пробрасываем --
15             старый вектор цел !
16         }
17
18         // 3. Успех: заменяем старое на новое
19         deallocate(arr_);
20         arr_ = newarr;
21         cap_ *= 2;
22     }
23 }
```

```

21     new (arr_ + sz_) T(value); // Может бросить --
    новектор в валидном состоянии
22     ++sz_;
23 }

```

17.11.3. Copy-and-swap idiom — строгая гарантия для operator=

```

1  class MyString {
2      char* data_;
3      size_t size_;
4  public:
5      // Строгая гарантия через copy-and-swap:
6      MyString& operator=(MyString other) { // (1) Копия --
        может бросить
7          swap(other); // (2) Swap --
        noexcept
8          return *this; // (3)
        Старые данные в other
9      } // other
        уничтожится в деструкторе
10
11     void swap(MyString& other) noexcept {
12         std::swap(data_, other.data_);
13         std::swap(size_, other.size_);
14     }
15 };

```

Идея: копирование (шаг 1) может бросить, но исходный объект ещё не тронут. swap (шаг 2) гарантированно не бросает. Если шаг 1 бросил — исходный объект остался в прежнем состоянии (строгая гарантия).

17.12. std::exception_ptr — исключения между потоками

std::exception_ptr позволяет **захватить** исключение, сохранить его и **перебросить** позже — в том числе в другом потоке.

17.12.1. Основные функции

Функция	Описание
std::current_exception()	Захватывает текущее исключение (внутри catch) и возвращает exception_ptr
std::rethrow_exception(ptr)	Перебрасывает исключение, хранимое в ptr
std::make_exception_ptr(e)	Создаёт exception_ptr из объекта e без входа в catch

17.12.2. Пример: передача исключения между потоками

```
1  #include <exception>
2  #include <thread>
3  #include <iostream>
4
5  void worker(std::exception_ptr& eptr) {
6      try {
7          // Какаято- работа, которая может бросить
8          throw std::runtime_error("error in worker thread");
9      } catch (...) {
10         eptr = std::current_exception(); // Захватываем исключение
11     }
12 }
13
14 int main() {
15     std::exception_ptr eptr = nullptr;
16
17     std::thread t(worker, std::ref(eptr));
18     t.join();
19
20     // Теперь в основном потоке :
21     if (eptr) {
22         try {
23             std::rethrow_exception(eptr); // Перебрасываем
24         } catch (const std::exception& e) {
25             std::cout << "From worker: " << e.what() << "\n";
26             // "From worker: error in worker thread"
27         }
28     }
29 }
```

17.12.3. Пример: сбор нескольких исключений

```
1  #include <exception>
2  #include <vector>
3
4  std::vector<std::exception_ptr> errors;
5
6  void process_item(int item) {
7      try {
8          if (item % 3 == 0)
9              throw std::runtime_error("bad item: " + std::to_string(
10                 item));
11         // Обычная обработка ...
12     } catch (...) {
13         errors.push_back(std::current_exception());
14     }
15 }
```



```

16 int main() {
17     for (int i = 0; i < 10; ++i) {
18         process_item(i);
19     }
20
21     std::cout << "Errors: " << errors.size() << "\n";
22     for (auto& eptr : errors) {
23         try {
24             std::rethrow_exception(eptr);
25         } catch (const std::exception& e) {
26             std::cout << "    - " << e.what() << "\n";
27         }
28     }
29 }

```

17.13. `std::nested_exception` — вложенные исключения

Иногда при обработке одного исключения возникает другое. C++11 позволяет **вложить** оригинальное исключение внутрь нового:

```

1  #include <exception>
2  #include <stdexcept>
3  #include <iostream>
4
5  void low_level() {
6      throw std::runtime_error("disk I/O error");
7  }
8
9  void mid_level() {
10     try {
11         low_level();
12     } catch (...) {
13         // std::throw_with_nested
14         // оборачивает текущее исключение в новое
15         std::throw_with_nested(
16             std::runtime_error("mid_level failed")
17         );
18     }
19 }
20
21 void high_level() {
22     try {
23         mid_level();
24     } catch (...) {
25         std::throw_with_nested(
26             std::runtime_error("high_level failed")
27         );
28     }
29 }

```

17.13.1. Распаковка вложенных исключений

```
1 void print_exception_chain(const std::exception& e, int depth = 0)
2 {
3     std::cout << std::string(depth * 2, ' ') << e.what() << "\n";
4     try {
5         std::rethrow_if_nested(e); // Перебрасывает вложенное,
6         // если есть
7     } catch (const std::exception& nested) {
8         print_exception_chain(nested, depth + 1);
9     } catch (...) {
10        std::cout << std::string((depth + 1) * 2, ' ') << "unknown
11        \n";
12    }
13 }
14
15 int main() {
16     try {
17         high_level();
18     } catch (const std::exception& e) {
19         print_exception_chain(e);
20     }
21 }
```

Вывод:

```
high_level failed
  mid_level failed
    disk I/O error
```

Это аналог «exception chaining» в Java и Python.

17.14. Стоимость исключений: zero-cost exception model

Современные компиляторы (GCC, Clang) используют **table-based** (zero-cost) модель исключений:

	Нормальный путь (без исключений)	При броске исключения
Накладные расходы	Нулевые. Никаких дополнительных инструкций	Высокие: поиск по таблицам, раскрутка стека
Размер бинарника	Увеличивается (таблицы раскрутки хранятся в секции <code>.eh_frame</code>)	—

Принцип: «платите только когда бросаете». На нормальном пути исполнения нет вообще никаких затрат — ни проверок, ни сохранения контекста. Но когда исключение летит, стоимость **очень высокая** (в 10–100 раз дороже обычного `return`).

Бросок исключения приблизительно

():

1. Выделение памяти для объекта исключения ~100 нс
 2. Поиск обработчика по таблицам (DWARF unwind) ~1–10 мкс
 3. Раскрутка стека вызовов (деструкторов) ~1–10 мкс
- Итого
: ~2–20 мкс Обычный

return: ~1 нс

Вывод: исключения — для **исключительных** ситуаций, а не для управления потоком выполнения. Для ожидаемых ошибок (парсинг, валидация) используйте `std::expected` или коды возврата.

17.15. Итого: чек-лист по исключениям

Правило	Почему
Бросайте наследников <code>std::exception</code>	Единообразная обработка
Ловите по <code>const&</code>	Нет копирования, нет срезки
Пробрасывайте через <code>throw;</code> (без аргумента)	Сохранение типа, нет срезки
Помечайте деструкторы <code>noexcept</code>	Они и так <code>noexcept</code> с C++11, но не бросайте из них
Помечайте <code>move</code> -конструкторы <code>noexcept</code>	Иначе <code>vector</code> будет копировать, а не перемещать
Используйте RAII для ресурсов	Раскрутка стека вызывает только деструкторы
Не используйте исключения в горячих путях	Бросок в ~1000× дороже <code>return</code>
Обработчики потомков — перед родителями	<code>catch</code> проверяется сверху вниз

18. Контейнеры и итераторы

18.1. `std::vector`: реализация изнутри

Начнём с устройства `std::vector` и в первую очередь рассмотрим реализацию метода `push_back`.

Структура вектора состоит из трёх полей: указателя на данные `arr_`, размера `sz_` и ёмкости `cap_`.

18.1.1. Почему нельзя писать `new T[newcap]`

При реаллокации наивная реализация могла бы выглядеть так:

```
1 T* newarr = new T[newcap]; // проблема!
```

Это не работает по двум причинам:

- Мы не знаем, есть ли у `T` конструктор по умолчанию.
- Нам нужно выделить *сырую* память и заполнять её по мере надобности, а не создавать все объекты сразу.

18.1.2. Placement new

Решение — выделить сырую память через `char`, а объекты создавать по месту с помощью **placement new**:

```
1 new (ptr) T(args...); // создаёт объект T по адресу ptr
```

Почему нельзя просто написать `newarr[i] = arr_[i]`? Потому что `newarr` указывает на сырые байты — объекта `T` там ещё нет, и присваивать можно только существующему объекту. `memset` тоже не подходит: он не работает корректно, если объект содержит указатель или ссылку на своё собственное поле (например, `std::string` с small-string optimization или декартово дерево).

18.1.3. Почему нельзя `delete[] arr_`

При освобождении старой памяти нельзя писать `delete[] arr_`: это вызовет деструктор для каждого из `cap_` слотов, хотя реальных объектов там только `sz_`. Нужно:

1. вручную вызвать деструктор для каждого существующего объекта;
2. освободить сырую память через `delete[] reinterpret_cast<char*>(arr_)`.

18.1.4. Реализация `reserve`

```
1 template<typename T>
2 class vector {
3     T* arr_;
4     size_t sz_;
5     size_t cap_;
```

```

6
7     void reserve(size_t newcap) {
8         T* newarr = reinterpret_cast<T*>(new char[newcap * sizeof(
9             T)]);
10        size_t i = 0;
11        try {
12            for (; i < sz_; ++i)
13                new (newarr + i) T(arr_[i]); // placement new
14        } catch (...) {
15            for (size_t k = 0; k < i; ++k)
16                (newarr + k)->~T();
17            delete[] reinterpret_cast<char*>(newarr);
18            throw;
19        }
20        for (size_t k = 0; k < sz_; ++k)
21            (arr_ + k)->~T();
22        delete[] reinterpret_cast<char*>(arr_);
23        arr_ = newarr;
24        cap_ = newcap;
25    }

```

Обработка исключений в `reserve`:

- Если `new char[...]` бросает `std::bad_alloc` — мы просто не выделили память, ничего не испорчено, вылетаем вверх.
- Если `placement new` бросает исключение в конструкторе на шаге `i` — нужно уничтожить уже созданные объекты (с 0 по `i-1`), освободить `newarr` и перебросить исключение.

18.1.5. Реализация `push_back`

```

1     void push_back(const T& value) {
2         if (sz_ == cap_)
3             reserve(cap_ > 0 ? cap_ * 2 : 1);
4         new (arr_ + sz_) T(value);
5         ++sz_;
6     }
7 }; // конец class vector

```

18.2. `vector<bool>`: специализация с битовым хранением

`vector<bool>` — это печально известная специализация, которая ломает многие интуитивные ожидания от вектора.

Идея: хранить элементы побитово, упаковывая по 8 штук в `char`. Это экономит память, но делает невозможным взять ссылку `bool&` на отдельный элемент — ссылка не может указывать на отдельный бит.

18.2.1. Прокси-класс BitReference

Чтобы сохранить синтаксис `v[i] = true`, вводится прокси-класс:

```
1  template<>
2  class vector<bool> {
3      char* arr_;
4      size_t sz_;
5      size_t cap_;
6
7      struct BitReference {
8          char* cell;
9          uint8_t idx;
10
11         BitReference(char* cell, uint8_t idx) : cell(cell), idx(
12             idx) {}
13
14         void operator=(bool b) {
15             if (b) *cell |= (1 << idx);
16             else *cell &= ~(1 << idx);
17         }
18
19         operator bool() const {
20             return *cell & (1 << idx);
21         }
22     };
23
24     public:
25     BitReference operator[](size_t idx) {
26         return BitReference(arr_ + idx / 8, idx % 8);
27     };
28 }
```

18.2.2. Трюк Ильи Мещерина: узнать тип выражения

```
1  template<typename T>
2  class Debug {
3      Debug(T) = delete;
4  };
5
6  int main() {
7      std::vector<bool> v(10);
8      v[5] = true;
9      // Debug d(v[5]); -- компилятор скажет тип : std::
10     __bit_reference
11 }
```

Почему это проблема: `vector<bool>` нарушает концепцию контейнера — из него нельзя взять настоящую ссылку на элемент (`bool& r = v[5]` не скомпилируется), и многие шаблонные алгоритмы ломаются при работе с ним. Поэтому для хранения битовых флагов часто предпочитают `std::vector<char>`

или `std::bitset`.

19. Библиотека <algorithm>

19.1. Зачем она вообще нужна

Стандартная библиотека предоставляет **готовые алгоритмы**, которые работают с любыми контейнерами через **итераторы**. Ключевая идея: алгоритм не знает, что за контейнер перед ним — он получает пару итераторов `[begin, end)` и работает с ними.

```
1 #include <algorithm>
2 #include <vector>
3
4 int main() {
5     int arr[] = {5, 3, 1, 4, 2};
6     std::sort(arr, arr + 5);
7
8     std::vector<int> v = {5, 3, 1, 4, 2};
9     std::sort(v.begin(), v.end());
10 }
```

19.2. `std::sort`

Сортирует диапазон **на месте**. Сложность: $O(n \log n)$ (introsort).

```
1 std::vector<int> v = {5, 3, 1, 4, 2};
2 std::sort(v.begin(), v.end()); // повозрастанию
3
4 std::sort(v.begin(), v.end(), [](int a, int b) {
5     return a > b; // поубыванию
6 });
```

Важно: компаратор должен задавать *строгий слабый порядок* (`<`, а не `<=`). Если написать `a >= b` — UB.

19.3. `std::find` и `std::find_if`

Линейный поиск. Возвращает итератор на первый подходящий элемент, или `last` если ничего не нашли.

```
1 std::vector<int> v = {1, 2, 3, 4, 5};
2
3 auto it = std::find(v.begin(), v.end(), 3);
4 if (it != v.end()) std::cout << *it; // 3
5
6 auto it2 = std::find_if(v.begin(), v.end(), [](int x) {
7     return x % 2 == 0; // первый чётный -> 2
8 });
```

`end()` как «не найдено» — универсальное соглашение, потому что алгоритм не знает о типе контейнера и не может вернуть `-1`.

19.4. `std::count` и `std::count_if`

```
1 std::vector<int> v = {1, 2, 2, 3, 2, 4};
2 int cnt = std::count(v.begin(), v.end(), 2);
3           // 3
4 int even = std::count_if(v.begin(), v.end(), [](int x){ return x
5                        %2==0; }); // 4
```

19.5. `std::for_each` и `std::transform`

```
1 std::vector<int> v = {1, 2, 3, 4, 5};
2
3 std::for_each(v.begin(), v.end(), [](int& x) { x *= 2; });
4 // v == {2, 4, 6, 8, 10}
5
6 std::vector<int> out(v.size());
7 std::transform(v.begin(), v.end(), out.begin(), [](int x) {
8     return x * x;
9 });
10 // out == {4, 16, 36, 64, 100}
```

19.6. `std::copy` и `std::back_inserter`

```
1 std::vector<int> src = {1, 2, 3, 4, 5};
2 std::vector<int> evens;
3
4 std::copy_if(src.begin(), src.end(), std::back_inserter(evens),
5             [](int x) { return x % 2 == 0; });
6 // evens == {2, 4}
```

`std::back_inserter` — итератор-адаптер, который при присваивании вызывает `push_back`. Нужен, когда выходной контейнер изначально пуст.

19.7. Идиома `erase-remove`

`std::remove` не удаляет элементы — он лишь сдвигает нужные в начало и возвращает итератор на новый логический конец. Реальное удаление делает `erase`:

```
1 std::vector<int> v = {1, 2, 3, 2, 4, 2, 5};
2 v.erase(std::remove(v.begin(), v.end(), 2), v.end());
3 // v == {1, 3, 4, 5}
```

19.8. `std::accumulate` из `<numeric>`

```
1 #include <numeric>
2 std::vector<int> v = {1, 2, 3, 4, 5};
3 int sum      = std::accumulate(v.begin(), v.end(), 0);
   // 15
4 int product = std::accumulate(v.begin(), v.end(), 1,
5                               [](int acc, int x) { return acc * x; });
   // 120
```

20. Дек (std::deque)

20.1. Инвалидация указателей и ссылок в vector

```
1 std::vector<int> v(10);
2 int* p = &v[5];
3 int& x = v[5];
4 v.push_back(1);           // реаллокация!
5 std::cout << *p;          // UB -- pointer invalidation
6 std::cout << x;           // UB -- reference invalidation
```

При push_back вектор мог реаллоцироваться. Старый кусок памяти нам больше не принадлежит — все указатели и ссылки на элементы, полученные до реаллокации, недействительны.

20.2. Два главных отличия дека от вектора

1. std::deque **не инвалидирует** указатели и ссылки на существующие элементы при добавлении новых.
2. Дек умеет push_front() и pop_front() за $O(1)$.

20.3. Внутреннее устройство дека

Дек хранит массив указателей на бакеты фиксированного размера — T**arr_.

- При push_back: идём вправо по бакету; когда бакет заполнен — выделяем новый бакет справа.
- При push_front: идём влево; когда бакет заполнен — выделяем новый бакет слева и кладём элемент с его конца.
- Дек хранит две пары: (номер бакета начала, индекс внутри него) и (номер бакета конца, индекс внутри него).

Почему не инвалидируются указатели? При переполнении внешнего массива реаллоцируется только он — перепривязываются указатели на бакеты. Сами бакеты с элементами остаются на месте.

operator[] за $O(1)$: зная индекс начала и размер бакета, пересчитываем номер бакета и смещение за константное время.

У дека нет reserve() и shrink_to_fit() — элементы разбросаны по бакетам, и их нельзя реаллоцировать одним куском.

21. STL: ассоциативные контейнеры

21.1. Обзор

Ассоциативные контейнеры хранят элементы, доступные по **ключу**, а не по позиции. Делятся на две группы:

Группа	Контейнеры	Сложность поиска
Упорядоченные (дерево)	map, set, multimap, multiset	$O(\log n)$
Неупорядоченные (хеш)	unordered_map, unordered_set, ...	$O(1)$ в среднем

21.2. std::set — упорядоченное множество

Хранит **уникальные** ключи в отсортированном порядке. Реализован красно-чёрным деревом.

```

1 #include <set>
2 std::set<int> s = {3, 1, 4, 1, 5}; // дубликат 1 игнорируется
3 // хранится отсортированно : 1 3 4 5
4 s.insert(2);
5 s.count(3);           // 1 есть() или 0 нет()
6 s.contains(3);        // C++20: bool
7 s.erase(4);
8 for (int x : s) std::cout << x; // обход в порядке возрастания

```

21.3. std::map — упорядоченный словарь

Хранит пары **ключ-значение** с уникальными ключами, отсортированными по ключу.

```

1 #include <map>
2 std::map<std::string, int> ages;
3 ages["Alice"] = 30; // вставка/изменение/через []
4 ages["Bob"] = 25;
5 ages.insert({"Carol", 28});
6
7 for (const auto& [name, age] : ages) // structured bindings (C
  ++17)
8     std::cout << name << ": " << age << '\n'; // в порядке ключей

```

21.3.1. Осторожно с operator[]

operator[] **создаёт** элемент со значением по умолчанию, если ключа нет — это меняет map и не работает на const map:

```

1 std::map<std::string, int> m;
2 int x = m["missing"]; // СОЗДАЁТ запись {"missing", 0}!
3
4 // безопасные способы чтения :
5 auto it = m.find("key");
6 if (it != m.end()) use(it->second);
7 int y = m.at("key"); // бросает std::out_of_range, если ключа нет

```

21.3.2. Эффективная вставка

```
1 m.insert_or_assign("key", 5);          // C++17:
   вставить или перезаписать
2 m.try_emplace("key", expensive());    // C++17: не вычисляет value,
   если ключ есть
3 auto [it, inserted] = m.emplace("k", 1); // inserted ==
   был ли вставлен
```

21.4. multiset и multimap

Допускают **дубликаты** ключей. У multimap нет operator[]. Все элементы с данным ключом получают через equal_range:

```
1 std::multimap<std::string, int> mm;
2 mm.insert({"a", 1});
3 mm.insert({"a", 2});          // дубликат ключа разрешён
4 auto [begin, end] = mm.equal_range("a");
5 for (auto it = begin; it != end; ++it) std::cout << it->second;
```

21.5. Неупорядоченные контейнеры (хеш-таблицы)

unordered_set / unordered_map хранят элементы в хеш-таблице: поиск/вставка в среднем $O(1)$, но порядок не определён.

```
1 #include <unordered_map>
2 std::unordered_map<std::string, int> h;
3 h["one"] = 1;          // O(1) в среднем
4 h.count("one");
```

21.5.1. Устройство хеш-таблицы

Элементы раскладываются по **корзинам (buckets)** по значению хеш-функции. При совпадении хешей (**коллизии**) элементы в одной корзине образуют цепочку. **Load factor** = число элементов / число корзин; при превышении порога происходит **rehash** — перестроение таблицы.

```
1 h.load_factor();          // текущая загрузка
2 h.max_load_factor(0.7);  // порог рехеша
3 h.reserve(1000);          // заранее выделить корзины,
   избежать рехешей
```

21.5.2. Свой хеш для пользовательского типа

```
1 struct Point { int x, y; };
2 struct PointHash {
3     size_t operator()(const Point& p) const {
```

```

4         return std::hash<int>{}(p.x) ^ (std::hash<int>{}(p.y) <<
           1);
5     }
6 };
7 std::unordered_map<Point, int, PointHash> grid;

```

21.6. Упорядоченные vs неупорядоченные: что выбрать

	map/set (дерево)	unordered (хеш)
Поиск/вставка	$O(\log n)$	$O(1)$ в среднем, $O(n)$ худший
Порядок обхода	отсортирован	произвольный
Требование к ключу	operator<	хеш + operator==
Range-запросы	да (lower_bound)	нет
Инвализация итераторов при вставке	нет	да (при рехеше)

Правило: нужен порядок или диапазонные запросы — map/set; нужна максимальная скорость поиска по точному ключу — unordered_*.

21.7. Кастомный компаратор для упорядоченных контейнеров

```

1 std::set<int, std::greater<int>> desc; // поубыванию
2 // свойкомпаратор :
3 struct ByLength {
4     bool operator()(const std::string& a, const std::string& b)
5         const {
6         return a.size() < b.size();
7     }
8 };
9 std::set<std::string, ByLength> by_len;

```

22. Адаптеры контейнеров

std::stack, std::queue и std::priority_queue — это **не контейнеры**, а **адаптеры**: они оборачивают контейнер и предоставляют ограниченный интерфейс поверх него.

```

1 std::stack<int> s; // по умолчанию на deque
2 std::stack<int, std::vector<int>> s2; // на vector
3 std::stack<int, std::list<int>> s3; // на list

```

22.1. `std::stack`

```
1  template<typename T, typename Container = std::deque<T>>
2  class stack {
3      Container c_;
4  public:
5      void    push(const T& val) { c_.push_back(val); }
6      void    pop()              { c_.pop_back(); }
7      T&      top()              { return c_.back(); }
8      bool    empty() const     { return c_.empty(); }
9      size_t  size() const      { return c_.size(); }
10 };
```

Почему `pop()` возвращает `void`? Если бы `pop()` возвращал значение, пришлось бы скопировать элемент перед удалением. Если конструктор копирования бросит исключение — элемент уже удалён и значение потеряно. Разделение на `top()` (посмотреть) и `pop()` (удалить) даёт exception safety. Вернуть ссылку тоже нельзя: после `pop()` она стала бы битой.

23. Итераторы и их категории

23.1. Что такое итератор

Итератор — это тип, который позволяет **обходить последовательность**. Это обобщение указателя: его можно разыменовывать (*it), инкрементировать (++it) и сравнивать (it != end). Обычный сырой указатель — полноценный итератор. Range-based for — синтаксический сахар:

```
1 for (int x : s) { ... }
2 // разворачивается в :
3 for (auto it = s.begin(); it != s.end(); ++it) { ... }
```

Достаточно реализовать begin() и end().

23.2. Категории итераторов

Категория	Что умеет	Примеры
InputIterator	*it, ++it, ==, !=	потокковые итераторы
ForwardIterator	+ гарантия повторного прохода	unordered_set, forward_list
BidirectionalIterator	+ --it	list, set, map
RandomAccessIterator	+ it+=n, it[n], <, >	deque
ContiguousIterator	+ гарантия непрерывности памяти	vector, array, указатель

Разница между Input и Forward: InputIterator не гарантирует, что при повторном проходе увидим те же данные (поток — прочитанное не вернётся). ForwardIterator эту гарантию даёт.

23.3. iterator_traits

Для получения информации об итераторе внутри шаблонной функции:

```
1 template<typename It>
2 void foo(It begin, It end) {
3     typename std::iterator_traits<It>::value_type x = *begin;
4     using Cat = typename std::iterator_traits<It>::
        iterator_category;
5 }
```

Доступные поля: value_type, reference, pointer, difference_type, iterator_category. Почему не auto x = *begin? Для vector<bool> auto выведет BitReference, а не bool. value_type вернёт правильный тип.

23.4. Реализация `std::distance`

```
1 template<typename It>
2 typename std::iterator_traits<It>::difference_type
3 distance(It first, It last) {
4     if constexpr (std::is_same_v<
5         typename std::iterator_traits<It>::iterator_category,
6         std::random_access_iterator_tag>)
7     {
8         return last - first; // O(1)
9     }
10    typename std::iterator_traits<It>::difference_type i = 0;
11    for (; first != last; ++first, ++i) {}
12    return i; // O(n)
13 }
```

`if constexpr` вычисляется на этапе компиляции: невыбранная ветка не компилируется вообще. Это позволяет писать `last - first` не боясь, что для `non-random-access` итератора это не скомпилируется.

23.5. Реализация итератора вектора

```
1 template<typename T>
2 class Vector {
3 public:
4     class Iterator {
5     public:
6         using iterator_category = std::random_access_iterator_tag;
7         using value_type = T;
8         using difference_type = std::ptrdiff_t;
9         using pointer = T*;
10        using reference = T&;
11
12        explicit Iterator(pointer ptr) : ptr_(ptr) {}
13
14        reference operator*() const { return *ptr_; }
15        pointer operator->() const { return ptr_; }
16
17        Iterator& operator++() { ++ptr_; return *this; }
18        Iterator operator++(int) { Iterator tmp = *this; ++ptr_;
19            return tmp; }
20        Iterator& operator--() { --ptr_; return *this; }
21        Iterator operator--(int) { Iterator tmp = *this; --ptr_;
22            return tmp; }
23
24        Iterator operator+(difference_type n) const { return
25            Iterator(ptr_ + n); }
26        Iterator operator-(difference_type n) const { return
27            Iterator(ptr_ - n); }
28        difference_type operator-(const Iterator& o) const {
29            return ptr_ - o.ptr_; }
30    };
31 }
```

```

25     Iterator& operator+=(difference_type n) { ptr_ += n;
        return *this; }
26     Iterator& operator-=(difference_type n) { ptr_ -= n;
        return *this; }
27     reference operator[](difference_type n) const { return
        ptr_[n]; }
28
29     bool operator==(const Iterator& o) const { return ptr_ ==
        o.ptr_; }
30     bool operator!=(const Iterator& o) const { return ptr_ !=
        o.ptr_; }
31     bool operator< (const Iterator& o) const { return ptr_ <
        o.ptr_; }
32     bool operator> (const Iterator& o) const { return ptr_ >
        o.ptr_; }
33     bool operator<=(const Iterator& o) const { return ptr_ <=
        o.ptr_; }
34     bool operator>=(const Iterator& o) const { return ptr_ >=
        o.ptr_; }
35
36     private:
37         pointer ptr_;
38     };
39
40     Vector() : data_(nullptr), size_(0), capacity_(0) {}
41     ~Vector() { delete[] data_; }
42
43     void push_back(const T& val) {
44         if (size_ == capacity_) reserve(capacity_ == 0 ? 1 :
            capacity_ * 2);
45         data_[size_++] = val;
46     }
47
48     T& operator[](std::size_t i) { return data_[i]; }
49     std::size_t size() const { return size_; }
50     std::size_t capacity() const { return capacity_; }
51
52     Iterator begin() { return Iterator(data_); }
53     Iterator end() { return Iterator(data_ + size_); }
54
55     private:
56     void reserve(std::size_t new_cap) {
57         T* tmp = new T[new_cap];
58         for (std::size_t i = 0; i < size_; ++i) tmp[i] = data_[i];
59         delete[] data_;
60         data_ = tmp; capacity_ = new_cap;
61     }
62
63     T* data_; std::size_t size_; std::size_t capacity_;
64 };

```

Пять type alias-ов в Iterator — контракт с iterator_traits. Без них

`std::sort` и `std::distance` не смогут узнать категорию итератора.

24. Потокные итераторы

24.1. `std::istream_iterator`

```
1 #include <iterator>
2 #include <vector>
3
4 int main() {
5     std::vector<int> v(
6         std::istream_iterator<int>(std::cin),
7         std::istream_iterator<int>() // итераторконец- (EOF)
8     );
9 }
```

Это **InputIterator** — одноразовый: прочитанное назад не вернуть. `istream_iterator<int>` без аргументов — итератор конца потока.

```
1 std::istream_iterator<int> in(std::cin), eof;
2 int sum = std::accumulate(in, eof, 0); // суммавсехчиселиз
   stdin
```

24.2. `std::ostream_iterator`

```
1 std::vector<int> v = {1, 2, 3, 4, 5};
2
3 std::copy(v.begin(), v.end(),
4           std::ostream_iterator<int>(std::cout, " "));
5 // вывод: 1 2 3 4 5
6
7 std::transform(v.begin(), v.end(),
8                std::ostream_iterator<int>(std::cout, "\n"),
9                [](int x) { return x * x; });
```

`ostream_iterator` — **OutputIterator**: присваивание `*it = val` вызывает `os << val << delim`.

24.3. Итераторы файловых потоков

```
1 #include <fstream>
2 #include <iterator>
3 #include <algorithm>
4
5 int main() {
6     std::ifstream fin("input.txt");
7     std::ofstream fout("output.txt");
8 }
```

```
9     std::vector<int> v(std::istream_iterator<int>(fin),
10                      std::istream_iterator<int>());
11     std::sort(v.begin(), v.end());
12     std::copy(v.begin(), v.end(),
13              std::ostream_iterator<int>(fout, "\n"));
14 }
```

Алгоритмы не знают, откуда данные читаются и куда пишутся — они работают с абстракцией итератора. В этом и есть смысл всей иерархии.

25. Аллокаторы и управление памятью

25.1. Зачем аллокатор нужен контейнеру

У каждого контейнера есть шаблонный параметр аллокатора:

```
1 std::list<int>      // = std::list<int, std::allocator<int>>
2 std::vector<int>    // = std::vector<int, std::allocator<int>>
```

Аллокатор — это **промежуточный класс между контейнером и оператором new**. Стек вызовов:

Container → Allocator → operator new → malloc → OS

25.2. Устройство стандартного аллокатора

У аллокатора четыре метода:

```
1 template<typename T>
2 struct allocator {
3     using value_type = T;
4
5     T* allocate(std::size_t n) {
6         if (n == 0) return nullptr;
7         return static_cast<T*>(::operator new(n * sizeof(T)));
8     }
9
10    void deallocate(T* p, std::size_t) {
11        ::operator delete(p);
12    }
13
14    template<typename... Args>
15    void construct(T* p, Args&&... args) {
16        ::new (static_cast<void*>(p)) T(std::forward<Args>(args)
17        ...);
18    }
19
20    void destroy(T* p) {
21        p->~T();
22    }
23 };
```

- `allocate` — выделяет сырую память, конструкторы не вызываются.
- `deallocate` — освобождает память, деструкторы не вызываются.
- `construct` — вызывает конструктор по адресу (placement new).
- `destroy` — явно вызывает деструктор, память не освобождает.

25.3. Проблема: list аллоцирует не T

`std::list<int>` хранит не просто `int`, а узлы:

```
1 template<typename T>
2 struct ListNode { T value; ListNode* prev; ListNode* next; };
```

Но аллокатор шаблонизирован по `T = int`. Решение комитета — **rebind**: вложенный шаблон в аллокаторе, который перепривязывает его к другому типу:

```
1 template<typename T>
2 struct allocator {
3     template<typename U>
4     struct rebind { using other = allocator<U>; };
5 };
6
7 // list внутриделает :
8 using NodeAlloc = typename Alloc::template rebind<ListNode<T>>::
9     other;
10 NodeAlloc node_alloc_; // аллокатор для узлов , а не для T
```

25.4. allocator_traits

`std::allocator_traits<Alloc>` — адаптер с единым интерфейсом к любому аллокатору и разумными дефолтами для отсутствующих методов.

```
1 template<typename Alloc>
2 struct allocator_traits {
3     using value_type      = typename Alloc::value_type;
4     using pointer         = value_type*;
5     using size_type       = std::size_t;
6     using difference_type = std::ptrdiff_t;
7
8     template<typename U>
9     using rebind_alloc = typename Alloc::template rebind<U>::other;
10
11     static pointer allocate(Alloc& a, size_type n)
12     { return a.allocate(n); }
13     static void deallocate(Alloc& a, pointer p, size_type n)
14     { a.deallocate(p, n); }
15
16     // если Alloc нет construct -- placement new по умолчанию
17     template<typename T, typename... Args>
18     static void construct(Alloc& a, T* p, Args&&... args)
19     { ::new (static_cast<void*>(p)) T(std::forward<Args>(args)
20     ...); }
21
22     // если Alloc нет destroy -- явный деструктор по умолчанию
23     template<typename T>
24     static void destroy(Alloc& a, T* p) { p->~T(); }
```

```

25 // политики распространения аллокатора
26 using propagate_on_container_copy_assignment = std::false_type
27 ;
27 using propagate_on_container_move_assignment = std::false_type
28 ;
28 using propagate_on_container_swap           = std::false_type
29 ;

```

Главная идея: контейнер должен работать с аллокатором *только через* `allocator_traits`. Тогда кастомный аллокатор может не реализовывать `construct/destroy` и всё равно работать.

25.4.1. Политики распространения аллокатора

Флаг	Что означает
<code>propagate_on_container_copy_assignment</code>	Копировать аллокатор при <code>operator=</code> копирования?
<code>propagate_on_container_move_assignment</code>	Копировать аллокатор при <code>operator=</code> перемещения?
<code>propagate_on_container_swap</code>	Менять аллокаторы местами при <code>swap</code> ?

По умолчанию все три — `false_type`. Это важно для `stateful` аллокаторов: если два вектора используют разные арены, при `swap` нужно либо перетащить элементы, либо поменять аллокаторы.

25.5. Allocator-aware контейнер

```

1  template<typename T, typename Alloc = std::allocator<T>>
2  class vector {
3      using AllocTraits = std::allocator_traits<Alloc>;
4
5      Alloc alloc_;
6      T*    arr_;
7      size_t sz_;
8      size_t cap_;
9
10 public:
11     explicit vector(const Alloc& a = Alloc())
12         : alloc_(a), arr_(nullptr), sz_(0), cap_(0) {}
13
14     void push_back(const T& val) {
15         if (sz_ == cap_) reserve(cap_ ? cap_ * 2 : 1);
16         AllocTraits::construct(alloc_, arr_ + sz_, val);
17         ++sz_;
18     }

```

```

19
20     ~vector() {
21         for (size_t i = 0; i < sz_; ++i)
22             AllocTraits::destroy(alloc_, arr_ + i);
23         AllocTraits::deallocate(alloc_, arr_, cap_);
24     }
25
26 private:
27     void reserve(size_t newcap) {
28         T* newarr = AllocTraits::allocate(alloc_, newcap);
29         size_t i = 0;
30         try {
31             for (; i < sz_; ++i)
32                 AllocTraits::construct(alloc_, newarr + i, arr_[i]);
33         } catch (...) {
34             for (size_t k = 0; k < i; ++k)
35                 AllocTraits::destroy(alloc_, newarr + k);
36             AllocTraits::deallocate(alloc_, newarr, newcap);
37             throw;
38         }
39         for (size_t k = 0; k < sz_; ++k)
40             AllocTraits::destroy(alloc_, arr_ + k);
41         AllocTraits::deallocate(alloc_, arr_, cap_);
42         arr_ = newarr; cap_ = newcap;
43     }
44 };

```

Stateless аллокатор (как `std::allocator`) не занимает места благодаря **EBO** (Empty Base Optimization).

25.6. Перегрузка `operator new` и `operator delete`

25.6.1. Глобальная перегрузка

```

1 void* operator new(std::size_t size) {
2     void* ptr = std::malloc(size);
3     if (!ptr) throw std::bad_alloc{};
4     return ptr;
5 }
6 void operator delete(void* ptr) noexcept { std::free(ptr); }
7
8 // C++14: sized delete
9 void operator delete(void* ptr, std::size_t size) noexcept { std::
    free(ptr); }

```

Затрагивает весь процесс. Используется для профилировщиков памяти и детекторов утечек.

25.6.2. Классовая перегрузка


```

1 struct Node {
2     int value; Node* next;
3
4     void* operator new(std::size_t size) {
5         return ::operator new(size); // делегируем глобальному
6     }
7     void operator delete(void* ptr) noexcept {
8         ::operator delete(ptr);
9     }
10 };

```

При `new Node` компилятор сначала ищет `operator new` в классе, и только если не нашёл — берёт глобальный.

25.6.3. Когда использовать что

Инструмент	Когда применять
Кастомный аллокатор	Своя стратегия памяти для конкретного контейнера
Классовая перегрузка <code>new/delete</code>	Свой пул для конкретного класса (например, узлы дерева)
Глобальная перегрузка <code>new/delete</code>	Профилирование, детектирование утечек для всей программы

26. Многопоточность в C++

Лирическое отступление. Данный раздел **не относится** к текущему курсу и приводится исключительно в ознакомительных целях. Многопоточность будет подробно рассмотрена в последующих курсах.

26.1. Определение потока исполнения

26.1.1. Что такое поток?

Поток исполнения (thread of execution) — это единственная последовательная цепочка команд, выполняемых процессором. Представьте это как «линию чтения» в книге: один палец ведёт по одной строке. Если у вас два пальца — вы читаете два куска книги одновременно (два потока).

Программа считается завершённой **только тогда**, когда завершились **все** её потоки. Даже если `main()` уже отработал, отсоединённые потоки могут ещё работать.

26.1.2. Аналогия

Представьте кухню ресторана:

- **Однопоточная программа** — один повар делает всё по очереди: режет овощи, варит суп, жарит мясо. Пока суп варится, повар просто ждёт.
- **Многопоточная программа** — несколько поваров работают параллельно. Один варит суп, другой жарит мясо, третий режет салат.

26.1.3. Логическая многопоточность vs Аппаратный параллелизм

Это не одно и то же:

Понятие	Суть	Пример
Логическая многопоточность	ОС быстро переключается между потоками на одном ядре, создавая иллюзию параллельности	1 ядро, 4 потока — ОС по очереди даёт каждому ~10 мс
Аппаратный параллелизм	Потоки реально выполняются одновременно на разных ядрах	4 ядра × 1 поток — каждый работает физически одновременно

Многопоточность появилась **раньше** многоядерных процессоров. Зачем? Чтобы решить проблему **блокирующего ввода-вывода**. Классический пример: в ранних ОС форматирование дискеты «вешало» всю систему, потому что процессор просто ждал завершения I/O-операции. С появлением потоков ОС могла переключиться на другую задачу, пока диск форматируется.

26.1.4. Типы многозадачности

Кооперативная многозадачность — поток *сам* решает, когда отдать управление другим. Если поток «зажадничал» и не отдаёт CPU — все остальные замирают.

```
Поток
A: работаю... работаю... ладно, передаю управление -> yield()
Поток
B: о, моя очередь ! работаю... yield()
Поток
A: снова !
```

Пример в современном C++ — **корутины** (`co_await`, `co_yield` в C++20).

Вытесняющая многозадачность — ОС *сама* прерывает потоки по таймеру (обычно каждые ~10 мс). Поток не может «зависнуть» и заблокировать всю систему.

```
Поток
A: работаю... работ--ОС
[]: Время вышло ! Переключаю на B. Поток
B: работаю...
```

Это стандарт для всех современных ОС (Windows, Linux, macOS).

26.2. Работа с потоками в C++

26.2.1. Создание потока

Потоки создаются через `std::thread` (доступен с C++11). Важная деталь: **поток запускается немедленно** в момент создания объекта — нет отдельного метода `start()`.

```
1 #include <iostream>
2 #include <thread>
3
4 void say_hello() {
5     std::cout << "Привет из потока !\n";
6 }
7
8 int main() {
9     std::thread t(say_hello); // Поток УЖЕ работает !
10    t.join();                 // Ждём завершения
11    return 0;
12 }
```

26.2.2. Передача аргументов в поток

```
1 void print_sum(int a, int b) {
2     std::cout << a + b << "\n";
3 }
4
```

```

5 int main() {
6     std::thread t(print_sum, 3, 7); //
        Аргументы передаются после функции
7     t.join();
8 }

```

Внимание: аргументы копируются в поток. Если нужна ссылка, используйте `std::ref()`:

```

1 void increment(int& x) { x++; }
2
3 int main() {
4     int value = 0;
5     std::thread t(increment, std::ref(value)); // Передаём ссылку
6     t.join();
7     std::cout << value; // 1
8 }

```

26.2.3. Использование лямбд

```

1 int main() {
2     int data = 42;
3     std::thread t([&data]() {
4         data *= 2;
5         std::cout << "В потоке: " << data << "\n";
6     });
7     t.join();
8     std::cout << "В main: " << data << "\n"; // 84
9 }

```

26.2.4. `join()` vs `detach()`

Метод	Что делает	Когда использовать
<code>join()</code>	Блокирует текущий поток до завершения вызванного	Когда нужен результат работы потока
<code>detach()</code>	Отсоединяет поток, он живёт самостоятельно	Фоновые задачи (логирование, мониторинг)

```

1 // Пример join() -- ждём результат
2 std::thread t(heavy_computation);
3 // ... тут main делает своё дело ...
4 t.join(); // Теперь точно результат готов
5
6 // Пример detach() -- fire and forget
7 std::thread logger(write_logs);
8 logger.detach(); // Пусть пишет лог в фоне

```

26.2.5. Опасность: забыть join/detach

Если объект `std::thread` уничтожается без вызова `join()` или `detach()`, программа **аварийно завершается** через `std::terminate()`:

```
1 void dangerous() {
2     std::thread t(some_work);
3     // Упс! Функция завершилась, t уничтожается
4     // без join/detach -> std::terminate()!
5 }
```

Безопасный паттерн — RAII-обёртка (или `std::jthread` из C++20, который сам делает `join` в деструкторе):

```
1 // C++20
2 void safe() {
3     std::jthread t(some_work);
4     // Деструктор jthread сам вызовет join()
5 }
```

26.3. Область памяти и состояние гонки (Data Race)

26.3.1. Что такое Memory Location?

Memory Location (область памяти) по стандарту — это:

- Объект **скалярного типа** (`int`, `char`, `float`, указатель и т. д.)
- Или **максимальная последовательность битовых полей ненулевой длины**

Важный нюанс: два соседних поля структуры — это **разные** области памяти, к ним можно обращаться параллельно без гонки. Но соседние битовые поля — это **одна** область памяти!

```
1 struct S {
2     int x;           // Отдельная memory location
3     int y;           // Отдельная memory location --
                       // можно менять параллельно x
4 };
5
6 struct Bits {
7     unsigned a : 4;  // a и b -- ОДНА memory location!
8     unsigned b : 4;  // Параллельное изменение a и b = Data Race
9     unsigned   : 0;  // Разделитель
10    unsigned c : 4;  // c -- отдельная memory location
11 };
```

26.3.2. Что такое Data Race?

Data Race (гонка данных) — это ситуация, когда:

1. Два потока обращаются к **одной области памяти**

2. Хотя бы один из них записывает
3. Между ними **нет синхронизации** (нет happens-before)
4. Хотя бы одно действие **не атомарное**

Data Race = Undefined Behavior (UB). Это не просто «неправильный результат». Компилятор имеет право на *что угодно*: удалить код, выдать мусор, отформатировать диск (теоретически).

26.3.3. Наглядный пример гонки

```

1  #include <thread>
2
3  int counter = 0;  // Общая переменная
4
5  void increment_1000() {
6      for (int i = 0; i < 1000; ++i) {
7          counter++;  // DATA RACE!
8      }
9  }
10
11 int main() {
12     std::thread t1(increment_1000);
13     std::thread t2(increment_1000);
14     t1.join();
15     t2.join();
16
17     // Ожидаем 2000, но результат непредсказуем !
18     // Может быть 1500, 1837, 2000, или что угодно
19     std::cout << counter << "\n";
20 }

```

Почему не 2000? Операция counter++ не атомарна. На уровне процессора это три шага:

- | | | |
|--|------------|---------|
| 1. Прочитать counter из памяти | -> регистр | (LOAD) |
| 2. Увеличить значение в регистре на 1 | | (ADD) |
| 3. Записать результат обратно в память | | (STORE) |

Если два потока выполняют это одновременно:

- | Поток | | |
|----------------------|----------------|------------------------|
| 1: LOAD counter (=0) | -> регистр = 0 | Поток |
| 2: LOAD counter (=0) | -> регистр = 0 | Поток |
| 1: ADD | -> регистр = 1 | Поток |
| 2: ADD | -> регистр = 1 | Поток |
| 1: STORE | -> counter = 1 | Поток |
| 2: STORE | -> counter = 1 | <- Инкремент потерян ! |

26.4. Распространённые ошибки и заблуждения

26.4.1. Ошибка №1: `volatile` «чинит» гонки

Нет! `volatile` — это указание компилятору:

- Не оптимизировать чтение/запись этой переменной
- Каждый раз читать значение из памяти, а не из регистра

Но `volatile` **НЕ обеспечивает**:

- Атомарность операций
- Барьеры памяти (memory barriers)
- Когерентность кэшей между ядрами

```
1 volatile int counter = 0; // НЕрешаетпроблему !
2
3 // counter++ попрежнему- = LOAD + ADD + STORE
4 // volatile лишьгарантирует , чтокомпиляторнезакэширует
5 // значениеврегистре , ногонкавсёравноесть
6
7 // Правильноерешение :
8 std::atomic<int> counter = 0; // Атомарнаяпеременная
```

`volatile` полезен для:

- Регистров аппаратуры (MMIO)
- Переменных, изменяемых обработчиками сигналов
- Переменных, изменяемых через `setjmp/longjmp`

26.4.2. Ошибка №2: Привязка к одному ядру решает гонки

Можно использовать системные вызовы для привязки потоков к одному ядру:

```
1 // Linuxспецифичный- код
2 cpu_set_t cpuset;
3 CPU_ZERO(&cpuset);
4 CPU_SET(0, &cpuset); // Привязатькядру 0
5 sched_setaffinity(0, sizeof(cpuset), &cpuset);
```

Визуально симптомы гонки **могут исчезнуть** (потоки не работают *физически* параллельно). Но по стандарту C++ это **всё равно UB**, потому что:

- Язык C++ «не знает» про ядра процессора
- Стандарт определяет поведение абстрактной машины, а не конкретного железа
- Компилятор может оптимизировать код, опираясь на правила стандарта

26.4.3. Ошибка №3: `std::cout` безопасен для многопоточности

Стандарт **гарантирует**, что в `std::cout` не будет Data Race (внутренний буфер защищён). Но **порядок вывода** может перемешиваться:

```
1 // Поток 1:
2 std::cout << "Hello " << "from " << "thread 1\n";
3
4 // Поток 2:
5 std::cout << "Hello " << "from " << "thread 2\n";
6
7 // Возможный вывод :
8 // Hello Hello from from thread 1thread 2
9 //
10 // Каждаяотдельнаяоперация << атомарна,
11 // номеждунимидругойпотокможетвклиниться
```

Решение — вывод одной строкой:

```
1 // Вариант 1: формируемстрокузаранее
2 std::string msg = "Hello from thread 1\n";
3 std::cout << msg; // Одинвызов << = одиннепрерывныйвывод
4
5 // Вариант 2: std::osyncstream (C++20)
6 std::osyncstream(std::cout) << "Hello " << "from " << "thread 1\n"
7 ;
8 // Весьвыводбуферизуетсяиотправляетсяатомарно
```

26.5. Синхронизация: Мьютексы

26.5.1. Что такое мьютекс?

Мьютекс (mutex = mutual exclusion) — это объект синхронизации, который гарантирует, что только **один поток** в каждый момент времени выполняет защищённый участок кода (критическую секцию).

Аналогия: туалетная кабинка с замком. Зашёл — закрыл (lock). Вышел — открыл (unlock). Если занято — жди.

26.5.2. Базовый интерфейс

```
1 #include <mutex>
2
3 std::mutex mtx;
4 int counter = 0;
5
6 void safe_increment() {
7     for (int i = 0; i < 1000; ++i) {
8         mtx.lock();           // Захватитьмьютексили (ждать)
9         counter++;           // Критическаясекция -- толькоодинпоток
10        mtx.unlock();        // Освободитьмьютекс
11    }
```



```

12 }
13
14 int main() {
15     std::thread t1(safe_increment);
16     std::thread t2(safe_increment);
17     t1.join();
18     t2.join();
19     std::cout << counter << "\n"; // Всегда 2000!
20 }

```

26.5.3. Методы мьютекса

Метод	Поведение
lock()	Захватить. Если занят — заблокировать текущий поток до освобождения
unlock()	Освободить мьютекс
try_lock()	Попробовать захватить. Если занят — не ждать , вернуть false

```

1 if (mtx.try_lock()) {
2     // Захватили! Делаем работу ...
3     mtx.unlock();
4 } else {
5     // Мьютекс занят , делаем что-то - другое
6 }

```

26.5.4. Правила и UB при работе с мьютексами

UB №1: Повторный захват в том же потоке (рекурсивный lock)

```

1 std::mutex mtx;
2
3 void foo() {
4     mtx.lock();
5     // ... делаем что-то - ...
6     mtx.lock(); // UB! Этот поток уже владеет мьютексом !
7                 // Deadlock или неопределённое поведение
8     mtx.unlock();
9     mtx.unlock();
10 }

```

Решение — `std::recursive_mutex` (если рекурсия действительно нужна):

```

1 std::recursive_mutex rmtx;
2
3 void foo() {
4     rmtx.lock();
5     rmtx.lock(); // Рекурсивный мьютекс допускает это
6     rmtx.unlock();

```

```

7     mtx.unlock(); // Нужно столько же unlock, сколько lock
8 }

```

UB №2: Разблокировка чужого мьютекса

```

1 std::mutex mtx;
2
3 // Поток 1:
4 mtx.lock();
5 // работает...
6
7 // Поток 2:
8 mtx.unlock(); // UB! Поток 2 не захватывал этот мьютекс !

```

Почему нет метода `is_locked()`?

Потому что он бесполезен: к моменту, когда вы получите ответ, состояние уже могло измениться:

```

1 // Бессмысленный гипотетический код :
2 if (!mtx.is_locked()) { // В этот момент мьютекс свободен
3     mtx.lock(); //
4     // Здесь другой поток мог его захватить !
5 }
6 // Именно поэтому существует try_lock() --
7 // атомарная проверка захвата +

```

26.6. RAII в многопоточности

26.6.1. Проблема голых `lock/unlock`

```

1 void dangerous_function() {
2     mtx.lock();
3
4     do_work(); // Что если тут исключение ?
5
6     if (error) {
7         return; // Забыли unlock!
8         // Мьютекс навсегда заблокирован !
9     }
10    mtx.unlock(); // Досюда можем не дойти
11 }

```

26.6.2. `std::lock_guard` — простая RAII-обёртка

Принцип RAII: конструктор захватывает ресурс, деструктор освобождает. Гарантирует освобождение при **любом** выходе из области видимости (`return`, исключение, `break`...).

```

1 void safe_function() {

```

```

2      std::lock_guard<std::mutex> lock(mtx); // lock()
        вконструкторе
3
4      do_work();           // Исключение? Нестрашно !
5
6      if (error) {
7          return;          // Деструктор lock_guard вызовет unlock()
8      }
9
10     // Деструктор lock_guard вызовет unlock()
11 }

```

С C++17 можно не указывать тип (STAD):

```

1  std::lock_guard lock(mtx); // Компиляторвыведеттипсам

```

26.6.3. std::unique_lock — гибкая обёртка

В отличие от lock_guard, позволяет:

- Откладывать захват (std::defer_lock)
- Досрочно освободить (unlock())
- Перемещать между областями видимости

```

1  void flexible_function() {
2      std::unique_lock<std::mutex> lock(mtx, std::defer_lock);
3      // МьютексещёНЕзахвачен
4
5      prepare_data();
6
7      lock.lock();           // Захватываем, когданужно
8      modify_shared_data();
9      lock.unlock();        // Освобождаемдосрочно
10
11     do_unrelated_work();   // Работаембезмьютекса
12 }

```

26.6.4. std::scoped_lock (C++17) — захват нескольких мьютексов

Главное преимущество: захватывает **несколько** мьютексов **без риска дедло-ка**:

```

1  std::mutex mtx1, mtx2;
2
3  void transfer(Account& from, Account& to, int amount) {
4      std::scoped_lock lock(mtx1, mtx2); //
        Оба мьютекса захвачены безопасно
5      from.balance -= amount;
6      to.balance += amount;

```

```

7 // Деструктор освободитобамьютекса
8 }

```

26.6.5. Сравнение RAII-обёрток

Свойство	lock_guard	unique_lock	scoped_lock
Несколько мьютексов	нет	нет	да
Отложенный захват	нет	да	нет
Досрочное освобождение	нет	да	нет
Перемещение	нет	да	нет
Стандарт	C++11	C++11	C++17

26.7. Проектирование потокобезопасных объектов

26.7.1. Уровни потокобезопасности

Уровень «Нейтральный» — объект сам по себе не защищён, но безопасен при внешней синхронизации.

```

1 int x = 0; // Нейтральный тип
2
3 // Безопасно, если обернуть в мьютекс :
4 std::mutex mtx;
5 {
6     std::lock_guard lock(mtx);
7     x++;
8 }

```

Уровень «Ниже нейтрального» — указатели. Даже если синхронизировать сам указатель, данные по нему могут быть не защищены:

```

1 std::mutex mtx;
2 int* ptr = new int(42);
3
4 // Поток 1:
5 {
6     std::lock_guard lock(mtx);
7     ptr = new int(100); // Защитили ptr
8 }
9
10 // Поток 2:
11 int local = *ptr; // А данные по указателю --
    незащищены !
12 // ptr
    может указывать на удалённую память

```

Уровень «Потокобезопасный» — все операции внутренне защищены. Пример: `std::atomic<int>`.

```

1 std::atomic<int> counter = 0;
2

```

```

3 // Поток 1:
4 counter++; // Атомарноибезопасно
5
6 // Поток 2:
7 counter++; // Атомарноибезопасно

```

26.7.2. API Race — коварная ловушка

Это ситуация, когда **каждый отдельный метод** защищён мьютексом (Data Race нет!), но **последовательность вызовов** создаёт логическую ошибку.

Классический пример: потокобезопасный стек

```

1 // Допустим, унасестьпотокобезопасный "" стек:
2 class ThreadSafeStack {
3     std::mutex mtx;
4     std::stack<int> data;
5 public:
6     bool empty() {
7         std::lock_guard lock(mtx);
8         return data.empty();
9     }
10    int top() {
11        std::lock_guard lock(mtx);
12        return data.top();
13    }
14    void pop() {
15        std::lock_guard lock(mtx);
16        data.pop();
17    }
18 };

```

Каждый метод по отдельности безопасен. Но посмотрите:

```

1 ThreadSafeStack stack;
2
3 // Поток 1:
4 if (!stack.empty()) { // 1. Проверили -- непуст
5                       // Между empty() и top() поток 2
5                       // вызвал pop()!
6         int val = stack.top(); // 2. Стекужепуст -> UB!
7         stack.pop();
8     }
9
10 // Поток 2 выполняется( междушагами 1 и 2 потока 1):
11 if (!stack.empty()) {
12     stack.pop(); // Забралпоследнийэлемент
13 }

```

Решение: объединение операций

```

1 class SafeStack {
2     std::mutex mtx;
3     std::stack<int> data;

```

```

4 public:
5     // Одна атомарная операция вместо трёх
6     std::optional<int> try_pop() {
7         std::lock_guard lock(mtx);
8         if (data.empty()) {
9             return std::nullopt; // Пусто -- верну ничего ""
10        }
11        int val = data.top();
12        data.pop();
13        return val;                // Проверка + чтение + удаление
14        // = одна операция
15    }
16    void push(int val) {
17        std::lock_guard lock(mtx);
18        data.push(val);
19    }
20 };
21
22 // Использование:
23 if (auto val = stack.try_pop()) {
24     std::cout << "Получили: " << *val << "\n";
25 } else {
26     std::cout << "Стек пуст\n";
27 }

```

26.8. Дедлоки (Deadlocks)

26.8.1. Что такое дедлок?

Дедлок (взаимная блокировка) — ситуация, когда два (или более) потока бесконечно ждут друг друга, потому что каждый держит ресурс, нужный другому.

26.8.2. Классический пример: перевод денег

```

1  std::mutex mtx_A, mtx_B; // Мьютексы для счетов А и В
2
3  // Поток 1: перевод А -> В
4  void transfer_A_to_B() {
5      mtx_A.lock();           // 1. Захватил А
6      // ... тут переключение контекста ...
7      mtx_B.lock();           // 3. Ждёт В... но В занят потоком 2
8      // -> ДЕДЛОК
9      // перевод...
10     mtx_B.unlock();
11     mtx_A.unlock();
12 }
13
14 // Поток 2: перевод В -> А
15 void transfer_B_to_A() {

```

```

15     mtx_B.lock();           // 2. Захватил B
16     // ... тут переключение контекста ...
17     mtx_A.lock();           // 4. Ждёт A... но A занят потоком 1
    -> ДЕДЛОК
18     // перевод...
19     mtx_A.unlock();
20     mtx_B.unlock();
21 }

```

Визуально:

Поток	
1: держит [A], хочет [B] -----+	+--- Обаждутьвечно !Поток
2: держит [B], хочет [A] -----+	

26.8.3. Задача об обедающих философях

5 философов сидят за круглым столом. Между каждыми двумя лежит одна вилка. Чтобы есть, нужны две вилки. Если каждый одновременно возьмёт левую вилку — все будут ждать правую вечно = дедлок.

26.8.4. Решения в C++

Решение 1: `std::lock()` — алгоритм безопасного захвата

```

1  std::mutex mtx_A, mtx_B;
2
3  void safe_transfer() {
4      std::lock(mtx_A, mtx_B); // Захватываетобабездедлока !
5      // std::lock используетвнутреннийалгоритм :
6      // try_lock наодном -> try_lock надругом ->
        еслинеполучилось ,
7      // отпуститьвсёипопробоватьзаново
8
9      // Оборачиваемв lock_guard с adopt_lock мьютексы(
        ужезахвачены )
10     std::lock_guard lockA(mtx_A, std::adopt_lock);
11     std::lock_guard lockB(mtx_B, std::adopt_lock);
12
13     // Делаемперевод ...
14     // Деструкторы lock_guard освободятмьютексы
15 }

```

Решение 2: `std::scoped_lock` (C++17) — всё в одну строку

```

1  void safe_transfer_modern() {
2      std::scoped_lock lock(mtx_A, mtx_B); // Всё! Безопасно RAII
3      // Делаемперевод ...
4  }

```

26.8.5. 4 необходимых условия дедлока (условия Коффмана)

Дедлок возможен **только** если выполнены **все четыре** условия одновременно:

1. **Взаимное исключение** — ресурс может быть захвачен только одним потоком
2. **Удержание и ожидание** — поток держит один ресурс и ждёт другой
3. **Невытесняемость** — ресурс нельзя отобрать у потока силой
4. **Циклическое ожидание** — замкнутая цепочка потоков, каждый ждёт следующего

Разрушение **любого** из условий предотвращает дедлок. `std::lock` разрушает условие №2 (захватывает всё или ничего).

26.9. Итого: шпаргалка

Проблема	Решение
Забыл <code>join/detach</code>	<code>std::jthread</code> (C++20)
Data Race	<code>std::mutex</code> + RAII-обёртки или <code>std::atomic</code>
Забыл <code>unlock</code>	<code>std::lock_guard</code> / <code>std::scoped_lock</code>
API Race	Объединение операций (<code>try_pop</code> и т. п.)
Дедлок	<code>std::scoped_lock(mtx1, mtx2, ...)</code>
<code>volatile</code> вместо <code>atomic</code>	Заменить на <code>std::atomic<T></code>
Перемешанный вывод	<code>std::osyncstream</code> (C++20)

27. Приведение типов (касты) в C++

27.1. «Запрещённые заклинания» — зачем нужны касты?

В C++ система типов — это ваш главный защитник. Она ловит ошибки на этапе компиляции: нельзя случайно сложить строку с указателем или передать `float` туда, где ожидается `std::vector`. Но иногда программисту **нужно** сказать компилятору: «Я знаю, что делаю — преобразуй этот тип в другой».

Для этого существуют **четыре заклинания** (каста), каждое со своей силой и опасностью. Лектор не зря называет их «запрещёнными» — неправильное использование ведёт к Undefined Behavior.

27.1.1. Почему не один каст, а четыре?

В языке C был всего один способ: (тип)выражение. Он делал *всё подряд* — и безопасные, и опасные преобразования — и программист не мог отличить одно от другого. C++ разделил эту грубую силу на четыре точных инструмента, чтобы:

- Код **явно показывал намерение** программиста
- Опасные операции **бросались в глаза** при чтении кода
- Можно было **искать** конкретные касты `grep`’ом (попробуйте найти `(int)` в миллионе строк — удачи)

27.2. C-style cast — «Грубая сила»

27.2.1. Синтаксис

```
1 double pi = 3.14159;  
2 int truncated = (int)pi;           // C-style cast  
3 int also_works = int(pi);          // Функциональный стиль то же самое )
```

27.2.2. Как это работает внутри?

C-style cast — это **не отдельная операция**. Компилятор перебирает касты в строгом порядке и применяет **первый подходящий**:

1. `const_cast` — попробовать снять/добавить `const`
2. `static_cast` — попробовать стандартное преобразование
3. `static_cast + const_cast` — комбинация
4. `reinterpret_cast` — попробовать переинтерпретировать байты
5. `reinterpret_cast + const_cast` — самая опасная комбинация

27.2.3. Почему это плохо?

```
1 class Base { virtual ~Base() = default; };
2 class Derived : public Base { public: int x = 42; };
3
4 void example() {
5     const Base* base_ptr = new Derived();
6
7     // C-style cast: что именно здесь происходит ?
8     Derived* d = (Derived*)base_ptr;
9     // Ответ: reinterpret_cast + const_cast снял( const
10    // И переинтерпретировал )
11    // Вы это поняли , просто глядя на код ? Врядли .
12
13    // Явный C++ каст -- намерение не прозрачно :
14    const Derived* d2 = static_cast<const Derived*>(base_ptr); //
    const сохранён
}
```

27.2.4. Главная опасность

C-style cast **никогда не выдаёт ошибку компиляции** для указателей. Он всегда найдёт способ — пусть даже через `reinterpret_cast`:

```
1 std::string str = "hello";
2 int* p = (int*)&str; // Компилируется! Но это полная чушь .
3                      // reinterpret_cast<int*>(&str) --
4                      // хотя бы видно , что опасно
5
6 // С C++ кастами:
7 // int* p = static_cast<int*>(&str); // Ошибка компиляции !
8 // Несовместимы.
9 // int* p = reinterpret_cast<int*>(&str); // Компилируется,
10 // но ЯВНО опасно
```

Правило: в C++ коде C-style cast использовать нельзя. Точка.

27.3. static_cast — «Безопасное преобразование»

27.3.1. Суть

`static_cast` создаёт **новый объект** другого типа, используя определённые правила преобразования. Это самый частый и самый безопасный каст.

Ключевое слово: **создаёт новый объект**. Старый остаётся нетронутым.

Если вы не знаете, какой каст нужен — скорее всего, нужен **static_cast**.

27.3.2. Сценарий 1: Числовые преобразования

```
1 double pi = 3.14159;
```

```

2 int truncated = static_cast<int>(pi); // 3 дробная(
    часть отброшена )
3
4 int big = 1'000'000;
5 short small = static_cast<short>(big); //
    Возможен переполнение ()
6
7 // Безкасткомпилятор выдаст предупреждение :
8 int x = 3.14; // Warning: implicit conversion from 'double' to '
    int'
9 int y = static_cast<int>(3.14); // Предупреждения нет --
    явно указали "знаю"

```

Как `static_cast<int>(3.14)` работает на уровне процессора?

Это не просто отбрасывание битов. Числа `int` и `double` хранятся совершенно по-разному:

```

double 3.14 в памяти (IEEE 754):
01000000 00001001 00011110 10111000 01010001 11101100 00000000
    00000000 знак
|   | экспонента | мантисса
|
int 3 в памяти :
00000000 00000000 00000000 00000011

```

`static_cast` **вычисляет** новое значение: извлекает мантиссу и экспоненту, считает целую часть и записывает в формате `int`. Это реальная работа процессора (инструкция `cvttsd2si` на x86).

27.3.3. Сценарий 2: Enum ↔ int

```

1 enum class Color { Red = 0, Green = 1, Blue = 2 };
2
3 Color c = Color::Green;
4 int num = static_cast<int>(c); // 1
5 Color back = static_cast<Color>(2); // Color::Blue
6
7 // Без static_cast:
8 // int num = c; // Ошибка! enum class не преобразуется неявно

```

27.3.4. Сценарий 3: void* → конкретный тип

```

1 // Классический паттерн из C API (pthread, qsort итд ..)
2 void callback(void* data) {
3     // Мы ЗНАЕМ, что передали int*
4     int* ptr = static_cast<int*>(data);
5     std::cout << *ptr << "\n";
6 }
7

```

```

8 int main() {
9     int value = 42;
10    callback(&value);    // int* неявно -> void*
11 }

```

27.3.5. Сценарий 4: Преобразование через конструктор

```

1 class Celsius {
2 public:
3     double temp;
4     explicit Celsius(double t) : temp(t) {}
5 };
6
7 double raw = 36.6;
8 // Celsius c = raw;    // Ошибка! Конструктор explicit
9 Celsius c = static_cast<Celsius>(raw);    // Явный вызов конструктора

```

27.3.6. Сценарий 5: Приведение в иерархии наследования (downcast)

```

1 class Animal {
2 public:
3     virtual ~Animal() = default;
4     virtual void speak() = 0;
5 };
6
7 class Dog : public Animal {
8 public:
9     void speak() override { std::cout << "Гав!\n"; }
10    void fetch() { std::cout << "Принёс палку!\n"; }
11 };
12
13 void process(Animal* animal) {
14     // Мы УБЕДИЛИСЬ, что это Dog например(, из контекста )
15     Dog* dog = static_cast<Dog*>(animal);
16     dog->fetch();    // Работает, если animal действительно Dog
17                     // UB, если animal -- не Dog!
18 }

```

Важно: `static_cast` не проверяет тип в рантайме. Если вы ошиблись — UB. Для безопасной проверки используйте `dynamic_cast`.

27.3.7. Чем `static_cast` НЕ может

```

1 int x = 42;
2 // static_cast<double*>(&x);    // Ошибка! int* и double*
   // несовместимы
3 // static_cast<std::string>(x);    // Ошибка! Нет конструктора
   // string(int)

```

```

4
5 const int cx = 10;
6 // static_cast<int&>(cx);           // Ошибка! Нemoжетснять const

```

27.4. reinterpret_cast — «Рентген памяти»

27.4.1. Суть

`reinterpret_cast` не создаёт новый объект. Он говорит: «Возьми эти байты в памяти и считай, что это другой тип». Никаких вычислений, никаких преобразований — чистая переинтерпретация.

27.4.2. Аналогия

Представьте книгу на русском языке. `static_cast` — это перевод на английский (создаётся новый текст с тем же смыслом). `reinterpret_cast` — это попытка прочитать русские буквы как английские (ерунда, но технически возможно).

27.4.3. Пример 1: Заглядываем «внутри» float

```

1 float f = 1.0f;
2
3 // Как float хранится в памяти ?
4 // IEEE 754: 0 01111111 000000000000000000000000 = 0x3F800000
5
6 // НЕПРАВИЛЬНО -- UB по стандарту нарушение (strict aliasing):
7 int bits_bad = *reinterpret_cast<int>(&f);
8
9 // ПРАВИЛЬНО -- через memcpy стандарт( разрешает):
10 int bits;
11 std::memcpy(&bits, &f, sizeof(float));
12 std::cout << std::hex << bits << "\n"; // 3f800000

```

Strict aliasing rule: обращение к объекту через указатель на «чужой» тип — UB (за исключением `char*`, `unsigned char*`, `std::byte*`).

27.4.4. Пример 2: Указатель → число и обратно

```

1 int x = 42;
2 int* ptr = &x;
3
4 // Сохраняем адрес как число
5 uintptr_t addr = reinterpret_cast<uintptr_t>(ptr);
6 std::cout << "Адрес: 0x" << std::hex << addr << "\n";
7
8 // Восстанавливаем указатель из числа
9 int* restored = reinterpret_cast<int>(addr);
10 std::cout << *restored << "\n"; // 42

```

```

11
12 // Это один из НЕМНОГИХ гарантированных сценариев для
    reinterpret_cast

```

27.4.5. Пример 3: Работа с сетевыми пакетами

```

1 // Типичный паттерн при работе с сырыми данными из сети
2 struct PacketHeader {
3     uint32_t magic;
4     uint16_t version;
5     uint16_t length;
6 };
7
8 void process_raw_data(char* buffer, size_t size) {
9     // Технически UB из-за strict aliasing и alignment
10    // Но широко используется на практике
11    auto* header = reinterpret_cast<PacketHeader*>(buffer);
12
13    // Безопасная альтернатива :
14    PacketHeader header_safe;
15    std::memcpy(&header_safe, buffer, sizeof(PacketHeader));
16
17    if (header_safe.magic == 0xDEADBEEF) {
18        // Обработка пакета ...
19    }
20 }

```

27.4.6. reinterpret_cast НЕ может снять const

```

1 const int x = 42;
2 // int* p = reinterpret_cast<int*>(&x); // Ошибка компиляции !
3 // Для снятия const нужен const_cast

```

27.4.7. Когда reinterpret_cast оправдан?

Практически **никогда** в обычном коде. Легитимные случаи:

- Преобразование указатель ↔ uintptr_t
- Преобразование между указателями на функции (с осторожностью)
- Низкоуровневый код (драйверы, аллокаторы, сериализация)
- Работа с C API, которое требует void*

27.5. const_cast — «Снятие защиты»

27.5.1. Суть

const_cast — единственный каст, который может **добавлять или убирать** модификаторы const и volatile. Никакой другой каст этого не умеет.

27.5.2. Мягкая форма: добавление const

Происходит **постоянно и неявно** — вы даже не замечаете:

```
1 std::vector<int> vec = {1, 2, 3};
2
3 // Когда вызываете size(), неявно происходит :
4 // vec неконстантный() -> const vec& константная( ссылка внутри
   size())
5 size_t s = vec.size(); // size() объявлен как : size_t size()
   const;
6
7 // Это эквивалентно :
8 const std::vector<int>& const_ref = vec; // Неявное добавление
   const
9 size_t s2 = const_ref.size();
10
11 // Явный const_cast не( нужен, но для понимания ):
12 const std::vector<int>& ref = const_cast<const> std::vector<int>&>(
   vec);
```

Добавление const — **бесплатная** операция. Процессор не выполняет никаких инструкций — это чисто компиляторная проверка.

27.5.3. Жёсткая форма: снятие const

Вот здесь начинается опасность:

```
1 void legacy_api(char* str) {
2     // Старая функция, которая НЕ модифицирует строку,
3     // но автор забыл написать const
4     std::cout << str << "\n";
5 }
6
7 void modern_code(const std::string& s) {
8     // Нужно вызвать legacy_api, но у нас const
9     // legacy_api(s.c_str()); // Ошибка! c_str() возвращает const
   char*
10
11     legacy_api(const_cast<char*>(s.c_str())); // Снимаем const
12     // Безопасно ТОЛЬКО если legacy_api реально не изменяет данные
   !
13 }
```

27.5.4. Главное правило и UB

```
1 // БЕЗОПАСНО: объект изначально не const, мы просто снимаем лишний
   "" const
2 int x = 42;
3 const int& ref = x; // Добавили const
4 int& mutable_ref = const_cast<int&>(ref); // Сняли const
5 mutable_ref = 100; // ОК -- x изначально не const
6 std::cout << x << "\n"; // 100
7
8 // UB: объект изначально объявлен как const
9 const int y = 42;
10 int& bad_ref = const_cast<int&>(y); // Компилируется...
11 bad_ref = 100; // UNDEFINED BEHAVIOR!
12 // Компилятор мог поместить y в read-only память
13 // Или заменить все обращения к y на литерал 42
```

Почему UB? Компилятор имеет право:

- Поместить const объект в **read-only секцию** памяти (запись = segfault)
- **Заинлайнить** значение: заменить все y на литерал 42 при компиляции
- Оптимизировать код, **предполагая**, что значение никогда не меняется

27.5.5. Реальный use case: избежание дублирования кода

Классический паттерн для operator[]:

```
1 class MyArray {
2     int data[100];
3 public:
4     // Константная версия -- основная логика
5     const int& operator[](size_t idx) const {
6         // Допустим, тут сложная проверка границ, логирование и т.д.
7         if (idx >= 100) throw std::out_of_range("Index out of
           bounds");
8         return data[idx];
9     }
10
11     // Неконстантная версия -- делегирует к константной
12     int& operator[](size_t idx) {
13         // Шаг 1: добавляем const к *this -> вызываем const версию-
14         const auto& result = static_cast<const MyArray&>(*this)[
           idx];
15         // Шаг 2: снимаем const с результата
16         return const_cast<int&>(result);
17         // Безопасно! Мы знаем, что *this не const мы(
           вне константного метода )
18     }
19 };
```

Этот паттерн рекомендован Скоттом Мейерсом в «Effective C++».

27.6. `dynamic_cast` — «Безопасный детектив» (краткий обзор)

`dynamic_cast` проверяет тип в **рантайме** через таблицу виртуальных функций (RTTI). Подробно изучается в теме ООП, но вот основная идея:

```
1 class Base {
2 public:
3     virtual ~Base() = default; // Виртуальный деструктор ОБЯЗАТЕЛЕН
4 };
5
6 class Derived : public Base {
7 public:
8     void special_method() { std::cout << "Я Derived!\n"; }
9 };
10
11 void safe_process(Base* base) {
12     // dynamic_cast проверяет реальный тип в рантайме
13     Derived* d = dynamic_cast<Derived*>(base);
14
15     if (d != nullptr) {
16         d->special_method(); // Гарантированно безопасно
17     } else {
18         std::cout << "Это не Derived\n";
19     }
20 }
21
22 // Для сылок : бросает std::bad_cast при неудаче
23 void safe_process_ref(Base& base) {
24     try {
25         Derived& d = dynamic_cast<Derived&>(base);
26         d.special_method();
27     } catch (const std::bad_cast& e) {
28         std::cout << "He Derived: " << e.what() << "\n";
29     }
30 }
```

Свойство	static_cast	dynamic_cast
Проверка в рантайме	нет	да
Нужен virtual	нет	да
Скорость	быстрый (0 затрат)	медленнее (RTTI)
При ошибке	UB	nullptr или исключение

27.7. Сводная таблица: какой каст когда

Задача	Каст	Пример
<code>int → double</code>	<code>static_cast</code>	<code>static_cast<double>(42)</code>
<code>enum → int</code>	<code>static_cast</code>	<code>static_cast<int>(Color::Red)</code>
<code>Base* → Derived*</code> (уверен)	<code>static_cast</code>	<code>static_cast<Derived*>(base)</code>
<code>Base* → Derived*</code> (не уверен)	<code>dynamic_cast</code>	<code>dynamic_cast<Derived*>(base)</code>
Посмотреть байты объекта	<code>memcpy</code>	Через <code>char*</code> или <code>memcpy</code>
Указатель ↔ число	<code>reinterpret_cast</code>	<code>reinterpret_cast<uintptr_t></code>
Снять <code>const</code> для legacy API	<code>const_cast</code>	<code>const_cast<char*>(c_str)</code>
<code>void* → конкретный тип</code>	<code>static_cast</code>	<code>static_cast<int*>(void_ptr)</code>
«Хочу чтобы скомпилировалось»	Ничего	Подумайте ещё раз о дизайне

27.8. Упражнения

27.8.1. Упражнение 1: Найди ошибку

Что не так в каждом фрагменте? Какой результат?

```

1 // Фрагмент А
2 float f = 3.14f;
3 int* p = (int*)&f;
4 std::cout << *p << "\n";
5
6 // Фрагмент В
7 const std::string name = "Alice";
8 std::string& ref = const_cast<std::string&>(name);
9 ref = "Bob";
10
11 // Фрагмент С
12 int x = 65;
13 char* c = static_cast<char*>(&x);

```

Решение:

- **Фрагмент А:** C-style cast выполняет `reinterpret_cast<int*>(&f)`. Чтение `*p` — UB (нарушение strict aliasing: `int*` указывает на объект типа `float`). На практике выведет 1078523331 (0x4048F5C3 — IEEE 754 представление 3.14f), но полагаться на это нельзя.
- **Фрагмент В:** UB! Объект `name` объявлен как `const`. Снятие `const` и модификация — Undefined Behavior. Компилятор мог разместить "Alice" в read-only памяти (segfault). Или заоптимизировать и продолжать «видеть» "Alice".
- **Фрагмент С:** Ошибка компиляции! `static_cast` не может преобразовать `int* → char*` (несовместимые типы указателей). Для этого нужен `reinterpret_cast<` — и это один из немногих случаев, где такой каст легален (через `char*` можно читать байты любого объекта).

27.8.2. Упражнение 2: Перепиши без C-style cast

Замените C-style касты на правильные C++ касты:

```
1 double d = 3.14;
2 int i = (int)d;
3
4 const char* s = "hello";
5 char* p = (char*)s;
6
7 Base* b = new Derived();
8 Derived* d = (Derived*)b;
9
10 void* raw = malloc(100);
11 int* arr = (int*)raw;
```

Решение:

```
1 double d = 3.14;
2 int i = static_cast<int>(d); // Числовое преобразование
3
4 const char* s = "hello";
5 char* p = const_cast<char*>(s); // Снятие const модификация( = UB
6     !)
7
8 Base* b = new Derived();
9 Derived* d = dynamic_cast<Derived*>(b); // Безопасный downcast
10     проверкой
11 // Или static_cast<Derived*>(b) если вына 100% уверены в типе
12
13 void* raw = malloc(100);
14 int* arr = static_cast<int*>(raw); // void* -> конкретный тип
```

27.8.3. Упражнение 3: Что выведет программа?

```
1 #include <iostream>
2
3 int main() {
4     int x = 42;
5     const int& cref = x;
6
7     int& ref = const_cast<int&>(cref);
8     ref = 100;
9
10     std::cout << "x = " << x << "\n";
11     std::cout << "cref = " << cref << "\n";
12     std::cout << "ref = " << ref << "\n";
13
14     return 0;
15 }
```

Решение:

Вывод:

```
x = 100
cref = 100
ref = 100
```

Это **не UB**, потому что `x` изначально объявлен как обычный (не `const`) `int`. Мы лишь добавили `const` через ссылку, а потом сняли его обратно. Все три имени (`x`, `cref`, `ref`) ссылаются на одну и ту же область памяти, поэтому изменение через `ref` видно через все имена.

Было бы UB, если бы `x` был объявлен как `const int x = 42;`.

27.8.4. Упражнение 4: Реализуй безопасное приведение

Напишите шаблонную функцию `safe_narrow`, которая преобразует числовой тип с проверкой потери данных:

```
1 template<typename To, typename From>
2 To safe_narrow(From value) {
3     // Вашкодздесь
4 }
5
6 // Использование:
7 int big = 1'000'000;
8 short s = safe_narrow<short>(big); // Должноброситьисключение !
9
10 double pi = 3.14;
11 int i = safe_narrow<int>(pi); // Должноброситьисключение !
    потеря( .14)
```

Решение:

```
1 #include <stdexcept>
2 #include <sstream>
3
4 template<typename To, typename From>
5 To safe_narrow(From value) {
6     To converted = static_cast<To>(value);
7
8     // Проверяем: еслипреобразоватьобратно ,
9     // получимлиисходноезначение ?
10    if (static_cast<From>(converted) != value) {
11        std::ostringstream oss;
12        oss << "Narrowing conversion lost data: " << value
13            << " became " << converted;
14        throw std::runtime_error(oss.str());
15    }
16    return converted;
17 }
18
19 // Тесты:
20 int main() {
```

```

21 // Безопасные преобразования
22 short s1 = safe_narrow<short>(42);           // OK: 42
        помещается в short
23 int i1 = safe_narrow<int>(3.0);              // OK: 3.0 -> 3
        без потерь
24
25 // Опасные -- бросают исключение
26 try {
27     short s2 = safe_narrow<short>(1'000'000); //
        Переполнение!
28 } catch (const std::runtime_error& e) {
29     std::cout << e.what() << "\n";
30     // "Narrowing conversion lost data: 1000000 became 16960"
31 }
32
33 try {
34     int i2 = safe_narrow<int>(3.14); // Потеря дробной части !
35 } catch (const std::runtime_error& e) {
36     std::cout << e.what() << "\n";
37     // "Narrowing conversion lost data: 3.14 became 3"
38 }
39 }

```

Идея: `static_cast` в обе стороны. Если после «туда-обратно» значение изменилось — данные потеряны.

27.8.5. Упражнение 5: Детектив по кастам

Определите, какой именно каст скрывается за каждым C-style cast. Помните порядок: `const_cast` → `static_cast` → `static_cast + const_cast` → `reinterpret_cast` → `reinterpret_cast + const_cast`.

```

1 // 1.
2 double d = 3.14;
3 int i = (int)d;
4
5 // 2.
6 const int x = 42;
7 int* p = (int*)&x;
8
9 // 3.
10 int* ptr = (int*)0xDEADBEEF;
11
12 // 4.
13 class A {};
14 class B : public A {};
15 A* a = new B();
16 B* b = (B*)a;
17
18 // 5.
19 const double cd = 2.71;
20 int ci = (int)cd;

```

Решение:

1. `static_cast<int>(d)` — стандартное числовое преобразование `double` → `int`. `const_cast` не подходит (типы разные), `static_cast` подходит первым.
2. `const_cast<int*>(&x)` — `&x` имеет тип `const int*`, а цель `int*`. Разница только в `const`, поэтому это `const_cast`. Первый в очереди, и он подходит.
3. `reinterpret_cast<int*>(0xDEADBEEF)` — целое число → указатель. `const_cast` не подходит (разные типы), `static_cast` не умеет `int` → указатель. Поэтому `reinterpret_cast`.
4. `static_cast<B*>(a)` — приведение вниз по иерархии (downcast). `const_cast` не подходит, `static_cast` подходит (есть отношение наследования).
5. `static_cast<int>(cd)` — `cd` это значение (rvalue), `const` на значениях не мешает. Это просто `static_cast<int>(cd)` — числовое преобразование.

27.8.6. Упражнение 6: Реализуй type-running безопасно

Напишите функцию, которая показывает побитовое представление любого типа:

```
1  template<typename T>
2  std::string bit_representation(const T& value);
3
4  // Использование:
5  std::cout << bit_representation(3.14f) << "\n";
6  // "01000000 01001000 11110101 11000011"
7
8  std::cout << bit_representation(42) << "\n";
9  // "00000000 00000000 00000000 00101010"
```

Решение:

```
1  #include <cstring>
2  #include <string>
3  #include <bitset>
4  #include <sstream>
5
6  template<typename T>
7  std::string bit_representation(const T& value) {
8      // Читаем байты через memcpу -- это БЕЗОПАСНО по стандарту
9      unsigned char bytes[sizeof(T)];
10     std::memcpy(bytes, &value, sizeof(T));
11
12     std::ostringstream oss;
13     for (size_t i = 0; i < sizeof(T); ++i) {
14         if (i > 0) oss << " ";
15         oss << std::bitset<8>(bytes[i]);
16     }
```

```

17     return oss.str();
18 }
19
20 int main() {
21     std::cout << "float 3.14f:  " << bit_representation(3.14f) <<
        "\n";
22     std::cout << "int 42:      " << bit_representation(42) << "\n";
23     std::cout << "double 3.14:  " << bit_representation(3.14) << "\n";
24     std::cout << "char 'A':    " << bit_representation('A') << "\n";
25 }

```

Ключевой момент: мы используем `std::memcpu`, а **не** `reinterpret_cast`. Это единственный стандартно-безопасный способ «посмотреть на байты» объекта. Чтение через `unsigned char*` / `std::byte*` тоже допустимо по стандарту.

27.9. Итоговая шпаргалка

Какойкаствыбрать	?
Нужнопреобразоватьзначение	?
(<code>int</code> -> <code>double</code> , <code>enum</code> -> <code>int</code> , <code>void*</code> -> <code>T*</code>)	
->	<code>static_cast</code>
Нужноснятьдобавить	/ <code>const</code> ?
->	<code>const_cast</code>
Нужнопосмотретьнабайты	/ указательчисло<->?
->	<code>reinterpret_cast</code> а(лучше <code>memcpu</code>)
Нужнобезопаснопривеститипвиерархии	?
->	<code>dynamic_cast</code>
Незнаетекакой	?
->	<code>static_cast</code> и(подумайте, правильнолиспроектирован код , еслиприходитсякастовать)
C-style cast?	
->	НИКОГДАв C++ коде

28. Иерархия `std::exception`

28.1. Зачем нужна единая иерархия исключений?

В разделе про `throw/catch` мы видели, что бросать можно **любой** объект. Но если каждый программист будет бросать свои типы — наступит хаос: как ловить исключения из чужих библиотек?

Стандартная библиотека определяет **базовый класс** `std::exception` и целое дерево наследников. Это позволяет:

- Ловить **все** стандартные исключения одним `catch(const std::exception&)`
- Ловить **конкретные** типы более специализированными блоками `catch`
- Создавать **свои** исключения, наследуясь от стандартных

28.2. Класс `std::exception`

Определён в заголовке `<exception>`.

28.2.1. Интерфейс

```
1 class exception {
2 public:
3     exception() noexcept;
4     exception(const exception&) noexcept;
5     exception& operator=(const exception&) noexcept;
6     virtual ~exception() noexcept;
7
8     virtual const char* what() const noexcept;
9 };
```

- Все специальные методы помечены `noexcept` — исключение **не должно** бросать исключений при своём создании/копировании.
- Единственный содержательный метод — `what()`: возвращает C-строку с описанием ошибки.
- `what()` объявлен `virtual` — наследники переопределяют его.

28.2.2. Пример использования

```
1 #include <iostream>
2 #include <exception>
3
4 int main() {
5     try {
6         throw std::exception(); // Базовоеисключениередко (
7         бросаютнапрямую )
8     } catch (const std::exception& e) {
```



```

8         std::cout << e.what() << "\n"; // "std::exception"
           зависит( отреализации )
9     }
10 }

```

28.3. Дерево стандартных исключений

```

std::exception
|
+-- std::logic_error ошибки ( логики программы )
|   +-- std::invalid_argument неверный ( аргумент )
|   +-- std::domain_error значение ( вне области определения )
|   +-- std::length_error превышение ( максимального размера )
|   +-- std::out_of_range индекс ( за пределами )
|
+-- std::runtime_error ошибки ( времени выполнения )
|   +-- std::range_error результат ( вне диапазона )
|   +-- std::overflow_error арифметическое ( переполнение )
|   +-- std::underflow_error арифметическое ( антипереполнение )
|   +-- std::system_error ошибки ( ОС, C++11 )
|
+-- std::bad_alloc нехватка ( памяти при new )
+-- std::bad_cast неудачный ( dynamic_cast )
+-- std::bad_typeid ( typeid от nullptr )
+-- std::bad_exception нарушение ( спецификации исключений )

```

28.4. std::logic_error и std::runtime_error

Оба определены в <stdexcept> и хранят строку сообщения.

28.4.1. Конструкторы

```

1  #include <stdexcept>
2
3  // Принимают std::string или const char*
4  std::logic_error err1("ошибка логики");
5  std::runtime_error err2("ошибка выполнения");
6
7  // what() вернёт переданную строку
8  std::cout << err1.what() << "\n"; // ошибка" логики"
9  std::cout << err2.what() << "\n"; // ошибка" выполнения"

```

28.4.2. Когда что бросать?

Тип	Смысл	Пример
logic_error	Ошибка, которую <i>можно</i> было предотвратить проверкой в коде	Передали отрицательный размер массива
runtime_error	Ошибка, которую <i>нельзя</i> предсказать заранее	Файл не найден, сеть недоступна

28.5. Наследники `std::logic_error`

28.5.1. `std::invalid_argument`

Аргумент функции имеет недопустимое значение.

```
1 #include <stdexcept>
2 #include <string>
3
4 int parse_positive(const std::string& s) {
5     int val = std::stoi(s); // сам stoi может бросить
6     // invalid_argument
7     if (val <= 0) {
8         throw std::invalid_argument("expected positive number, got
9         : " + s);
10    }
11    return val;
12 }
13
14 int main() {
15     try {
16         parse_positive("-5");
17     } catch (const std::invalid_argument& e) {
18         std::cout << "Ошибка: " << e.what() << "\n";
19         // Ошибка: expected positive number, got: -5"
20     }
21 }
```

28.5.2. `std::out_of_range`

Индекс или ключ вне допустимого диапазона.

```
1 #include <vector>
2 #include <stdexcept>
3
4 int main() {
5     std::vector<int> v = {10, 20, 30};
6
7     try {
8         int val = v.at(100); // бросает std::out_of_range
9     } catch (const std::out_of_range& e) {
10    }
```

```

10         std::cout << e.what() << "\n"; //
           сообщениезависитотреализации
11     }
12
13     // Длясравнения : v[100] -- UB, никакогоисключения
14 }

```

28.5.3. `std::length_error`

Попытка создать объект, превышающий максимально допустимый размер.

```

1  #include <vector>
2  #include <stdexcept>
3
4  int main() {
5      try {
6          std::vector<int> v;
7          v.reserve(v.max_size() + 1); // бросает std::length_error
8      } catch (const std::length_error& e) {
9          std::cout << e.what() << "\n";
10     }
11 }

```

28.5.4. `std::domain_error`

Математическая функция получила значение вне области определения.

```

1  #include <cmath>
2  #include <stdexcept>
3
4  double safe_sqrt(double x) {
5      if (x < 0) {
6          throw std::domain_error("sqrt of negative number");
7      }
8      return std::sqrt(x);
9  }

```

28.6. Наследники `std::runtime_error`

28.6.1. `std::overflow_error` и `std::underflow_error`

```

1  #include <stdexcept>
2  #include <limits>
3
4  unsigned int safe_multiply(unsigned int a, unsigned int b) {
5      if (a != 0 && b > std::numeric_limits<unsigned int>::max() / a
6          ) {
7          throw std::overflow_error("unsigned multiplication
8              overflow");
9      }
10 }

```

```

7     }
8     return a * b;
9 }
10
11 int main() {
12     try {
13         auto result = safe_multiply(1'000'000'000u, 5u);
14     } catch (const std::overflow_error& e) {
15         std::cout << e.what() << "\n";
16     }
17 }

```

28.6.2. `std::system_error` (C++11)

Обёртка над ошибками операционной системы. Хранит `std::error_code`.

```

1 #include <system_error>
2 #include <fstream>
3
4 void open_file(const std::string& path) {
5     std::ifstream f(path);
6     if (!f.is_open()) {
7         throw std::system_error(
8             std::make_error_code(std::errc::
9                 no_such_file_or_directory),
10            "cannot open: " + path
11        );
12    }
13 }
14
15 int main() {
16     try {
17         open_file("/nonexistent/file.txt");
18     } catch (const std::system_error& e) {
19         std::cout << "Код: " << e.code() << "\n";
20         std::cout << "Что: " << e.what() << "\n";
21     }
22 }

```

28.7. Отдельные исключения (не наследуют `logic/runtime_error`)

28.7.1. `std::bad_alloc`

Бросается оператором `new` при нехватке памяти.

```

1 #include <new>
2 #include <iostream>
3
4 int main() {
5     try {
6         // Попытка выделить абсурдно много памяти

```

```

7      int* p = new int[1'000'000'000'000LL];
8  } catch (const std::bad_alloc& e) {
9      std::cout << "Нехватка памяти: " << e.what() << "\n";
10 }
11
12 // Альтернатива: new(std::nothrow) возвращает nullptr
    вместоисключения
13 int* p = new(std::nothrow) int[1'000'000'000'000LL];
14 if (p == nullptr) {
15     std::cout << "Не удалось выделить память \n";
16 }
17 }

```

28.7.2. std::bad_cast

Бросается при неудачном dynamic_cast к ссылке.

```

1 #include <typeinfo>
2
3 class Base { public: virtual ~Base() = default; };
4 class Derived : public Base {};
5 class Other : public Base {};
6
7 int main() {
8     Derived d;
9     Base& ref = d;
10
11     try {
12         Other& o = dynamic_cast<Other&>(ref); // Неудача!
13     } catch (const std::bad_cast& e) {
14         std::cout << e.what() << "\n"; // "std::bad_cast"
15     }
16 }

```

28.8. Создание собственных исключений

```

1 #include <stdexcept>
2 #include <string>
3
4 class HttpError : public std::runtime_error {
5     int status_code_;
6 public:
7     HttpError(int code, const std::string& msg)
8         : std::runtime_error(msg), status_code_(code) {}
9
10    int status_code() const noexcept { return status_code_; }
11 };
12
13 class NotFoundError : public HttpError {
14 public:

```

```

15     explicit NotFoundError(const std::string& url)
16         : HttpError(404, "Not found: " + url) {}
17 };
18
19 void fetch(const std::string& url) {
20     throw NotFoundError(url);
21 }
22
23 int main() {
24     try {
25         fetch("/api/users/999");
26     } catch (const HttpError& e) {
27         std::cout << "HTTP " << e.status_code() << ": " << e.what
28             () << "\n";
29         // "HTTP 404: Not found: /api/users/999"
30     }
31     // Также можно поймать как std::runtime_error или std::
    exception

```

Рекомендации:

- Наследуйтесь от `std::runtime_error` или `std::logic_error`, а не от того же `std::exception` — так вы получаете хранение строки бесплатно.
- Все методы исключений делайте по-эксперт (кроме конструктора, которому нужно скопировать строку).

29. std::optional (C++17)

29.1. Зачем нужен std::optional?

Как обозначить «значения нет»? Исторически использовались костыли:

Способ	Пример	Проблема
Магическое значение	<code>return -1;</code>	А если -1 — допустимый результат?
Указатель	<code>return nullptr;</code>	Динамическая память, ownership неясен
Пара <code>bool + value</code>	<code>std::pair<bool, T></code>	Можно прочитать value при false
Исключение	<code>throw NotFound{}</code>	Дорого, если «нет значения» — обычная ситуация

`std::optional<T>` — это контейнер, который хранит **либо значение типа T, либо ничего**. Определён в `<optional>`.

29.2. Создание

29.2.1. Конструкторы

```
1 #include <optional>
2 #include <string>
3
4 // 1. Пустой optional
5 std::optional<int> empty; // нетзначения
6 std::optional<int> also_empty = std::nullopt; // тожесамое , явно
7
8 // 2. Сзначением
9 std::optional<int> opt1 = 42; // содержит 42
10 std::optional<int> opt2(42); // тожесамое
11 std::optional<std::string> opt3("hello"); // содержит "hello"
12
13 // 3. in-place конструированиебез ( лишнихкопий )
14 std::optional<std::string> opt4(std::in_place, 5, 'x'); //
    содержит "xxxxx"
15 // Эквивалентно: std::string(5, 'x')
16
17 // 4. make_optional выводит( типавтоматически )
18 auto opt5 = std::make_optional(3.14); // std::optional<
    double>
19 auto opt6 = std::make_optional<std::string>(5, 'x'); // "xxxxx"
```

29.3. Проверка наличия значения

29.3.1. `has_value()` и `operator bool()`

```
1 std::optional<int> opt = 42;
2 std::optional<int> empty;
3
4 // Способ 1: has_value()
5 if (opt.has_value()) {
6     std::cout << "Есть значение\n";
7 }
8
9 // Способ 2: неявное преобразование в bool
10 if (opt) {
11     std::cout << "Есть значение\n"; // то же самое
12 }
13
14 if (!empty) {
15     std::cout << "Пусто\n";
16 }
```

Оба способа эквивалентны. `operator bool()` — просто сокращение для `has_value()`.

29.4. Доступ к значению

29.4.1. `operator*()` и `operator->()`

Доступ без проверки. Если значения нет — UB.

```
1 std::optional<std::string> opt = "hello";
2
3 // operator* -- доступ к значению
4 std::string& s = *opt;
5 std::cout << s << "\n"; // "hello"
6
7 // operator-> -- доступ к членам
8 std::cout << opt->size() << "\n"; // 5
9
10 // UB! Нет значения, но обращаемся:
11 std::optional<int> empty;
12 // int x = *empty; // UB -- поведение не определено
```

29.4.2. `value()`

Доступ с проверкой. Если значения нет — бросает `std::bad_optional_access`.

```
1 #include <optional>
2 #include <iostream>
3
4 int main() {
5     std::optional<int> opt = 42;
6     std::cout << opt.value() << "\n"; // 42
7 }
```



```

7
8     std::optional<int> empty;
9     try {
10         int x = empty.value(); // Бросает исключение !
11     } catch (const std::bad_optional_access& e) {
12         std::cout << e.what() << "\n"; // "bad optional access"
13     }
14 }

```

29.4.3. value_or(default)

Возвращает значение, если есть, иначе — переданное значение по умолчанию.

```

1 std::optional<int> opt = 42;
2 std::optional<int> empty;
3
4 std::cout << opt.value_or(0) << "\n"; // 42 есть (значение)
5 std::cout << empty.value_or(0) << "\n"; // 0 пусто (, вернул
    default)
6
7 // Удобно для конфигурации :
8 std::optional<std::string> user_name; // пользователь не указали
9 std::string name = user_name.value_or("Anonymous");

```

29.5. Модификация

29.5.1. emplace(args...)

Создаёт значение «на месте», разрушая предыдущее (если было).

```

1 std::optional<std::string> opt;
2
3 opt.emplace(5, 'a'); // Создаёт "aaaaa" внутри optional
4 std::cout << *opt << "\n"; // "aaaaa"
5
6 opt.emplace("hello"); // Старое разрушено, создано "hello"
7 std::cout << *opt << "\n"; // "hello"

```

29.5.2. reset()

Разрушает значение (если было), optional становится пустым.

```

1 std::optional<std::string> opt = "hello";
2 std::cout << opt.has_value() << "\n"; // 1 (true)
3
4 opt.reset();
5 std::cout << opt.has_value() << "\n"; // 0 (false)
6
7 // Эквивалентно:
8 opt = std::nullopt;

```

29.5.3. Присваивание

```
1 std::optional<int> opt;
2
3 opt = 42;           // Теперь содержит 42
4 opt = 100;          // Теперь содержит 100
5 opt = std::nullopt; // Теперь пустой
6 opt = {};           // Тоже пустой
```

29.6. Сравнение

`std::optional` поддерживает все операторы сравнения:

```
1 std::optional<int> a = 5;
2 std::optional<int> b = 10;
3 std::optional<int> c;           // пустой
4 std::optional<int> d;           // пустой
5
6 // optional с optional
7 std::cout << (a < b) << "\n";   // 1 (true): 5 < 10
8 std::cout << (a == b) << "\n";  // 0 (false)
9 std::cout << (c == d) << "\n";  // 1 (true): оба пустые
10 std::cout << (c < a) << "\n";   // 1 (true): пустой <
    любое значение
11
12 // optional с значением
13 std::cout << (a == 5) << "\n";  // 1 (true)
14 std::cout << (c == 5) << "\n";  // 0 (false): пустой != значение
15
16 // optional с nullopt
17 std::cout << (c == std::nullopt) << "\n"; // 1 (true)
18 std::cout << (a == std::nullopt) << "\n"; // 0 (false)
```

Правила:

- Два пустых `optional` **равны**
- Пустой `optional` **меньше** любого непустого
- Два непустых сравниваются по значениям внутри

29.7. Монадический интерфейс (C++23)

C++23 добавил три метода для удобной цепочки операций.

29.7.1. `and_then(f)`

Если есть значение — применяет `f` (которая сама возвращает `optional`). Если пусто — возвращает пустой `optional`.

```

1 // C++23
2 std::optional<std::string> get_env(const char* name); //
   может вернуть nullopt
3 std::optional<int> parse_int(const std::string& s); //
   может вернуть nullopt
4
5 // Цепочка: получить переменную среды -> распарсить как int
6 std::optional<int> port = get_env("PORT").and_then(parse_int);
7
8 // Без and_then (C++17):
9 std::optional<int> port2;
10 if (auto env = get_env("PORT")) {
11     port2 = parse_int(*env);
12 }

```

29.7.2. transform(f)

Если есть значение — применяет `f` и оборачивает результат в `optional`. Если пусто — возвращает пустой.

```

1 // C++23
2 std::optional<std::string> name = "hello";
3 std::optional<size_t> len = name.transform([](const std::string& s
   ) {
4     return s.size();
5 });
6 // len содержит 5
7
8 std::optional<std::string> empty;
9 std::optional<size_t> no_len = empty.transform([](const std::
   string& s) {
10     return s.size();
11 });
12 // no_len пуст

```

29.7.3. or_else(f)

Если есть значение — возвращает его. Если пусто — вызывает `f` (которая возвращает `optional`).

```

1 // C++23
2 std::optional<std::string> primary_config(); // может быть
   nullopt
3 std::optional<std::string> fallback_config(); // может быть
   nullopt
4
5 // Попробовать primary, если нет -- fallback
6 auto config = primary_config().or_else(fallback_config);

```

29.8. Где хранится значение?

`std::optional<T>` хранит объект **внутри себя** (на стеке), без динамической памяти. Упрощённая реализация:

```
1  template<typename T>
2  class optional {
3      alignas(T) unsigned char storage_[sizeof(T)]; // Сырая память
4      bool has_value_;
5
6      T& get() { return *reinterpret_cast<T*>(storage_); }
7      const T& get() const { return *reinterpret_cast<const T*>(
8          storage_); }
9
10 public:
11     optional() : has_value_(false) {}
12
13     optional(const T& value) : has_value_(true) {
14         new (storage_) T(value); // placement new
15     }
16
17     ~optional() {
18         if (has_value_) get().~T(); // явный вызов деструктора
19     }
20
21     bool has_value() const { return has_value_; }
22     T& operator*() { return get(); }
23     T& value() {
24         if (!has_value_) throw std::bad_optional_access();
25         return get();
26     }
27
28     void reset() {
29         if (has_value_) {
30             get().~T();
31             has_value_ = false;
32         }
33     };
34 }
```

$\text{sizeof}(\text{std::optional}<T>) \approx \text{sizeof}(T) + \text{sizeof}(\text{bool}) + \text{выравнивание}$.

29.9. Типичные паттерны использования

29.9.1. Функция, которая может не вернуть результат

```
1  #include <optional>
2  #include <vector>
3  #include <algorithm>
4
5  // Найти элемент в векторе
```

```

6  template<typename T>
7  std::optional<size_t> find_index(const std::vector<T>& v, const T&
   target) {
8      for (size_t i = 0; i < v.size(); ++i) {
9          if (v[i] == target) return i;
10     }
11     return std::nullopt; // Ненашли
12 }
13
14 int main() {
15     std::vector<int> v = {10, 20, 30, 40, 50};
16
17     if (auto idx = find_index(v, 30)) {
18         std::cout << "Найден на позиции " << *idx << "\n"; // 2
19     } else {
20         std::cout << "Не найден\n";
21     }
22 }

```

29.9.2. Ленивая инициализация (кэширование)

```

1  class Config {
2      mutable std::optional<std::string> cached_name_;
3
4  public:
5      const std::string& get_name() const {
6          if (!cached_name_) {
7              cached_name_ = expensive_computation(); //
              Считаю один раз
8          }
9          return *cached_name_;
10     }
11
12     void invalidate() { cached_name_.reset(); } // Сбросить кэш
13 };

```

29.9.3. Необязательные параметры структуры

```

1  struct UserProfile {
2      std::string name;
3      std::string email;
4      std::optional<std::string> phone; // Может отсутствовать
5      std::optional<int> age;           // Может отсутствовать
6  };
7
8  void print_profile(const UserProfile& u) {
9      std::cout << "Name: " << u.name << "\n";
10     std::cout << "Email: " << u.email << "\n";
11 }

```

```
12     if (u.phone) {
13         std::cout << "Phone: " << *u.phone << "\n";
14     }
15
16     std::cout << "Age: " << u.age.value_or(-1) << "\n";
17 }
```

30. std::expected (C++23)

30.1. Зачем нужен std::expected?

std::optional говорит: «значение есть или его нет». Но почему его нет? optional не даёт ответа.

std::expected<T, E> — это контейнер, который хранит **либо значение типа T (успех), либо ошибку типа E (неудача)**. Определён в <expected> (C++23).

Тип	Хранит	Когда использовать
optional<T>	T или ничего	«Может не быть значения»
expected<T,E>	T или E	«Может быть ошибка, и важно знать какая»

Это альтернатива исключениям для случаев, когда ошибка — **ожидаемая** часть логики (не исключительная ситуация):

	Исключения	std::expected
Когда использовать	Действительно исключительные ситуации	Ожидаемые ошибки (парсинг, I/O, валидация)
Стоимость ошибки	Дорого (раскрутка стека)	Дёшево (просто union)
Видимость в API	Скрыта (надо читать документацию)	Явна в типе возвращаемого значения

30.2. Создание

30.2.1. Создание со значением (успех)

```
1 #include <expected>
2 #include <string>
3
4 // Неявноиззначения
5 std::expected<int, std::string> result = 42; //
   Содержит значение 42
6
7 // Явно
8 std::expected<int, std::string> result2(42);
9
10 // in-place конструирование
11 std::expected<std::string, int> result3(std::in_place, 5, 'x');
   // "xxxxx"
```

30.2.2. Создание с ошибкой (неудача)

```
1 // Через std::unexpected
2 std::expected<int, std::string> err = std::unexpected("file not
   found");
3
4 // Явное создание
5 std::expected<int, std::string> err2(std::unexpected("parse error"
   ));
6
7 // in-place конструирование ошибки
8 std::expected<int, std::string> err3(std::unexpected, "timeout");
```

Важно: нельзя просто присвоить ошибку напрямую — нужна обёртка `std::unexpected`, чтобы компилятор отличил значение от ошибки:

```
1 // Если T и E -- один тип, без unexpected была бы неоднозначность :
2 std::expected<std::string, std::string> result = "hello"; //
   Это значение !
3 std::expected<std::string, std::string> error =
4     std::unexpected("oops"); //
   Это ошибка !
```

30.3. Проверка состояния

30.3.1. `has_value()` и `operator bool()`

```
1 std::expected<int, std::string> ok = 42;
2 std::expected<int, std::string> err = std::unexpected("fail");
3
4 if (ok.has_value()) { /* true */ }
5 if (ok) { /* true */ }
6
7 if (err.has_value()) { /* false */ }
8 if (!err) { /* true */ }
```

30.4. Доступ к значению

30.4.1. `operator*()` и `operator->()`

Доступ без проверки. Если содержит ошибку — UB.

```
1 std::expected<std::string, int> result = "hello";
2
3 std::string& s = *result;
4 std::cout << s << "\n"; // "hello"
5 std::cout << result->size() << "\n"; // 5
6
7 // UB:
8 std::expected<int, std::string> err = std::unexpected("fail");
9 // int x = *err; // UB!
```


30.4.2. value()

Доступ с проверкой. Если ошибка — бросает `std::bad_expected_access<E>`.

```
1 std::expected<int, std::string> ok = 42;
2 std::cout << ok.value() << "\n"; // 42
3
4 std::expected<int, std::string> err = std::unexpected("file not
   found");
5 try {
6     int x = err.value(); // Бросаетисключение !
7 } catch (const std::bad_expected_access<std::string>& e) {
8     // e.error() возвращаетхранимуюошибку
9     std::cout << "Ошибка: " << e.error() << "\n"; // "file not
   found"
10 }
```

30.4.3. value_or(default)

```
1 std::expected<int, std::string> ok = 42;
2 std::expected<int, std::string> err = std::unexpected("fail");
3
4 std::cout << ok.value_or(0) << "\n"; // 42
5 std::cout << err.value_or(0) << "\n"; // 0
```

30.5. Доступ к ошибке

30.5.1. error()

Возвращает хранимую ошибку. Если содержит значение (а не ошибку) — UB.

```
1 std::expected<int, std::string> err = std::unexpected("timeout");
2
3 std::cout << err.error() << "\n"; // "timeout"
4
5 // UB:
6 std::expected<int, std::string> ok = 42;
7 // auto e = ok.error(); // UB! Естьзначение , анеошибка
```

30.6. Модификация

30.6.1. emplace(args...)

```
1 std::expected<std::string, int> result = std::unexpected(404);
2
3 // Заменяемошибкуназначение
4 result.emplace(5, 'x'); // Теперьсодержит "xxxxx"
5 std::cout << *result << "\n"; // "xxxxx"
```

30.6.2. Присваивание

```
1 std::expected<int, std::string> result = 42;
2
3 result = 100; // Значение 100
4 result = std::unexpected("new error"); // Теперь ошибка
5 result = 200; // Снова значение
```

30.7. Монадический интерфейс (C++23)

Те же три метода, что и у `optional`, плюс `or_else` работает с ошибкой.

30.7.1. `and_then(f)`

Если значение — применяет `f` (возвращающую `expected`). Если ошибка — пробрасывает ошибку.

```
1 using Result = std::expected<int, std::string>;
2
3 Result parse_int(const std::string& s) {
4     try {
5         return std::stoi(s);
6     } catch (...) {
7         return std::unexpected("parse error: " + s);
8     }
9 }
10
11 Result validate_positive(int x) {
12     if (x > 0) return x;
13     return std::unexpected("must be positive");
14 }
15
16 // Цепочка: парсим -> проверяем
17 Result result = parse_int("42").and_then(validate_positive);
18 // result содержит 42
19
20 Result fail = parse_int("abc").and_then(validate_positive);
21 // fail содержит ошибку "parse error: abc"
22 // validate_positive даже не вызывалась !
```

30.7.2. `transform(f)`

Если значение — применяет `f`, оборачивает результат в `expected`. Если ошибка — пробрасывает.

```
1 std::expected<int, std::string> result = 42;
2
3 auto doubled = result.transform([](int x) { return x * 2; });
4 // doubled содержит 84
5
```

```

6 std::expected<int, std::string> err = std::unexpected("fail");
7 auto still_err = err.transform([](int x) { return x * 2; });
8 // still_err содержит ошибку "fail"

```

30.7.3. or_else(f)

Если значение — возвращает его. Если ошибка — вызывает `f` с ошибкой.

```

1 using Result = std::expected<int, std::string>;
2
3 Result primary_source();
4 Result fallback_source(const std::string& error);
5
6 // Если primary не удался -- пробуем fallback,
   передавая ему текст ошибки
7 auto result = primary_source().or_else(fallback_source);

```

30.7.4. transform_error(f)

Преобразует ошибку, не трогая значение. Полезно для конвертации типов ошибок.

```

1 std::expected<int, int> result = std::unexpected(404);
2
3 // Преобразуем int-ошибку в строку
4 auto with_msg = result.transform_error([](int code) {
5     return "HTTP error " + std::to_string(code);
6 });
7 // with_msg имеет тип std::expected<int, std::string>
8 // Содержит ошибку "HTTP error 404"

```

30.8. Где хранится значение?

Как и `optional`, хранит данные **внутри себя** без динамической памяти. Внутри — `union` из `T` и `E`:

```

1 // Упрощённая реализация
2 template<typename T, typename E>
3 class expected {
4     union {
5         T value_;
6         E error_;
7     };
8     bool has_value_;
9
10 public:
11     expected(const T& val) : value_(val), has_value_(true) {}
12
13     expected(std::unexpected<E> err)
14         : error_(std::move(err).error()), has_value_(false) {}

```

```

15
16     ~expected() {
17         if (has_value_) value_.~T();
18         else            error_.~E();
19     }
20
21     bool has_value() const { return has_value_; }
22     T& operator*() { return value_; }
23     E& error() { return error_; }
24
25     T& value() {
26         if (!has_value_) throw std::bad_expected_access(error_);
27         return value_;
28     }
29 };

```

$\text{sizeof}(\text{std}::\text{expected}<\text{T}, \text{E}>) \approx \text{max}(\text{sizeof}(\text{T}), \text{sizeof}(\text{E})) + \text{sizeof}(\text{bool})$
+ выравнивание.

30.9. `std::expected<void, E>`

Специализация для функций, которые не возвращают значение, но могут вернуть ошибку.

```

1  std::expected<void, std::string> save_file(const std::string& path
2      ) {
3      std::ofstream f(path);
4      if (!f.is_open()) {
5          return std::unexpected("cannot open: " + path);
6      }
7      f << "data";
8      if (f.fail()) {
9          return std::unexpected("write failed");
10     }
11     return {}; // Успех (void)
12 }
13
14 int main() {
15     auto result = save_file("/tmp/test.txt");
16     if (!result) {
17         std::cerr << "Ошибка: " << result.error() << "\n";
18     }
19 }

```

30.10. Типичные паттерны использования

30.10.1. Парсинг с информативной ошибкой

```

1  #include <expected>
2  #include <string>

```

```

3  #include <charconv>
4
5  enum class ParseError {
6      empty_input,
7      invalid_format,
8      out_of_range,
9  };
10
11 std::expected<int, ParseError> parse_int(std::string_view sv) {
12     if (sv.empty()) {
13         return std::unexpected(ParseError::empty_input);
14     }
15
16     int result;
17     auto [ptr, ec] = std::from_chars(sv.data(), sv.data() + sv.
        size(), result);
18
19     if (ec == std::errc::invalid_argument) {
20         return std::unexpected(ParseError::invalid_format);
21     }
22     if (ec == std::errc::result_out_of_range) {
23         return std::unexpected(ParseError::out_of_range);
24     }
25
26     return result;
27 }
28
29 int main() {
30     auto r1 = parse_int("42");
31     if (r1) std::cout << "OK: " << *r1 << "\n";
32
33     auto r2 = parse_int("abc");
34     if (!r2) {
35         switch (r2.error()) {
36             case ParseError::empty_input:    std::cout << "Пусто\n"; break;
37             case ParseError::invalid_format: std::cout << "Формат\n"; break;
38             case ParseError::out_of_range:   std::cout << "
        Диапазон\n"; break;
39         }
40     }
41 }

```

30.10.2. Цепочка операций

```

1  using Result = std::expected<int, std::string>;
2
3  Result read_config_value(const std::string& key);
4  Result validate_range(int value);

```

```

5 Result apply_multiplier(int value);
6
7 // Элегантная цепочка без вложенных if:
8 auto final_result = read_config_value("timeout")
9     .and_then(validate_range)
10    .and_then(apply_multiplier)
11    .transform([](int x) { return x + 1; });
12
13 // Если любой шаг вернул ошибку -- она пробрасывается до конца
14 if (final_result) {
15     std::cout << "Результат: " << *final_result << "\n";
16 } else {
17     std::cerr << "Ошибка: " << final_result.error() << "\n";
18 }

```

30.11. Сводная таблица: optional vs expected vs исключения

	optional<T>	expected<T,E>	Исключения
Информация об ошибке	Нет	Да (тип E)	Да
Стоимость ошибки	Нулевая	Нулевая	Высокая
Стоимость успеха	Нулевая	Нулевая	Нулевая
Можно забыть проверить	Да	Да	Нет
Видимость в API	Явная	Явная	Скрытая
Стандарт	C++17	C++23	C++98

31. CRTP — Curiously Recurring Template Pattern

31.1. Что такое CRTP?

CRTP (Curiously Recurring Template Pattern) — паттерн, при котором класс наследуется от шаблонного базового класса, **передавая себя** в качестве шаблонного параметра:

```
1 template<typename Derived>
2 class Base {
3     // Base знает"" типсвоегонаследника
4 };
5
6 class MyClass : public Base<MyClass> {
7     // MyClass наследуетсяот Base<MyClass>
8 };
```

Выглядит странно: класс наследуется от шаблона, параметризованного *самим собой*. Отсюда слово «curiously» (любопытно) в названии.

31.1.1. Ключевая идея

Базовый класс Base<Derived> может **вызывать методы наследника** на этапе компиляции, без виртуальных функций:

```
1 template<typename Derived>
2 class Base {
3 public:
4     void interface() {
5         // Приводим this ктипунаследникаивызываемегометод
6         static_cast<Derived*>(this)->implementation();
7     }
8 };
9
10 class Concrete : public Base<Concrete> {
11 public:
12     void implementation() {
13         std::cout << "Concrete::implementation()\n";
14     }
15 };
16
17 int main() {
18     Concrete c;
19     c.interface(); // "Concrete::implementation()"
20 }
```

Base::interface() вызывает Derived::implementation() через static_cast. Никаких виртуальных таблиц, никаких косвенных вызовов — всё разрешается **на этапе компиляции**.

31.2. Зачем это нужно? Проблема виртуальных функций

Виртуальные функции — стандартный механизм полиморфизма в C++. Но у них есть цена:

	Виртуальные функции	CRTP
Полиморфизм	Динамический (runtime)	Статический (compile-time)
Накладные расходы	vptr + косвенный вызов через vtable	Нулевые (всё инлайнится)
Объект содержит vptr	Да (+8 байт на x86-64)	Нет
Можно хранить в одном контейнере	Да (vector<Base*>)	Нет (разные типы)
Определяется в	Runtime	Compile-time

CRTP — это **статический полиморфизм**: поведение определяется на этапе компиляции, а не в рантайме. Код получается таким же гибким, но **быстрее**, потому что компилятор видит конкретные типы и может инлайнить вызовы.

31.3. Как работает static_cast в CRTP?

Центральный приём CRTP — приведение this к типу наследника:

```
1 template<typename Derived>
2 class Base {
3 public:
4     Derived& self() {
5         return static_cast<Derived&>(*this);
6     }
7     const Derived& self() const {
8         return static_cast<const Derived&>(*this);
9     }
10 };
```

Почему это безопасно? Потому что Base<Derived> используется **только** как базовый класс для Derived. Объект всегда имеет тип Derived (или его наследника), поэтому static_cast вниз по иерархии здесь корректен.

Когда это опасно? Если кто-то нарушит контракт:

```
1 class A : public Base<A> { /* ... */ };
2 class B : public Base<A> { /* ... */ }; // BUG! B наследует Base<
    A>, а не Base<B>
3                                     // static_cast<A*>(this)
                                     // в объекте B = UB
```

Защита от этой ошибки рассмотрена в разделе [31.10](#).

31.4. Применение 1: Статический полиморфизм

Классический пример — интерфейс фигур без виртуальных функций.

31.4.1. С виртуальными функциями (обычный подход)

```
1 class Shape {
2 public:
3     virtual ~Shape() = default;
4     virtual double area() const = 0;
5     virtual void draw() const = 0;
6 };
7
8 class Circle : public Shape {
9     double radius_;
10 public:
11     explicit Circle(double r) : radius_(r) {}
12     double area() const override { return 3.14159 * radius_ *
        radius_; }
13     void draw() const override { std::cout << "Circle r=" <<
        radius_ << "\n"; }
14 };
```

31.4.2. С CRTP (статический полиморфизм)

```
1 template<typename Derived>
2 class Shape {
3 public:
4     double area() const {
5         return static_cast<const Derived*>(this)->area_impl();
6     }
7     void draw() const {
8         static_cast<const Derived*>(this)->draw_impl();
9     }
10 };
11
12 class Circle : public Shape<Circle> {
13     double radius_;
14 public:
15     explicit Circle(double r) : radius_(r) {}
16     double area_impl() const { return 3.14159 * radius_ * radius_;
        }
17     void draw_impl() const { std::cout << "Circle r=" << radius_
        << "\n"; }
18 };
19
20 class Square : public Shape<Square> {
21     double side_;
22 public:
23     explicit Square(double s) : side_(s) {}
24     double area_impl() const { return side_ * side_; }
25     void draw_impl() const { std::cout << "Square s=" << side_ <<
        "\n"; }
26 };
```

Использование с шаблонами вместо указателей на базовый класс:

```
1 // Шаблонная функция -- принимает любую фигуру
2 template<typename ShapeT>
3 void print_area(const Shape<ShapeT>& shape) {
4     std::cout << "Area: " << shape.area() << "\n";
5 }
6
7 int main() {
8     Circle c(5.0);
9     Square s(3.0);
10
11     print_area(c); // "Area: 78.5398"
12     print_area(s); // "Area: 9"
13
14     c.draw();      // "Circle r=5"
15     s.draw();      // "Square s=3"
16 }
```

Что происходит при компиляции:

- `Shape<Circle>::area()` инлайнится: компилятор подставляет `Circle::area_impl()` напрямую.
- Никакого `vptr`, никакого косвенного вызова через таблицу.
- `print_area` инстанцируется для каждого типа фигуры — получаются две отдельные, полностью оптимизированные функции.

31.5. Применение 2: Добавление функциональности (mixins)

CRTP позволяет базовому классу «добавлять» методы, которые используют интерфейс наследника. Это называется **mixin**.

31.5.1. Пример: автоматические операторы сравнения

До C++20 (до `operator<=>`) приходилось писать все 6 операторов вручную. CRTP позволяет вывести их из одного:

```
1 // Mixin: требует от наследника только operator< и operator==
2 template<typename Derived>
3 class Comparable {
4 public:
5     friend bool operator>(const Derived& a, const Derived& b) {
6         return b < a;
7     }
8     friend bool operator<=(const Derived& a, const Derived& b) {
9         return !(b < a);
10    }
11    friend bool operator>=(const Derived& a, const Derived& b) {
12        return !(a < b);
13    }
14    friend bool operator!=(const Derived& a, const Derived& b) {
```

```

15         return !(a == b);
16     }
17 };
18
19 class Temperature : public Comparable<Temperature> {
20     double value_;
21 public:
22     explicit Temperature(double v) : value_(v) {}
23
24     // Нужно определить только два оператора :
25     friend bool operator<(const Temperature& a, const Temperature&
26         b) {
27         return a.value_ < b.value_;
28     }
29     friend bool operator==(const Temperature& a, const Temperature
30         & b) {
31         return a.value_ == b.value_;
32     }
33 };
34
35 int main() {
36     Temperature t1(36.6), t2(37.0);
37
38     std::cout << (t1 < t2) << "\n";    // 1 -- определён вручную
39     std::cout << (t1 > t2) << "\n";    // 0 -- сгенерирован CRTP
40     std::cout << (t1 <= t2) << "\n";   // 1 -- сгенерирован CRTP
41     std::cout << (t1 != t2) << "\n";   // 1 -- сгенерирован CRTP
42 }

```

Примечание: в C++20 для этого есть оператор `<=>` (spaceship operator), который решает ту же задачу средствами языка. Но паттерн `mixin` через CRTP применим и к другим задачам.

31.5.2. Пример: автоматический оператор `!=` и постфиксный `++`

```

1 // Mixin для итераторов : из prefix ++ и == генерирует != и postfix
2 // ++
3 template<typename Derived>
4 class IteratorMixin {
5 public:
6     // Постфиксный ++ через префиксный
7     Derived operator++(int) {
8         Derived old = static_cast<Derived&>(*this);
9         ++static_cast<Derived&>(*this); // Вызывает prefix ++
10        return old;
11    }
12
13    friend bool operator!=(const Derived& a, const Derived& b) {
14        return !(a == b);
15    }
16 };

```

```

16
17 class Counter : public IteratorMixin<Counter> {
18     int value_;
19 public:
20     explicit Counter(int v) : value_(v) {}
21
22     // Определяем только prefix ++ и ==
23     Counter& operator++() { ++value_; return *this; }
24
25     friend bool operator==(const Counter& a, const Counter& b) {
26         return a.value_ == b.value_;
27     }
28
29     int get() const { return value_; }
30 };
31
32 int main() {
33     Counter c(0);
34     Counter end(5);
35
36     while (c != end) {          // != сгенерирован CRTP
37         std::cout << c.get() << " ";
38         c++;                    // postfix ++ сгенерирован CRTP
39     }
40     // 0 1 2 3 4
41 }

```

31.6. Применение 3: Подсчёт объектов

CRTP позволяет создать счётчик экземпляров, **индивидуальный для каждого класса**:

```

1 template<typename Derived>
2 class Counter {
3     static inline int count_ = 0; // inline static (C++17)
4 public:
5     Counter() { ++count_; }
6     Counter(const Counter&) { ++count_; }
7     ~Counter() { --count_; }
8
9     static int alive() { return count_; }
10 };
11
12 class Dog : public Counter<Dog> {
13     std::string name_;
14 public:
15     explicit Dog(std::string n) : name_(std::move(n)) {}
16 };
17
18 class Cat : public Counter<Cat> {
19     std::string name_;

```

```

20 public:
21     explicit Cat(std::string n) : name_(std::move(n)) {}
22 };
23
24 int main() {
25     Dog d1("Шарик"), d2("Бобик");
26     Cat c1("Мурка");
27
28     std::cout << "Собак: " << Dog::alive() << "\n"; // 2
29     std::cout << "Кошек: " << Cat::alive() << "\n"; // 1
30
31     {
32         Dog d3("Пекс");
33         std::cout << "Собак: " << Dog::alive() << "\n"; // 3
34     }
35     std::cout << "Собак: " << Dog::alive() << "\n"; // 2
36 }

```

Почему без CRTP это не работает?

```

1 // Без CRTP: один счётчик на ВСЕ классы
2 class CounterBad {
3     static inline int count_ = 0;
4 public:
5     CounterBad() { ++count_; }
6     ~CounterBad() { --count_; }
7     static int alive() { return count_; }
8 };
9
10 class Dog : public CounterBad {};
11 class Cat : public CounterBad {};
12
13 // Dog::alive() и Cat::alive() -- это ОДИН И ТОТ ЖЕ счётчик !
14 // Потому что Dog и Cat наследуют ОДИН и тот же CounterBad

```

С CRTP Counter<Dog> и Counter<Cat> — это **разные классы**, каждый со своей статической переменной count_.

31.7. Применение 4: Singleton через CRTP

```

1 template<typename Derived>
2 class Singleton {
3 protected:
4     Singleton() = default;
5     ~Singleton() = default;
6
7     Singleton(const Singleton&) = delete;
8     Singleton& operator=(const Singleton&) = delete;
9
10 public:
11     static Derived& instance() {
12         static Derived inst; // Meyer's singleton

```

```

13         return inst;
14     }
15 };
16
17 class Logger : public Singleton<Logger> {
18     friend class Singleton<Logger>; // Даём доступ к private
19     конструктору
20     Logger() = default;
21
22 public:
23     void log(const std::string& msg) {
24         std::cout << "[LOG] " << msg << "\n";
25     }
26 };
27
28 class Config : public Singleton<Config> {
29     friend class Singleton<Config>;
30     Config() = default;
31
32     int port_ = 8080;
33 public:
34     int port() const { return port_; }
35 };
36
37 int main() {
38     Logger::instance().log("started"); // "[LOG] started"
39     std::cout << Config::instance().port() << "\n"; // 8080
40
41     // Logger l; // Ошибка компиляции ! Конструктор protected
42 }

```

31.8. Применение 5: Метод clone() без повторения кода

При глубоком наследовании каждый класс должен переопределять clone() с правильным типом. CRTP автоматизирует это:

```

1 #include <memory>
2
3 class Animal {
4 public:
5     virtual ~Animal() = default;
6     virtual std::unique_ptr<Animal> clone() const = 0;
7     virtual void speak() const = 0;
8 };
9
10 // CRTP прослойка-, которая реализует clone() за наследника
11 template<typename Derived>
12 class Cloneable : public Animal {
13 public:
14     std::unique_ptr<Animal> clone() const override {
15         // Derived ОБЯЗАНА иметь копирующий конструктор
16     }
17 };

```

```

16         return std::make_unique<Derived>(
17             static_cast<const Derived&>(*this)
18         );
19     }
20 };
21
22 // Теперь наследникам не нужно писать clone() вручную:
23 class Dog : public Cloneable<Dog> {
24     std::string name_;
25 public:
26     explicit Dog(std::string n) : name_(std::move(n)) {}
27     void speak() const override {
28         std::cout << name_ << " говорит: Гав!\n";
29     }
30 };
31
32 class Cat : public Cloneable<Cat> {
33     std::string name_;
34 public:
35     explicit Cat(std::string n) : name_(std::move(n)) {}
36     void speak() const override {
37         std::cout << name_ << " говорит: Мяу!\n";
38     }
39 };
40
41 int main() {
42     std::unique_ptr<Animal> dog = std::make_unique<Dog>("Шарик");
43     dog->speak(); // Шарик" говорит: Гав!"
44
45     auto cloned = dog->clone();
46     cloned->speak(); // Шарик" говорит: Гав!" -- это КОПИЯ
47 }

```

Без CRTP пришлось бы копировать реализацию `clone()` в каждый класс.

31.9. Применение 6: Статический интерфейс («концепты до концептов»)

До C++20 CRTP использовали для проверки интерфейса на этапе компиляции:

```

1 template<typename Derived>
2 class Printable {
3 public:
4     // Derived обязан иметь метод to_string()
5     friend std::ostream& operator<<(std::ostream& os, const
6         Derived& obj) {
7         os << obj.to_string();
8         return os;
9     }
10 };

```

```

11 class Point : public Printable<Point> {
12     double x_, y_;
13 public:
14     Point(double x, double y) : x_(x), y_(y) {}
15
16     std::string to_string() const {
17         return "(" + std::to_string(x_) + ", " + std::to_string(y_)
18             + ")";
19     };
20
21 class Color : public Printable<Color> {
22     int r_, g_, b_;
23 public:
24     Color(int r, int g, int b) : r_(r), g_(g), b_(b) {}
25
26     std::string to_string() const {
27         return "rgb(" + std::to_string(r_) + "," +
28             std::to_string(g_) + "," + std::to_string(b_) + ")"
29             ;
30     };
31
32 int main() {
33     Point p(1.5, 2.3);
34     Color c(255, 128, 0);
35
36     std::cout << p << "\n"; // "(1.500000, 2.300000)"
37     std::cout << c << "\n"; // "rgb(255,128,0)"
38 }

```

Наследник определяет `to_string()`, а CRTP-база автоматически генерирует оператор `<<`. Если наследник забудет определить `to_string()` — ошибка компиляции.

31.10. Защита от ошибок

31.10.1. Проблема: неправильный шаблонный аргумент

```

1 class A : public Base<A> {};
2 class B : public Base<A> {}; // Ошибка! B передал A вместо B
3                               // static_cast<A*>(this)
                               // наобъекте B = UB

```

31.10.2. Решение: проверка в конструкторе

```

1 template<typename Derived>
2 class Base {
3     // Конструктор private, только Derived может вызывать через (
4     // наследование)

```



```

4     Base() = default;
5     friend Derived; // Только Derived имеет доступ к конструктору
6 public:
7     void interface() {
8         static_cast<Derived*>(this)->implementation();
9     }
10 };
11
12 class A : public Base<A> {}; // OK: A -- friend Base<A>
13 // class B : public Base<A> {}; // Ошибка компиляции !
14 // B не friend Base<A>,
//     не может вызывать конструктор

```

31.10.3. Решение C++20: static_assert с концептами

```

1 template<typename Derived>
2 class Base {
3 public:
4     Base() {
5         // Проверяем на этапе компиляции, что Derived
//         действительно наследник
6         static_assert(std::is_base_of_v<Base<Derived>, Derived>,
7             "CRTP: Derived must inherit from Base<
//             Derived>");
8     }
9 };

```

31.11. CRTP и множественное наследование

Класс может наследовать **несколько** CRTP-баз — каждая добавляет свою функциональность:

```

1 template<typename Derived>
2 class Serializable {
3 public:
4     std::string serialize() const {
5         return static_cast<const Derived*>(this)->serialize_impl()
6         ;
7     }
8 };
9
10 template<typename Derived>
11 class Loggable {
12 public:
13     void log(const std::string& msg) const {
14         std::cout << "[" << static_cast<const Derived*>(this)->
15             class_name()
16             << "]" << msg << "\n";
17     }
18 };

```

```

16 };
17
18 // User наследуетоба mixina'
19 class User : public Serializable<User>, public Loggable<User> {
20     std::string name_;
21     int age_;
22 public:
23     User(std::string name, int age) : name_(std::move(name)), age_
        (age) {}
24
25     // Для Serializable
26     std::string serialize_impl() const {
27         return "{\\"name\\":\\" + name_ + "\",\\"age\\":\\" + std::
            to_string(age_) + "\\"}";
28     }
29
30     // Для Loggable
31     const char* class_name() const { return "User"; }
32 };
33
34 int main() {
35     User u("Alice", 25);
36     std::cout << u.serialize() << "\n"; // {"name":"Alice","age
        ":25}
37     u.log("created"); // [User] created
38 }

```

31.12. CRTP vs виртуальные функции: когда что?

Критерий	Виртуальные функции	CRTP
Тип известен на этапе компиляции	Не обязательно	Да, всегда
Нужен <code>vector<Base*></code>	Да, это единственный способ	Нет (разные разные типы)
Критична производительность	Косвенный вызов, нет инлайнинга	Нулевые затраты инлайнинга
Количество типов растёт в рантайме (плагины)	Подходит	Не подходит
Добавление общей функциональности (mixin)	Неудобно	Идеально
Простота понимания	Привычно, интуитивно	Требуется знание

Правило:

- Если набор типов известен **на этапе компиляции** и важна производительность — CRTP.
- Если нужен **runtime-полиморфизм** (контейнер разнородных объектов, плагины, фабрики) — виртуальные функции.

- Если нужно **добавить поведение** к разным классам (mixin) — CRTP.

31.13. Пример из реального мира: `std::enable_shared_from_this`

Стандартная библиотека использует CRTP в `std::enable_shared_from_this<T>`:

```

1 #include <memory>
2
3 class Widget : public std::enable_shared_from_this<Widget> {
4 public:
5     std::shared_ptr<Widget> get_ptr() {
6         return shared_from_this(); // Метод из CRTP-базы
7     }
8
9     void do_work() {
10         // Безопасно создаём shared_ptr на себя
11         auto self = shared_from_this();
12         // Можно передать self в асинхронную операцию,
13         // гарантируя, что объект не будет уничтожен
14     }
15 };
16
17 int main() {
18     auto w = std::make_shared<Widget>();
19     auto w2 = w->get_ptr(); // w и w2 разделяют владение
20
21     std::cout << w.use_count() << "\n"; // 2
22 }

```

`enable_shared_from_this<Widget>` — это CRTP-база. Она хранит внутри `weak_ptr<Widget>`, который инициализируется при создании `shared_ptr<Widget>`. Метод `shared_from_this()` возвращает `shared_ptr`, созданный из этого `weak_ptr`.

31.14. Deducing this (C++23) — замена CRTP?

C++23 ввёл **deducing this** — явный объектный параметр, который упрощает многие сценарии, ранее требовавшие CRTP:

```

1 // CRTP до( C++23):
2 template<typename Derived>
3 class Comparable {
4     friend bool operator!=(const Derived& a, const Derived& b) {
5         return !(a == b);
6     }
7 };
8
9 class Int : public Comparable<Int> {
10     int val_;
11 public:
12     explicit Int(int v) : val_(v) {}
13     friend bool operator==(const Int& a, const Int& b) {
14         return a.val_ == b.val_;
15     }
16 };

```

```

15     }
16 };

```

```

1 // Deducing this (C++23):
2 class Comparable23 {
3     // this передаётся явно, тип выводится автоматически
4     bool operator!=(this const auto& self, const auto& other) {
5         return !(self == other);
6     }
7 };
8
9 class Int23 : public Comparable23 {
10     int val_;
11 public:
12     explicit Int23(int v) : val_(v) {}
13     friend bool operator==(const Int23& a, const Int23& b) {
14         return a.val_ == b.val_;
15     }
16 };

```

С deducing this не нужен шаблонный параметр — компилятор сам выводит конкретный тип `self`. Код проще и безопаснее (нет риска передать неправильный тип в шаблон базы).

Однако CRTP не устарел:

- Deducing this доступен только с C++23
- CRTP работает и для статических членов (подсчёт объектов, singleton)
- Многие библиотеки и кодовые базы используют CRTP
- Понимание CRTP необходимо для чтения существующего кода

31.15. Итого: шпаргалка по CRTP

Применение	Суть
Статический полиморфизм	Вызов методов наследника без <code>virtual</code>
Mixin (добавление функциональности)	Генерация операторов, <code>operator<<</code> , <code>clone()</code> и т. д.
Подсчёт объектов	Отдельный <code>static</code> счётчик для каждого класса
Singleton	Шаблонный <code>instance()</code> для любого класса
<code>enable_shared_from_this</code>	Получение <code>shared_ptr</code> на <code>this</code>
Избежание дублирования	Один <code>clone()</code> , один <code>serialize()</code> в базе

32. Указатели на функции, методы, поля. Лямбды. Type erasure. `std::function`

32.1. Указатели на функции

32.1.1. Что такое указатель на функцию

Функция в C++ занимает определённое место в памяти (сегмент кода). Указатель на функцию хранит **адрес точки входа** функции и позволяет вызывать её косвенно — через указатель.

```
1 #include <iostream>
2
3 int add(int a, int b) { return a + b; }
4 int mul(int a, int b) { return a * b; }
5
6 int main() {
7     // Объявление указателя на функцию      int(int,int)
8     int (*op)(int, int) = nullptr;
9
10    op = add;           // имя функции неявно приводится к указателю
11    std::cout << op(2, 3) << "\n";    // 5
12
13    op = &mul;          // явное взятие адреса    -- тоже допустимо
14    std::cout << op(2, 3) << "\n";    // 6
15 }
```

Синтаксис объявления:

возвращаемый_тип (*имя)(список_параметров)

Скобки вокруг *имя обязательны: без них получится объявление функции, возвращающей указатель.

32.1.2. Использование `typedef` и `using`

Синтаксис указателей на функции громоздкий. Упрощаем:

```
1 // C-style typedef
2 typedef int (*BinaryOp)(int, int);
3
4 // C++11 using предпочтительно()
5 using BinaryOp = int(*)(int, int);
6
7 BinaryOp op = add;
8 std::cout << op(4, 5) << "\n";    // 9
```

32.1.3. Передача указателя на функцию как параметр

```

1 void apply_and_print(int a, int b, int (*func)(int, int)) {
2     std::cout << func(a, b) << "\n";
3 }
4
5 // Илис using:
6 using BinaryOp = int (*)(int, int);
7 void apply_and_print(int a, int b, BinaryOp func) {
8     std::cout << func(a, b) << "\n";
9 }
10
11 int main() {
12     apply_and_print(10, 20, add); // 30
13     apply_and_print(10, 20, mul); // 200
14 }

```

Это классический приём в C — callback-функции. Именно так работают `qsort`, `signal` и другие C-функции.

32.1.4. Массив указателей на функции

```

1 int sub(int a, int b) { return a - b; }
2
3 int main() {
4     BinaryOp ops[] = {add, mul, sub};
5     for (auto op : ops) {
6         std::cout << op(10, 3) << " "; // 13 30 7
7     }
8 }

```

32.1.5. Указатели на функции и шаблоны

Шаблонная функция не является одной функцией — это семейство. Нужно указать конкретную инстанциацию:

```

1 template<typename T>
2 T maximum(T a, T b) { return a > b ? a : b; }
3
4 int main() {
5     int (*p)(int, int) = maximum<int>; // OK:
6     // конкретная инстанция
7     // int (*q)(int, int) = maximum; // Ошибка: какая именно?
8     std::cout << p(3, 7) << "\n"; // 7
9 }

```

32.1.6. Указатели на лямбда-функции (C++17)

C++17 лямбда является частью типа функции:

```

1 void foo() noexcept {}
2 void bar() {}
3
4 void (*p1)() noexcept = foo;    // OK
5 void (*p2)()           = foo;    // OK: noexcept -> обычный неявное (
    приведение)
6 // void (*p3)() noexcept = bar; // Ошибка: обычный -> noexcept
    нельзя

```

32.2. Указатели на методы (member function pointers)

32.2.1. Синтаксис

Методы класса — это **не обычные функции**. Они принимают скрытый параметр `this`. Поэтому обычный указатель на функцию для них **не подходит**:

```

1 struct Calculator {
2     int add(int a, int b) { return a + b; }
3     int mul(int a, int b) { return a * b; }
4 };
5
6 int main() {
7     // Тип: указатель на метод   Calculator, принимающий (int,int),
        // возвращающий int
8     int (Calculator::*method)(int, int) = &Calculator::add;
9
10    Calculator calc;
11    // Вызов через объект :
12    std::cout << (calc.*method)(2, 3) << "\n";    // 5
13
14    // Вызов через указатель на объект :
15    Calculator* ptr = &calc;
16    std::cout << (ptr->*method)(2, 3) << "\n";    // 5
17
18    method = &Calculator::mul;
19    std::cout << (calc.*method)(2, 3) << "\n";    // 6
20 }

```

Важно: оператор `&` при взятии адреса метода **обязателен** (в отличие от обычных функций, где имя неявно конвертируется в указатель).

32.2.2. Синтаксис с `using`

```

1 using CalcMethod = int (Calculator::*)(int, int);
2 CalcMethod m = &Calculator::add;

```

32.2.3. Указатели на `const`-методы

```

1 struct Foo {
2     int value() const { return 42; }
3 };
4
5 // Обратителен внимание на const:
6 int (Foo::*p)() const = &Foo::value;
7
8 const Foo f;
9 std::cout << (f.*p)() << "\n"; // 42

```

32.2.4. Указатели на статические методы

Статические методы **не имеют** `this`, поэтому для них используется **обычный** указатель на функцию:

```

1 struct Utils {
2     static int square(int x) { return x * x; }
3 };
4
5 int (*p)(int) = &Utils::square; // обычный указатель на функцию !
6 std::cout << p(5) << "\n"; // 25

```

32.2.5. Использование `std::invoke` (C++17)

Операторы `.*` и `->*` неудобны. `std::invoke` унифицирует вызов любого callable-объекта:

```

1 #include <functional>
2
3 Calculator calc;
4 auto method = &Calculator::add;
5
6 // Вместо (calc.*method)(2, 3):
7 std::cout << std::invoke(method, calc, 2, 3) << "\n"; // 5
8
9 // Работает и с указателем на объект:
10 std::cout << std::invoke(method, &calc, 2, 3) << "\n"; // 5

```

32.3. Указатели на поля (pointer to data member)

32.3.1. Синтаксис

Можно взять «адрес» поля класса. На самом деле это не настоящий адрес, а **сместе-ние** поля относительно начала объекта:

```

1 struct Point {
2     double x;
3     double y;
4     double z;

```



```

5 };
6
7 int main() {
8     // Указатель на поле типа double класса Point
9     double Point::*coord = &Point::x;
10
11     Point p{1.0, 2.0, 3.0};
12
13     std::cout << p.*coord << "\n";    // 1.0 (x)
14
15     coord = &Point::y;
16     std::cout << p.*coord << "\n";    // 2.0 (y)
17
18     coord = &Point::z;
19     std::cout << p.*coord << "\n";    // 3.0 (z)
20 }

```

32.3.2. Практическое применение: обобщённый доступ

```

1 struct Student {
2     std::string name;
3     int age;
4     double gpa;
5 };
6
7 // Функция, сортирующая по полю
8 template<typename T>
9 void sort_by(std::vector<Student>& v, T Student::*field) {
10     std::sort(v.begin(), v.end(),
11         [field](const Student& a, const Student& b) {
12             return a.*field < b.*field;
13         });
14 }
15
16 int main() {
17     std::vector<Student> students = {
18         {"Alice", 20, 3.8},
19         {"Bob", 22, 3.5},
20         {"Carol", 19, 3.9}
21     };
22
23     sort_by(students, &Student::age);    // по возрасту
24     sort_by(students, &Student::gpa);    // по GPA
25     sort_by(students, &Student::name);   // по имени
26 }

```

32.3.3. Доступ через std::invoke

std::invoke работает и с указателями на поля:

```

1 Student s{"Alice", 20, 3.8};
2 auto ptr = &Student::name;
3 std::cout << std::invoke(ptr, s) << "\n"; // "Alice"

```

32.4. Лямбда-выражения

32.4.1. Мотивация

До C++11 для передачи «кусочка логики» в алгоритм нужно было писать отдельную функцию или функтор (класс с `operator()`):

```

1 // Функтор -- классс operator()
2 struct IsEven {
3     bool operator()(int x) const { return x % 2 == 0; }
4 };
5
6 std::vector<int> v = {1, 2, 3, 4, 5, 6};
7 auto it = std::find_if(v.begin(), v.end(), IsEven{});

```

Это слишком многословно для простой проверки. Лямбды решают эту проблему.

32.4.2. Базовый синтаксис

```

1 auto it = std::find_if(v.begin(), v.end(),
2     [](int x) { return x % 2 == 0; });

```

Общий вид лямбда-выражения:

[захват](параметры) спецификаторы -> тип_возврата { тело }

- [захват] — какие переменные из окружающего контекста доступны внутри лямбды;
- (параметры) — как у обычной функции;
- спецификаторы — `mutable`, `noexcept`, `constexpr` и др.;
- -> тип_возврата — можно опустить, если компилятор выведет сам;
- { тело } — код лямбды.

32.4.3. Что такое лямбда на самом деле?

Лямбда — это **синтаксический сахар** для создания анонимного класса с `operator()`. Компилятор генерирует примерно следующее:

```

1 // Лямбда:
2 auto f = [](int x) { return x * 2; };
3
4 // Компиляторгенерируетчтото - вроде:

```

```

5 class __lambda_1 {
6 public:
7     int operator()(int x) const { // const по умолчанию !
8         return x * 2;
9     }
10 };
11 auto f = __lambda_1{};

```

Каждая лямбда имеет **уникальный тип**, даже если тело идентично:

```

1 auto f1 = [](int x) { return x; };
2 auto f2 = [](int x) { return x; };
3 // decltype(f1) != decltype(f2) -- это разные типы !

```

32.4.4. Захват переменных (capture)

[] — без захвата. Лямбда не видит локальные переменные:

```

1 int threshold = 10;
2 // auto f = [](int x) { return x > threshold; }; // Ошибка!

```

[=] --- захват всех по значению (копия).

```

1 int threshold = 10;
2 auto f = [=](int x) { return x > threshold; };
3 // threshold скопирован внутри лямбды
4 threshold = 100; // не повлияет на f
5 std::cout << f(50); // true сравнивается с 10

```

[&] --- захват всех по ссылке.

```

1 int count = 0;
2 auto increment = [&]() { ++count; };
3 increment();
4 increment();
5 std::cout << count; // 2

```

Осторожно: захваченная ссылка может стать висячей!

```

1 std::function<int()> make_counter() {
2     int n = 0;
3     return [&n]() { return ++n; }; // ОШИБКА: n
4     // уничтожен при выходе !
5 }

```

Поимённый захват

```

1 int a = 1, b = 2, c = 3;
2 auto f = [a, &b](int x) {
3     // a -- копия (read-only, тк.. operator() const)
4     // b -- ссылка (можно менять)
5     // c -- не захвачен, недоступен
6     b += a + x;
7 };

```

Смешанный захват

```
1 auto f = [=, &b]() { ... }; // всепозначению , b посылке
2 auto g = [&, a]() { ... }; // всепосылке , a позначению
```

32.4.5. Захват this

Внутри метода класса лямбда может захватить this:

```
1 class Timer {
2     int interval_ = 100;
3 public:
4     auto make_callback() {
5         // [this] -- захват указателя this
6         return [this]() {
7             std::cout << "interval = " << interval_ << "\n";
8         };
9     }
10
11     auto make_safe_callback() {
12         // [*this] (C++17) -- захват копии всего объекта
13         return [*this]() {
14             std::cout << "interval = " << interval_ << "\n";
15         };
16     }
17 };
```

[this] — захватывает **указатель**. Если объект уничтожен, будет dangling pointer.

[*this] (C++17) — захватывает **копию** объекта. Безопасно, но дорого для больших объектов.

32.4.6. Init-capture (обобщённый захват, C++14)

Позволяет создать новую переменную внутри захвата:

```
1 // Переместить вектор в лямбду (не копировать):
2 std::vector<int> data = {1, 2, 3, 4, 5};
3 auto f = [data = std::move(data)]() {
4     for (int x : data) std::cout << x << " ";
5 };
6 // data здесь пуст -- он перемещён в лямбду
7
8 // Захват с переименованием:
9 int x = 42;
10 auto g = [value = x * 2]() { return value; }; // value == 84
11
12 // Захват unique_ptr не копируемый (тип):
13 auto ptr = std::make_unique<int>(42);
14 auto h = [p = std::move(ptr)]() { return *p; };
```

32.4.7. Ключевое слово mutable

По умолчанию `operator()` лямбды — `const`. Чтобы изменять захваченные по значению переменные, нужен `mutable`:

```
1 int counter = 0;
2 auto inc = [counter]() mutable {
3     return ++counter; // OK: mutable позволяет менять копию
4 };
5 std::cout << inc() << "\n"; // 1
6 std::cout << inc() << "\n"; // 2
7 std::cout << counter << "\n"; // 0 -- оригинал не изменён !
```

Без `mutable`:

```
1 auto bad = [counter]() {
2     // return ++counter; // Ошибка: operator() const
3 };
```

32.4.8. Лямбда без захвата конвертируется в указатель на функцию

Лямбда **без захвата** может быть неявно преобразована в указатель на функцию с подходящей сигнатурой:

```
1 int (*fp)(int, int) = [](int a, int b) { return a + b; };
2 std::cout << fp(3, 4) << "\n"; // 7
3
4 // Это позволяет передавать лямбды в С API:
5 void c_api(void (*callback)(int)) { callback(42); }
6 c_api([](int x) { std::cout << x << "\n"; }); // OK
```

Лямбда **с захватом** в указатель на функцию **не конвертируется**:

```
1 int offset = 10;
2 // int (*fp)(int) = [offset](int x) { return x + offset; }; //
   Ошибка!
```

32.4.9. Generic-лямбды (C++14)

Параметры лямбды можно объявить как `auto` — компилятор сделает шаблонный `operator()`:

```
1 auto print = [](const auto& x) {
2     std::cout << x << "\n";
3 };
4 print(42); // int
5 print(3.14); // double
6 print("hello"s); // std::string
```

Что генерирует компилятор:

```
1 class __lambda {
2 public:
3     template<typename T>
```

```

4     void operator()(const T& x) const {
5         std::cout << x << "\n";
6     }
7 };

```

32.4.10. Шаблонные лямбды (C++20)

C++20 позволяет явно указать шаблонные параметры:

```

1 // Ограничение: только целочисленные типы
2 auto sum = [<typename T>(const std::vector<T>& v) requires std::
   integral<T> {
3     T result = 0;
4     for (auto x : v) result += x;
5     return result;
6 };
7
8 std::vector<int> vi = {1, 2, 3};
9 std::cout << sum(vi) << "\n"; // 6
10
11 // std::vector<double> vd = {1.0, 2.0};
12 // sum(vd); // Ошибка: double не integral
13 \end{lstlisting}
14
15 \subsection{Рекурсивные лямбды}
16     Лямбда не может вызывать сама себя напрямую
17     , потому что \texttt{auto} ещё не выведен в момент
18     ~ определения тела . Решения:
19
20 \begin{lstlisting}
21 // 1. Через std::function с накладными расходами ):
22 std::function<int(int)> factorial = [&factorial](int n) -> int {
23     return n <= 1 ? 1 : n * factorial(n - 1);
24 };
25
26 // 2. Передача самой себя как параметра без ( накладных расходов ):
27 auto factorial2 = [](auto self, int n) -> int {
28     return n <= 1 ? 1 : n * self(self, n - 1);
29 };
30 std::cout << factorial2(factorial2, 5) << "\n"; // 120
31
32 // 3. C++23: deducing this
33 auto factorial3 = [](this auto self, int n) -> int {
34     return n <= 1 ? 1 : n * self(n - 1);
35 };

```

32.4.11. Immediately Invoked Lambda Expression (IILE)

Лямбду можно вызвать сразу на месте — удобно для инициализации сложных const-переменных:

```
1 const auto config = [&]() {  
2     Config c;  
3     c.width = parse_int(argv[1]);  
4     c.height = parse_int(argv[2]);  
5     c.fullscreen = (argc > 3);  
6     return c;  
7 }(); // <-- вызовнаместе !  
8 // config -- const, инициализированодинраз
```

32.5. Указатели на функции, функторы и лямбды — сравнение

Свойство	Указатель	Функтор	Лямбда
Состояние (захват)	Нет	Да	Да
inline-оптимизация	Нет ¹	Да	Да
Передача в C API	Да	Нет	Только без захвата
Тип известен	Да	Да	Нет ²

32.6. Type erasure (стирание типа)

32.6.1. Проблема

Мы хотим хранить callable-объекты разных типов в одном контейнере или передавать их через единый интерфейс. Но:

- Указатели на функции не хранят состояние.
- Каждая лямбда — уникальный тип, нельзя положить в один `std::vector`.
- Шаблоны — compile-time решение, а нам нужен runtime полиморфизм.

```
1 auto f = [x=1](int a) { return a + x; };  
2 auto g = [y=2](int a) { return a * y; };  
3 // decltype(f) != decltype(g)  
4  
5 // std::vector<???
```

32.6.2. Идея type erasure

Type erasure — техника, позволяющая «стереть» конкретный тип объекта, сохранив возможность вызывать его операции через единый интерфейс. Это runtime-полиморфизм **без** наследования пользователем от базового класса.

Концептуально type erasure состоит из трёх компонентов:

1. **Внешний класс** (wrapper) — то, с чем работает пользователь. Имеет value-семантику (копируется, перемещается).
2. **Внутренний абстрактный интерфейс** — определяет набор операций.
3. **Шаблонная реализация** — конкретная обёртка вокруг пользовательского типа, реализующая интерфейс.

32.6.3. Пример: собственный AnyCallable

Реализуем упрощённый аналог `std::function<int(int)>`:

```

1  class AnyCallable {
2      // Шаг 1: абстрактный интерфейс
3      struct Concept {
4          virtual int call(int) = 0;
5          virtual ~Concept() = default;
6          virtual std::unique_ptr<Concept> clone() const = 0;
7      };
8
9      // Шаг 2: шаблонная реализация для любого callable
10     template<typename F>
11     struct Model : Concept {
12         F func_;
13         Model(F f) : func_(std::move(f)) {}
14         int call(int x) override { return func_(x); }
15         std::unique_ptr<Concept> clone() const override {
16             return std::make_unique<Model>(func_);
17         }
18     };
19
20     // Шаг 3: хранение через указатель на базу
21     std::unique_ptr<Concept> ptr_;
22
23     public:
24         // Шаблонный конструктор -- принимает что угодно callable
25         template<typename F>
26         AnyCallable(F f) : ptr_(std::make_unique<Model<F>>(std::move(f))) {}
27
28         // Копирование через ( clone )
29         AnyCallable(const AnyCallable& other) : ptr_(other.ptr_->clone()) {}
30         AnyCallable& operator=(const AnyCallable& other) {
31             ptr_ = other.ptr_->clone();
32             return *this;
33         }
34
35         // Перемещение
36         AnyCallable(AnyCallable&&) = default;
37         AnyCallable& operator=(AnyCallable&&) = default;
38

```

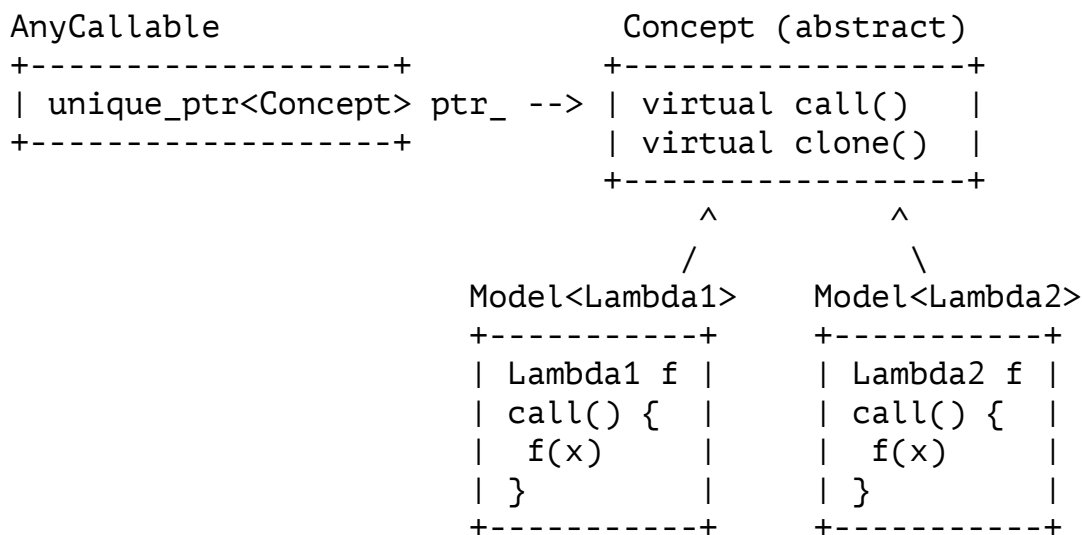


```

39 // Вызов
40 int operator()(int x) { return ptr_->call(x); }
41 };
42
43 int main() {
44     AnyCallable f = [x=1](int a) { return a + x; };
45     AnyCallable g = [y=2](int a) { return a * y; };
46
47     std::vector<AnyCallable> funcs = {f, g};
48     for (auto& func : funcs) {
49         std::cout << func(10) << " "; // 11 20
50     }
51 }

```

32.6.4. Как это работает (диаграмма)



Ключевая идея: шаблонный конструктор AnyCallable(F f) «стирает» тип F, оборачивая его в Model<F>, а хранит указатель на нешаблонный Concept.

32.6.5. Type erasure в стандартной библиотеке

Type erasure — не экзотика. Стандартная библиотека активно его использует:

Класс	Что стирает
std::function<R(Args...)>	Любой callable с подходящей сигнатурой
std::any (C++17)	Любой копируемый тип
std::move_only_function (C++23)	Любой move-only callable
std::format_args	Аргументы форматирования

32.7. std::function

32.7.1. Что это

`std::function<R(Args...)>` — обёртка из `<functional>`, которая может хранить **любой** вызываемый объект с подходящей сигнатурой:

- обычную функцию;
- лямбду (в том числе с захватом);
- функтор (объект с `operator()`);
- указатель на метод (через `std::bind` или лямбду);
- результат `std::bind`.

```
1 #include <functional>
2
3 int add(int a, int b) { return a + b; }
4
5 struct Multiplier {
6     int factor;
7     int operator()(int x) const { return x * factor; }
8 };
9
10 int main() {
11     // Хранит обычную функцию
12     std::function<int(int, int)> f1 = add;
13     std::cout << f1(2, 3) << "\n";          // 5
14
15     // Хранит лямбду с захватом
16     int offset = 10;
17     std::function<int(int)> f2 = [offset](int x) { return x +
18         offset; };
19     std::cout << f2(5) << "\n";              // 15
20
21     // Хранит функтор
22     std::function<int(int)> f3 = Multiplier{3};
23     std::cout << f3(7) << "\n";              // 21
24 }
```

32.7.2. Проверка на пустоту

`std::function` может быть пустой (не содержать callable):

```
1 std::function<void()> f;          // пустая
2 if (f) {                          // проверка на непустоту
3     f();
4 } else {
5     std::cout << "empty!\n";      // сюда попадём
6 }
```

```

7
8 f = []() { std::cout << "hello\n"; };
9 if (f) f(); // "hello"
10
11 f = nullptr; // сновапустая
12 // f(); // выбросит std::bad_function_call!

```

32.7.3. Накладные расходы std::function

std::function использует type erasure внутри, что влечёт расходы:

1. **Динамическая аллокация.** Если callable не помещается в SBO (Small Buffer Optimization, обычно 16–32 байт), выделяется память в куче.
2. **Виртуальный вызов.** Каждый operator() — это косвенный вызов через vtable (не инлайнится).
3. **Копирование.** При копировании std::function копируется и хранимый callable.

Когда это важно:

```

1 // Плохо: std::function в hot loop
2 void process(const std::vector<int>& data,
3             std::function<int(int)> transform) { // type erasure
4     for (int x : data)
5         result.push_back(transform(x)); //
6         косвенныйвызовнакаждойитерации
7 }
8 // Лучше: шаблонныйпараметр (zero-cost)
9 template<typename F>
10 void process(const std::vector<int>& data, F transform) {
11     for (int x : data)
12         result.push_back(transform(x)); // прямойвызов ,
13         инлайнится
14 }

```

Правило: используйте std::function, когда вам **нужно хранить** callable (в поле класса, в контейнере, в callback). Для **передачи** callable в функцию предпочитайте шаблоны.

32.7.4. Хранение указателей на методы

```

1 struct Greeter {
2     std::string name;
3     void hello() const {
4         std::cout << "Hello, " << name << "!\n";
5     }
6     void hello_n(int n) const {
7         for (int i = 0; i < n; ++i)

```

```

8         std::cout << "Hello, " << name << "!\n";
9     }
10 };
11
12 // Метод без аргументов : первый аргумент -- объект
13 std::function<void(const Greeter&)> f = &Greeter::hello;
14 Greeter g{"Alice"};
15 f(g); // "Hello, Alice!"
16
17 // Метод с аргументами :
18 std::function<void(const Greeter&, int)> f2 = &Greeter::hello_n;
19 f2(g, 3); // три раза "Hello, Alice!"
20
21 // Привязка объекта через лямбду :
22 Greeter bob{"Bob"};
23 std::function<void()> f3 = [&bob]() { bob.hello(); };
24 f3(); // "Hello, Bob!"

```

32.7.5. std::move_only_function (C++23)

std::function требует, чтобы хранимый callable был **копируемым**. Это не работает с unique_ptr в захвате:

```

1 auto ptr = std::make_unique<int>(42);
2 // std::function<int()> f = [p = std::move(ptr)]() { return *p; };
3 // Ошибка: лямбда не копируемая !

```

C++23 вводит std::move_only_function — аналог, который перемещается, но не копируется:

```

1 #include <functional>
2
3 auto ptr = std::make_unique<int>(42);
4 std::move_only_function<int()> f = [p = std::move(ptr)]() { return
    *p; };
5 std::cout << f() << "\n"; // 42
6 // auto g = f; // Ошибка: нельзя копировать
7 auto g = std::move(f); // OK: перемещение

```

32.7.6. Пример: система событий (observer pattern)

```

1 #include <functional>
2 #include <vector>
3 #include <string>
4
5 class EventBus {
6     std::unordered_map<std::string,
7         std::vector<std::function<void(const std::string&>>>
8         handlers_;
9 public:

```

```

9     void subscribe(const std::string& event,
10                   std::function<void(const std::string&)> handler
11                   ) {
12         handlers_[event].push_back(std::move(handler));
13     }
14
15     void emit(const std::string& event, const std::string& data) {
16         if (auto it = handlers_.find(event); it != handlers_.end()) {
17             for (auto& handler : it->second) {
18                 handler(data);
19             }
20         }
21     };
22
23     int main() {
24         EventBus bus;
25
26         bus.subscribe("click", [](const std::string& data) {
27             std::cout << "Button clicked: " << data << "\n";
28         });
29
30         bus.subscribe("click", [](const std::string& data) {
31             std::cout << "Logging click: " << data << "\n";
32         });
33
34         bus.subscribe("keypress", [](const std::string& data) {
35             std::cout << "Key pressed: " << data << "\n";
36         });
37
38         bus.emit("click", "OK");
39         // Button clicked: OK
40         // Logging click: OK
41
42         bus.emit("keypress", "Enter");
43         // Key pressed: Enter
44     }

```

32.8. Итого

Механизм	Когда использовать
Указатель на функцию	Взаимодействие с C API; compile-time dispatch таблицы
Указатель на метод	Обобщённый вызов методов; reflection-подобные паттерны
Указатель на поле	Обобщённый доступ к полям; сортировка по полю
Лямбда + auto/шаблон	Передача логики в алгоритмы; zero-cost абстракция
<code>std::function</code>	Хранение callback'ов; runtime полиморфизм callable
Type erasure (свой)	Когда нужен runtime полиморфизм без навязывания базового класса

33. Move-семантика в C++: value categories, rvalue references, perfect forwarding

33.1. Value Categories (Категории значений)

Главное заблуждение относительно move-семантики заключается в том, что категории **lvalue** и **rvalue** — это свойства объектов. На самом деле — это **синтаксические свойства выражений** (expressions). Один и тот же объект в разных строчках кода может быть представлен выражениями разных категорий.

Иерархия категорий выглядит следующим образом:

Группа	Категории
expression	glvalue, rvalue
glvalue	lvalue, xvalue
rvalue	xvalue, prvalue

33.1.1. Lvalue (Long-lived)

Выражения, имеющие имя, адрес в памяти и устойчивую сущность:

- Любой идентификатор (имя переменной) — всегда **lvalue**.
- Результаты присваивания (`=`, `+=`), префиксный инкремент/декремент (`++x`), результат разыменования (`*p`).
- Результат функции, возвращающей **lvalue**-ссылку (`T&`).

33.1.2. Rvalue (Short-lived)

Временные значения, которые будут уничтожены сразу после использования:

- Литералы (кроме строковых), результат арифметических операций ($a + b$), постфиксный инкремент/декремент ($x++$).
- Результат функции, возвращающей тип по значению (T) или `rvalue`-ссылку ($T\&\&$).
- Результат приведения типа (**Cast**) к типу, не являющемуся `lvalue`-ссылкой.

33.1.3. Тернарный оператор и оператор запятая

Результат тернарного оператора является `lvalue` только если оба его подвыражения — `lvalue`. Оператор запятая принимает категорию своего **правого** операнда.

33.2. RValue Reference (Rvalue-ссылки)

Это новый вид ссылок ($T\&\&$), введённый для реализации семантики перемещения.

- **Инициализация:** `rvalue`-ссылку можно инициализировать только `rvalue`-выражениями. Попытка привязать её к `lvalue` вызовет ошибку компиляции.
- **Свойство имени:** если `rvalue`-ссылке дано имя (например, `int&& x = 5;`), то само выражение `x` является `lvalue`, так как это идентификатор.
- **Константность:** нельзя привязать неконстантную ссылку к константному объекту — обычные правила константности сохраняются.

```
1 int&& r = 42;      // OK: привязываемк rvalue
2 // int&& r2 = r; // Ошибка: r — lvalue y( негоестьямя )
3
4 const int&& cr = 10; // OK
5 // int&& r3 = cr;   // Ошибка: нельзяотбросить const
```

33.3. Move Semantics (Семантика перемещения)

Семантика перемещения позволяет «забирать» ресурсы у временных объектов вместо их копирования, что критически важно для производительности.

33.3.1. std::move

`std::move` **ничего не перемещает физически** и не генерирует процессорных инструкций. Это инструмент для компилятора, преобразующий `lvalue` в `rvalue`.

Реализация `std::move`:

```
1 template<typename T>
2 constexpr std::remove_reference_t<T>&& move(T&& t) noexcept {
3     return static_cast<std::remove_reference_t<T>&&>(t);
4 }
```

- Принимает аргумент по **универсальной ссылке** (`T&&`), чтобы работать с любыми категориями значений.
- Использует `std::remove_reference_t<T>`, чтобы получить чистый тип без ссылок.
- Выполняет `static_cast` к типу `чистый_тип&&`.

Результат: выражение `std::move(x)` всегда является `rvalue`, что заставляет компилятор выбирать **перемещающий конструктор** или **оператор присваивания перемещением**.

```
1 std::vector<int> a = {1, 2, 3};
2 std::vector<int> b = std::move(a); // a теперь в "valid but
   unspecified" состоянии
```

33.4. Perfect Forwarding (Идеальная передача)

Проблема идеальной передачи возникает в функциях-обёртках (например, `emplace_back`), которые должны передать аргументы дальше, сохранив их оригинальную категорию значения.

33.4.1. Универсальные ссылки (Universal References)

Если параметр шаблона функции имеет вид `T&&`, он может принимать и `lvalue`, и `rvalue`:

```
1 template<typename T>
2 void wrapper(T&& arg) {
3     // arg может быть lvalue-ссылкой или rvalue-ссылкой в зависимости от T
4 }
```

33.4.2. Сворачивание ссылок (Reference Collapsing)

При выводе типов шаблонов компилятор «схлопывает» ссылки по следующим правилам:

Т выводится как	Параметр	Результат
X&	T&&	X& & & → X&
X&	T&&	X& && → X&
X&&	T&&	X&& & → X&
X&&	T&&	X&& && → X&&

Правило одной строкой: **только два rvalue дают rvalue**, любое & «заражает» результат.

33.4.3. std::forward

std::forward необходим, чтобы внутри функции превратить аргумент обратно в rvalue, если он изначально был таковым (поскольку внутри функции у него есть имя и он стал lvalue):

```
1 template<typename T>
2 void wrapper(T&& arg) {
3     target(std::forward<T>(arg)); // сохраняем категорию значения
4 }
```

```
1 // Реализация std::forward упрощённо():
2 template<typename T>
3 T&& forward(std::remove_reference_t<T>& t) noexcept {
4     return static_cast<T&&>(t);
5 }
```

33.5. Variadic Templates (Шаблоны с переменным количеством аргументов)

В контексте perfect forwarding шаблоны с переменным числом аргументов позволяют принять **произвольное число аргументов любых категорий** и передать их дальше без потери информации о категориях:

```
1 template<typename... Args>
2 void emplace_back_impl(Args&&... args) {
3     // каждый аргумент в пакете может иметь свою категорию значения
4     construct(std::forward<Args>(args)...);
5 }
```

- Args&&... args — пакет универсальных ссылок; каждый элемент независимо выводит свою категорию.
- std::forward<Args>(args)... — раскрывает пакет, применяя forward к каждому элементу отдельно.
- Такой подход используется в std::vector::emplace_back, std::make_unique, std::make_shared и других фабричных функциях стандартной библиотеки.

```

1 // Пример: своя фабрика объектов      perfect forwarding
2 template<typename T, typename... Args>
3 T* create(Args&&... args) {
4     return new T(std::forward<Args>(args)...);
5 }
6
7 struct Point { int x, y; };
8 auto* p = create<Point>(1, 2); // конструируется напрямую ,
    без лишних копий

```

34. Вариативные шаблоны (variadic templates)

34.1. Что это

Вариативный шаблон принимает **произвольное число** параметров любых типов. Это позволило реализовать в библиотеке printf-подобные функции, `std::tuple`, `std::make_unique` и др.

```

1 template<typename... Args>          // Args -- пакет параметров типов
2 void print(Args... args);          // args -- пакет аргументов значений
3
4 print(1, 2.5, "hello");             // Args = {int, double, const char
    *}

```

34.1.1. Пакеты параметров (parameter packs)

- `typename... Args` — пакет шаблонных параметров.
- `Args... args` — пакет функциональных параметров.
- `args...` — **развёртка (expansion)** пакета.
- `sizeof...(Args)` — число элементов в пакете.

34.2. Рекурсивная обработка пакета (классический способ)

До C++17 пакет обрабатывали рекурсией: «голова + хвост» с базовым случаем:

```

1 void print() {} // базовый случай : пустой пакет
2
3 template<typename Head, typename... Tail>
4 void print(Head h, Tail... tail) {
5     std::cout << h; // обработали первый
6     if (sizeof...(tail) > 0) std::cout << ", ";
7     print(tail...); // рекурсия по остатку
8 }
9 print(1, 2, 3); // 1, 2, 3

```

34.3. Fold-выражения (C++17)

Свёртки (fold expressions) применяют оператор ко всему пакету без рекурсии:

```
1 template<typename... Args>
2 auto sum(Args... args) {
3     return (args + ...);    // унарная правая свёртка : a1 + (a2
4                             + (a3 + ...))
5 }
6 sum(1, 2, 3, 4);    // 10
```

Четыре формы свёрток:

Форма	Раскрытие
(pack op ...)	унарная правая: $e_1 \text{ op } (\dots \text{ op } e_n)$
(... op pack)	унарная левая: $((e_1 \text{ op } e_2) \text{ op } \dots)$
(pack op ... op init)	бинарная правая (с начальным значением)
(init op ... op pack)	бинарная левая

```
1 template<typename... Args>
2 void print(Args... args) {
3     ((std::cout << args << ' '), ...);    // свёртка по запятой
4 }
5 template<typename... Args>
6 bool all_true(Args... args) {
7     return (args && ...);    // все истинны
8 }
```

34.4. Perfect forwarding в вариативных шаблонах

Универсальная фабрика передаёт аргументы дальше **без потери** категории (lvalue/rvalue) через `std::forward`:

```
1 template<typename T, typename... Args>
2 std::unique_ptr<T> make(Args&&... args) {
3     return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
4 }
```

`Args&&...` — пакет **forwarding-ссылок**; `std::forward<Args>(args)...` возвращает пакет, сохраняя категорию каждого аргумента.

34.5. Пример: типобезопасный кортеж и доступ по индексу

```
1 template<typename... Ts>
2 struct Tuple {};    // упрощённо; в стандарте -- std::tuple
3
4 std::tuple<int, std::string, double> t{1, "hi", 3.14};
5 auto x = std::get<1>(t);    // "hi"
6 auto [a, b, c] = t;    // structured bindings
```

34.6. Развёртка в разных контекстах

```
1 template<typename... Args>
2 void demo(Args... args) {
3     f(args...);           // f(a1, a2, a3)
4     g(h(args)...);        // g(h(a1), h(a2), h(a3))
5     int arr[] = {args...}; // инициализация массива
6     std::vector v{args...};
7 }
```

35. Метапрограммирование: constexpr, метафункции, SFINAE, концепты

35.1. Что такое метапрограммирование

Шаблонное метапрограммирование (TMP) — вычисления, выполняемые на этапе компиляции. Результат — типы и константы, «зашитые» в программу, без затрат в runtime.

35.2. constexpr — вычисления на этапе компиляции

constexpr-функция может быть вычислена компилятором, если её аргументы известны на этапе компиляции:

```
1 constexpr int factorial(int n) {
2     return (n <= 1) ? 1 : n * factorial(n - 1);
3 }
4 constexpr int f5 = factorial(5); // 120, вычислено компилятором
5 int arr[factorial(4)];           // 24 -- можно как размер массива
```

- const — неизменяемость (может вычисляться в runtime).
- constexpr — **может** вычисляться на этапе компиляции.
- consteval (C++20) — **обязан** вычисляться на этапе компиляции (immediate function).
- constexpr (C++20) — гарантирует статическую инициализацию.

35.2.1. if constexpr (C++17)

Ветвление на этапе компиляции: «неподходящая» ветка **не компилируется**:

```
1 template<typename T>
2 auto to_string(T value) {
3     if constexpr (std::is_arithmetic_v<T>)
4         return std::to_string(value); // только для чисел
5     else
6         return value;                 // для остального
7 }
```

35.3. Метафункции и type traits

Метафункция — шаблон, «возвращающий» тип или значение через вложенный `::type` или `::value`. Стандартные метафункции живут в `<type_traits>`.

```
1 #include <type_traits>
2 std::is_integral<int>::value;           // true
3 std::is_integral_v<int>;                // true суффикс( _v --
    значение)
4 std::remove_const<const int>::type;     // int
5 std::remove_const_t<const int>;         // int суффикс( _t -- тип)
6 std::is_same_v<int, int32_t>;           // true/false
    в зависимости от платформы
```

35.3.1. Своя метафункция

```
1 // вернуть "" тип: убрать указатель
2 template<typename T> struct remove_ptr    { using type = T; };
3 template<typename T> struct remove_ptr<T*> { using type = T; };
4 // специализация
5 template<typename T> using remove_ptr_t = typename remove_ptr<T>::
    type;
6
7 // вернуть "" значение: длина массива на этапе компиляции
8 template<typename T, size_t N>
9 constexpr size_t array_size(T(&)[N]) { return N; }
```

35.4. SFINAE

SFINAE (Substitution Failure Is Not An Error): если при подстановке шаблонных аргументов сигнатура оказывается некорректной, эта перегрузка **молча исключается** из набора кандидатов, а не вызывает ошибку.

```
1 #include <type_traits>
2 // версия только для целочисленных T
3 template<typename T,
4         std::enable_if_t<std::is_integral_v<T>, int> = 0>
5 void process(T x) { std::cout << "integral\n"; }
6
7 // версия только для вещественных T
8 template<typename T,
9         std::enable_if_t<std::is_floating_point_v<T>, int> = 0>
10 void process(T x) { std::cout << "floating\n"; }
11
12 process(42);           // "integral"
13 process(3.14);         // "floating"
```

35.4.1. Идиома detection (проверка наличия члена)

```

1 // есть ли у T метод .size()?
2 template<typename T, typename = void>
3 struct has_size : std::false_type {};
4
5 template<typename T>
6 struct has_size<T, std::void_t<decltype(std::declval<T>().size())
7     >>
8     : std::true_type {};
9
10 has_size<std::vector<int>>>::value; // true
11 has_size<int>>::value;             // false

```

35.5. Концепты (C++20)

Концепты — именованные ограничения на типы. Они заменяют громоздкий SFINAE, дают **читаемые ошибки** и явно документируют требования.

```

1 #include <concepts>
2 template<typename T>
3 concept Numeric = std::is_arithmetic_v<T>; // своё ограничение
4
5 template<Numeric T> // T обязан быть числом
6 T square(T x) { return x * x; }
7
8 square(5); // OK
9 // square("s"); // понятная ошибка: "s" не удовлетворяет Numeric

```

35.5.1. Способы применить ограничение

```

1 template<std::integral T> void f(T); //
   на параметр шаблона
2 template<typename T> requires std::integral<T> // requires-clause
3 void g(T);
4 void h(std::integral auto x); //
   сокращённый синтаксис

```

35.5.2. requires-выражение — задаём свой концепт

```

1 template<typename T>
2 concept Container = requires(T c) {
3     c.begin(); // должен быть begin()
4     c.end();   // должен быть end()
5     c.size();  // должен быть size()
6     typename T::value_type; // должен быть вложенный тип
7     value_type
8 };

```

35.6. Зачем всё это: библиотечный код

Метапрограммирование — основа стандартной библиотеки: `std::vector` выбирает оптимальные операции по traits типа, алгоритмы диспетчеризуются по категории итератора, `std::variant`/`std::tuple` построены на вариативных шаблонах. В прикладном коде стоит предпочитать `if constexpr` и концепты как наиболее читаемые инструменты.